

PERP CITY · SECURITY REVIEW

# Perp City Protocol Review.

A security review of Protocol Review.

---

| CLIENT    | PROJECT         | AUDITORS | AUDIT WINDOW              | FINAL REPORT   |
|-----------|-----------------|----------|---------------------------|----------------|
| Perp City | Protocol Review | Guardian | May 27 – July 31,<br>2026 | August 7, 2026 |

§ 00

# Contents.

263 pages — overview, confidence ranking, scope, 203 findings, disclaimer, and about Guardian.

|      |                                    |        |
|------|------------------------------------|--------|
| § 01 | Overview                           | P. 03  |
| § 02 | Confidence Ranking                 | P. 04  |
| § 03 | Scope & Classifications            | P. 05  |
|      | Files in Scope                     | P. 06  |
| § 04 | Findings                           | P. 08  |
|      | Main Review · 46 findings          | P. 24  |
|      | Remediation Review 1 · 62 findings | P. 84  |
|      | Remediation Review 2 · 95 findings | P. 160 |
| § 05 | Disclaimer                         | P. 262 |
| § 06 | About Guardian                     | P. 263 |

§ 01

# Overview.

Guardian reviewed Protocol Review across 3 review rounds. The final report includes 4 Critical, 32 High, 56 Medium, 49 Low, 62 Informational findings.

|                        |                                                                                     |
|------------------------|-------------------------------------------------------------------------------------|
| AUDIT FIRM             | REPOSITORY                                                                          |
| Guardian               | StrobeLabs/beacons, StrobeLabs/perpcity-contracts                                   |
| CLIENT                 | METHODOLOGY                                                                         |
| Perp City              | Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2 |
| PROJECT                | MAIN REVIEW COMMIT                                                                  |
| Protocol Review        | 0987d0030b42                                                                        |
| PREPARED BY            | REMEDATION REVIEW 1 COMMIT                                                          |
| Guardian               | 0987d0030b42                                                                        |
| AUDIT WINDOW           | REMEDATION REVIEW 2 COMMIT                                                          |
| May 27 – July 31, 2026 | 01570f5ff0f3                                                                        |
| FINAL REPORT           |                                                                                     |
| August 7, 2026         |                                                                                     |

## 1.1 Confidence Ranking

1 / 5

Very Low Confidence

Guardian assigns a Confidence Ranking of 1 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

|                     |                |                     |                 |                      |
|---------------------|----------------|---------------------|-----------------|----------------------|
| 1                   | 2              | 3                   | 4               | 5                    |
| VERY LOW CONFIDENCE | LOW CONFIDENCE | MODERATE CONFIDENCE | HIGH CONFIDENCE | VERY HIGH CONFIDENCE |

## 1.2 Findings at a Glance

| SEVERITY                     | TOTAL | PENDING | ACK | PARTIAL | RESOLVED |
|------------------------------|-------|---------|-----|---------|----------|
| <span>■</span> Critical      | 4     | 0       | 2   | 0       | 2        |
| <span>■</span> High          | 32    | 0       | 19  | 0       | 13       |
| <span>■</span> Medium        | 56    | 0       | 29  | 0       | 27       |
| <span>■</span> Low           | 49    | 0       | 26  | 0       | 23       |
| <span>■</span> Informational | 62    | 0       | 45  | 0       | 17       |
| All severities               | 203   | 0       | 121 | 0       | 82       |

## § 02

# Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

| CONFIDENCE RANKING                              | DEFINITION AND RECOMMENDATION                                                                                                                                                                                               | RISK PROFILE                                                                              |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>5 Very High Confidence</b>                   | Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit. | 0 High/Critical findings and few Low/Medium severity findings.                            |
| <b>4 High Confidence</b>                        | Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.    | 0-1 High/Critical findings per engagement week and little to no Medium severity findings. |
| <b>3 Moderate Confidence</b>                    | Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.                                                            | Some High/Critical findings, with various Medium severity findings.                       |
| <b>2 Low Confidence</b>                         | Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.                                                                     | Over 6 High and Critical findings across the engagement.                                  |
| <b>1 Very Low Confidence</b><br><b>ASSIGNED</b> | The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.                                                                           | Over 12 High and Critical findings across the engagement.                                 |

## § 03

# Scope & Classifications.

Guardian reviewed Protocol Review in `StrobeLabs/beacons`, `StrobeLabs/perpcity-contracts`. The scope includes 2,624 source lines across 67 files.

## 3.1 Severity Matrix

|            | IMP · HIGH | IMP · MED | IMP · LOW |
|------------|------------|-----------|-----------|
| LIK · HIGH | Critical   | High      | Medium    |
| LIK · MED  | High       | Medium    | Low       |
| LIK · LOW  | Medium     | Low       | Low       |

### IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

### IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

### IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

### LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

### LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

### LIKELIHOOD · LOW

Unlikely to ever occur in production.

## Files in Scope 1/2.

| FILE                                                          | NSLOC | TOTAL |
|---------------------------------------------------------------|-------|-------|
| src/BeaconRegistry.sol                                        | 21    | 34    |
| src/libraries/BindingLib.sol                                  | 18    | 30    |
| src/libraries/Constants.sol                                   | 11    | 19    |
| src/libraries/TwAvg.sol                                       | 104   | 233   |
| src/verifiers/ECDSA/ECDSAVerifier.sol                         | 37    | 68    |
| src/verifiers/ECDSA/ECDSAVerifierFactory.sol                  | 7     | 15    |
| src/core/standalone/StandaloneBeacon.sol                      | 67    | 95    |
| src/core/standalone/StandaloneBeaconFactory.sol               | 11    | 23    |
| src/core/identity/IdentityBeacon.sol                          | 40    | 63    |
| src/core/identity/IdentityBeaconFactory.sol                   | 8     | 17    |
| src/core/group/GroupManager.sol                               | 59    | 76    |
| src/core/group/GroupManagerFactory.sol                        | 10    | 21    |
| src/core/group/MemberBeacon.sol                               | 36    | 57    |
| src/core/composite/CompositeBeacon.sol                        | 44    | 67    |
| src/core/composite/CompositeBeaconFactory.sol                 | 9     | 18    |
| src/core/base/CallerBound.sol                                 | 13    | 22    |
| src/core/factories/IdentityFactory.sol                        | 9     | 18    |
| src/core/factories/LBCGBMFactory.sol                          | 15    | 34    |
| src/core/factories/WeightedSumCompositeFactory.sol            | 10    | 19    |
| src/core/standalone/transforms/Bounded.sol                    | 36    | 63    |
| src/core/standalone/transforms/Unbounded.sol                  | 24    | 41    |
| src/core/standalone/preprocessors/Argmax.sol                  | 19    | 30    |
| src/core/standalone/preprocessors/Identity.sol                | 13    | 24    |
| src/core/standalone/preprocessors/TernaryToBinary.sol         | 23    | 48    |
| src/core/standalone/preprocessors/Threshold.sol               | 14    | 28    |
| src/core/standalone/basefns/CGBM.sol                          | 60    | 109   |
| src/core/standalone/basefns/DGBM.sol                          | 40    | 73    |
| src/core/group/transforms/GMNormalize.sol                     | 53    | 76    |
| src/core/group/transforms/Softmax.sol                         | 62    | 91    |
| src/core/group/groupfns/ContinuousAllocation.sol              | 54    | 92    |
| src/core/group/groupfns/DiscreteAllocation.sol                | 63    | 105   |
| src/core/group/groupfns/Dominance.sol                         | 42    | 76    |
| src/core/group/groupfns/RelativeDominance.sol                 | 66    | 110   |
| src/core/composite/composers/WeightedSum.sol                  | 19    | 34    |
| src/core/standalone/transforms/factories/BoundedFactory.sol   | 7     | 17    |
| src/core/standalone/transforms/factories/UnboundedFactory.sol | 7     | 15    |

## Files in Scope 2/2.

| FILE                                                                        | NSLOC | TOTAL |
|-----------------------------------------------------------------------------|-------|-------|
| src/core/standalone/preprocessors/factories/ArgmaxFactory.sol               | 7     | 15    |
| src/core/standalone/preprocessors/factories/IdentityPreprocessorFactory.sol | 7     | 15    |
| src/core/standalone/preprocessors/factories/TernaryToBinaryFactory.sol      | 7     | 16    |
| src/core/standalone/preprocessors/factories/ThresholdFactory.sol            | 7     | 16    |
| src/core/standalone/preprocessors/base/BasePreprocessor.sol                 | 20    | 30    |
| src/core/standalone/basefns/factories/CGBMFactory.sol                       | 7     | 19    |
| src/core/standalone/basefns/factories/DGBMFactory.sol                       | 7     | 18    |
| src/core/group/transforms/factories/GMNormalizeFactory.sol                  | 7     | 15    |
| src/core/group/transforms/factories/SoftmaxFactory.sol                      | 7     | 16    |
| src/core/group/groupfns/factories/ContinuousAllocationFactory.sol           | 7     | 18    |
| src/core/group/groupfns/factories/DiscreteAllocationFactory.sol             | 7     | 18    |
| src/core/group/groupfns/factories/DominanceFactory.sol                      | 7     | 18    |
| src/core/group/groupfns/factories/RelativeDominanceFactory.sol              | 7     | 20    |
| src/core/composite/composers/factories/WeightedSumFactory.sol               | 7     | 15    |
| src/core/group/transforms/base/BaseGroupTransform.sol                       | 11    | 19    |
| src/AccountingToken.sol                                                     | 32    | 66    |
| src/ModuleRegistry.sol                                                      | 13    | 18    |
| src/Perp.sol                                                                | 290   | 499   |
| src/PerpFactory.sol                                                         | 70    | 91    |
| src/ProtocolFeeManager.sol                                                  | 22    | 36    |
| src/modules/Fees.sol                                                        | 18    | 34    |
| src/modules/Funding.sol                                                     | 11    | 18    |
| src/modules/MarginRatios.sol                                                | 16    | 30    |
| src/modules/PriceImpact.sol                                                 | 14    | 21    |
| src/modules/Pricing.sol                                                     | 11    | 18    |
| src/libraries/Constants.sol                                                 | 25    | 40    |
| src/libraries/Errors.sol                                                    | 25    | 27    |
| src/libraries/Events.sol                                                    | 63    | 72    |
| src/libraries/PerpLogic.sol                                                 | 596   | 770   |
| src/libraries/SharedStructs.sol                                             | 150   | 174   |
| src/libraries/SignedFixedPointMathLib.sol                                   | 15    | 31    |

## § 04

# Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

| ID          | TITLE                                                    | CATEGORY      | SEVERITY | STATUS       | PAGE  |
|-------------|----------------------------------------------------------|---------------|----------|--------------|-------|
| MAIN REVIEW |                                                          |               |          |              |       |
| H-01        | TWAP accounting attributes elapsed time to the new index | Math          | HIGH     | RESOLVED     | p. 24 |
| H-02        | CompositeBeacon TWAP Can Be Manipulated                  | Gaming        | HIGH     | RESOLVED     | p. 25 |
| H-03        | refreshRatesAndEmas uses post-swap price                 | Logical Error | HIGH     | RESOLVED     | p. 27 |
| H-04        | Post-Swap Donation Misallocates LP Fees                  | Logical Error | HIGH     | RESOLVED     | p. 29 |
| H-05        | Validations can prevent AMM/index convergence            | Logical Error | HIGH     | ACKNOWLEDGED | p. 31 |
| H-06        | adjustTaker Allows Expo Below Init Margin                | Validation    | HIGH     | RESOLVED     | p. 33 |
| H-07        | Insurance Fund Charged For Solvent Positions             | Logical Error | HIGH     | RESOLVED     | p. 34 |
| H-08        | ContinuousAllocation Ignores Relevance                   | Math          | HIGH     | RESOLVED     | p. 35 |
| H-09        | Dominance Double-Discounts Classes                       | Math          | HIGH     | RESOLVED     | p. 37 |
| H-10        | Perp Markets Accept Stale Beacon Prices                  | Oracle        | HIGH     | RESOLVED     | p. 38 |
| H-11        | Bad-Debt Socialization Uses Inflated totalMargin         | Logical Error | HIGH     | ACKNOWLEDGED | p. 39 |
| H-12        | DAlloc Amplifies Low-Relevance Markets                   | Math          | HIGH     | RESOLVED     | p. 41 |

## Findings continued.

| ID   | TITLE                                                                               | CATEGORY      | SEVERITY | STATUS       | PAGE  |
|------|-------------------------------------------------------------------------------------|---------------|----------|--------------|-------|
| M-01 | CGBM reverts on exact neutral predictions                                           | Math          | MEDIUM   | RESOLVED     | p. 43 |
| M-02 | CGBM reverts for one-sided prediction streak                                        | Math          | MEDIUM   | RESOLVED     | p. 44 |
| M-03 | ECDSA replay protection is signature-based and nonces are not enforced              | Signatures    | MEDIUM   | RESOLVED     | p. 45 |
| M-04 | Public one-time binding lets attackers permanently grief unbound mutable components | Logical Error | MEDIUM   | RESOLVED     | p. 46 |
| M-05 | Missing touch On Module Updates                                                     | Configuration | MEDIUM   | RESOLVED     | p. 47 |
| M-06 | Missing Check In Liquidation Flow                                                   | Validation    | MEDIUM   | RESOLVED     | p. 48 |
| M-07 | Fee withdrawals bypass bad-debt socialization                                       | Logical Error | MEDIUM   | ACKNOWLEDGED | p. 49 |
| M-08 | Liquidation Fee Excluded From Eligibility                                           | Rewards       | MEDIUM   | RESOLVED     | p. 50 |
| M-09 | Irrelevant CAlloc Reverts                                                           | Math          | MEDIUM   | RESOLVED     | p. 51 |
| M-10 | GMNormalize Clamp Breaks GM Invariant                                               | Math          | MEDIUM   | RESOLVED     | p. 52 |
| M-11 | Blacklisted Users Are Not Liquidatable                                              | DoS           | MEDIUM   | RESOLVED     | p. 54 |
| M-12 | Predicted UniV4 Pools Can Be Pre-Initialized                                        | DoS           | MEDIUM   | RESOLVED     | p. 55 |
| M-13 | Same-Block Beacon EMA Desync                                                        | Oracle        | MEDIUM   | RESOLVED     | p. 56 |
| M-14 | DAlloc Irrelevant Wins Mutate State                                                 | Math          | MEDIUM   | RESOLVED     | p. 57 |
| M-15 | setBeacon() Does Not Accrue Before Switching                                        | Oracle        | MEDIUM   | RESOLVED     | p. 59 |
| M-16 | Liquidation Fees Use Post-Liquidation Price                                         | Math          | MEDIUM   | RESOLVED     | p. 60 |

## Findings continued.

| ID   | TITLE                                                       | CATEGORY      | SEVERITY | STATUS       | PAGE  |
|------|-------------------------------------------------------------|---------------|----------|--------------|-------|
| M-17 | Unnecessary Maker Position Close DoS                        | DoS           | MEDIUM   | RESOLVED     | p. 61 |
| M-18 | Users can avoid socialized losses                           | Gaming        | MEDIUM   | RESOLVED     | p. 62 |
| M-19 | Maker liquidation can get stuck                             | Validation    | MEDIUM   | ACKNOWLEDGED | p. 64 |
| M-20 | Non-Standard EIP-712 Array Hash                             | Signatures    | MEDIUM   | RESOLVED     | p. 66 |
| L-01 | CGBM can accepts degenerate parameter sets                  | Validation    | LOW      | RESOLVED     | p. 67 |
| L-02 | DGBM can accept degenerate parameter sets                   | Validation    | LOW      | RESOLVED     | p. 68 |
| L-03 | Hardcoded chain-specific addresses create deployment risk   | Configuration | LOW      | RESOLVED     | p. 69 |
| L-05 | Timelock defaults to zero, allowing immediate admin changes | Configuration | LOW      | RESOLVED     | p. 70 |
| L-06 | Module Registry Is Not Enforced                             | Configuration | LOW      | RESOLVED     | p. 71 |
| L-07 | Signed Fixed-Point Rounds Toward Zero                       | Math          | LOW      | RESOLVED     | p. 72 |
| L-08 | Health Check Adds One To Position Value                     | Math          | LOW      | RESOLVED     | p. 73 |
| L-09 | Liquidation Fees Are Socialized During Bad Debt             | Rewards       | LOW      | RESOLVED     | p. 74 |
| L-10 | WeightedSum accumulates rounding error per term             | Math          | LOW      | RESOLVED     | p. 75 |
| L-11 | DAlloc Ties Bias Lower Classes                              | Math          | LOW      | RESOLVED     | p. 77 |
| L-12 | Beacon Index Is Downcast Without Bounds Check               | Validation    | LOW      | RESOLVED     | p. 79 |

## Findings continued.

| ID   | TITLE                                  | CATEGORY         | SEVERITY      | STATUS   | PAGE  |
|------|----------------------------------------|------------------|---------------|----------|-------|
| L-13 | Unbounded Transform Silently Saturates | Math             | LOW           | RESOLVED | p. 80 |
| I-01 | Unused Measurement Scale               | Superfluous Code | INFORMATIONAL | RESOLVED | p. 82 |
| I-02 | Mixed Composite Beacon Updates         | Warning          | INFORMATIONAL | RESOLVED | p. 83 |

### REMEDIATION REVIEW 1

|      |                                                                                               |               |          |              |       |
|------|-----------------------------------------------------------------------------------------------|---------------|----------|--------------|-------|
| C-01 | Zero-For-One Tick Replay Can Skip Current Boundary                                            | Logical Error | CRITICAL | RESOLVED     | p. 84 |
| C-02 | Dust maker liquidity hides residual exposure from OI and margin checks                        | Logical Error | CRITICAL | RESOLVED     | p. 86 |
| H-01 | Spot-Mark Backstops Can Seize Healthy Positions                                               | Gaming        | HIGH     | ACKNOWLEDGED | p. 87 |
| H-02 | Insurance Spend Skews Solvency                                                                | Logical Error | HIGH     | RESOLVED     | p. 89 |
| H-03 | Taker Liquidation Sandwich                                                                    | MEV           | HIGH     | ACKNOWLEDGED | p. 90 |
| H-04 | Transient-price maker cycles under-account capacity below executable liquidity                | Logical Error | HIGH     | ACKNOWLEDGED | p. 91 |
| H-05 | Uninsured bad debt omitted from socialization claim base                                      | Logical Error | HIGH     | RESOLVED     | p. 92 |
| H-06 | Full maker liquidation cannot book bad debt when residual perp remains                        | Logical Error | HIGH     | RESOLVED     | p. 93 |
| H-07 | Stripping in-range liquidity while OOR capacity remains freezes underwater taker liquidations | Logical Error | HIGH     | ACKNOWLEDGED | p. 94 |
| H-08 | Bounded Drift Can Point Outward                                                               | Math          | HIGH     | ACKNOWLEDGED | p. 95 |
| H-09 | Relative Dominance Stationary Bias                                                            | Math          | HIGH     | ACKNOWLEDGED | p. 97 |

## Findings continued.

| ID   | TITLE                                                            | CATEGORY      | SEVERITY | STATUS       | PAGE   |
|------|------------------------------------------------------------------|---------------|----------|--------------|--------|
| H-10 | CGBM Misses Its Volatility Budget                                | Math          | HIGH     | ACKNOWLEDGED | p. 98  |
| H-11 | Ternary Gate Differs From Specification                          | Logical Error | HIGH     | ACKNOWLEDGED | p. 101 |
| M-01 | Winning Takers Can Exit Before Maker Losses Settle               | Logical Error | MEDIUM   | ACKNOWLEDGED | p. 102 |
| M-02 | Bad Debt Ignored In Health Checks                                | Logical Error | MEDIUM   | ACKNOWLEDGED | p. 104 |
| M-03 | Insolvent Liquidations Pay Fees                                  | Logical Error | MEDIUM   | ACKNOWLEDGED | p. 105 |
| M-04 | Module Registry Approvals Are Append-Only                        | Configuration | MEDIUM   | ACKNOWLEDGED | p. 106 |
| M-05 | Taker Fees Wrap Total Margin                                     | Logical Error | MEDIUM   | RESOLVED     | p. 107 |
| M-06 | Lazy Funding Wraps Total Margin                                  | Logical Error | MEDIUM   | RESOLVED     | p. 108 |
| M-07 | Dust Signals Escape Variance Tracking                            | Math          | MEDIUM   | RESOLVED     | p. 110 |
| M-08 | Softmax Creates Allocation Drift                                 | Math          | MEDIUM   | ACKNOWLEDGED | p. 111 |
| M-09 | CGBM Price Drift After Exponentiation                            | Math          | MEDIUM   | ACKNOWLEDGED | p. 113 |
| M-10 | Neutral Ticks Amplify CGBM Moves                                 | Math          | MEDIUM   | ACKNOWLEDGED | p. 115 |
| M-11 | Confidence Ordering Can Invert                                   | Math          | MEDIUM   | RESOLVED     | p. 117 |
| M-12 | Zero-liquidity gap poisons AMM EMA to force liquidation          | Informational | MEDIUM   | ACKNOWLEDGED | p. 118 |
| M-13 | Maker conversion checks quote debt before valuing residual perps | DoS           | MEDIUM   | RESOLVED     | p. 119 |
| L-01 | Bad Debt Can Wrap Around uint128                                 | Math          | LOW      | RESOLVED     | p. 120 |

## Findings continued.

| ID   | TITLE                                                                    | CATEGORY      | SEVERITY      | STATUS       | PAGE   |
|------|--------------------------------------------------------------------------|---------------|---------------|--------------|--------|
| L-02 | Missing Settled Position Preview View                                    | Informational | LOW           | ACKNOWLEDGED | p. 122 |
| L-03 | Split Reports Alter CGBM Movement                                        | Math          | LOW           | ACKNOWLEDGED | p. 123 |
| L-04 | Unsafe Minimum Variance Bounds                                           | Math          | LOW           | RESOLVED     | p. 124 |
| L-05 | Valid History Can Brick Unbounded                                        | Math          | LOW           | RESOLVED     | p. 125 |
| L-06 | Bounded Accumulates Hidden Windup                                        | Math          | LOW           | RESOLVED     | p. 126 |
| L-07 | Stale taker margin snapshots can bypass dynamic risk tiers               | Configuration | LOW           | RESOLVED     | p. 127 |
| L-08 | Same-liquidation swap fees accrue before insolvency bad debt is recorded | Informational | LOW           | ACKNOWLEDGED | p. 128 |
| L-09 | Rounded-up maker capacity proration causes dust-scale cap drift          | Rounding      | LOW           | RESOLVED     | p. 129 |
| L-10 | V4 rounding dust can block final maker exit without opposite capacity    | DoS           | LOW           | RESOLVED     | p. 130 |
| L-11 | Bounded round-trip can brick perps on first zero-delta update            | DoS           | LOW           | RESOLVED     | p. 131 |
| L-12 | Alpha Zero Removes Confidence                                            | Configuration | LOW           | ACKNOWLEDGED | p. 132 |
| L-13 | Unbounded Can Publish Zero                                               | Configuration | LOW           | ACKNOWLEDGED | p. 133 |
| L-14 | Unbounded Exceeds Consumer Domains                                       | Math          | LOW           | ACKNOWLEDGED | p. 134 |
| L-15 | Bounded Round Trips Lose Precision                                       | Math          | LOW           | ACKNOWLEDGED | p. 135 |
| L-16 | TWAP Accumulator Can Overflow                                            | Math          | LOW           | ACKNOWLEDGED | p. 136 |
| I-01 | Oracle Reports Lack Expiry                                               | Warning       | INFORMATIONAL | ACKNOWLEDGED | p. 137 |

## Findings continued.

| ID   | TITLE                                                                 | CATEGORY         | SEVERITY      | STATUS       | PAGE   |
|------|-----------------------------------------------------------------------|------------------|---------------|--------------|--------|
| I-02 | Composite Reads Mixed Rounds                                          | Warning          | INFORMATIONAL | ACKNOWLEDGED | p. 138 |
| I-03 | Beacon Switch Keeps Old EMA                                           | Configuration    | INFORMATIONAL | RESOLVED     | p. 139 |
| I-04 | Funding Uses Cached Rate                                              | Warning          | INFORMATIONAL | RESOLVED     | p. 140 |
| I-05 | Timelock Survives Owner Transfer                                      | Warning          | INFORMATIONAL | RESOLVED     | p. 142 |
| I-06 | Util Fees Use Latest Beacon Spot                                      | Warning          | INFORMATIONAL | RESOLVED     | p. 144 |
| I-07 | Complementary Probability Rounding                                    | Warning          | INFORMATIONAL | ACKNOWLEDGED | p. 145 |
| I-08 | One-Sided Updates Can Vanish                                          | Informational    | INFORMATIONAL | ACKNOWLEDGED | p. 146 |
| I-09 | Tick spam raises taker swap gas along crossed path                    | Gas Griefing     | INFORMATIONAL | ACKNOWLEDGED | p. 147 |
| I-10 | Dust ranges can add zero-output exact-output liquidation tolls        | Informational    | INFORMATIONAL | ACKNOWLEDGED | p. 148 |
| I-11 | Maker-to-taker conversion skips MIN_OPENING_MARGIN floor              | Informational    | INFORMATIONAL | RESOLVED     | p. 149 |
| I-12 | Zero-capacity shadow liquidity dilutes honest LP fees                 | Informational    | INFORMATIONAL | RESOLVED     | p. 150 |
| I-13 | openTaker utilization failures occur after swap execution             | Gas Optimization | INFORMATIONAL | RESOLVED     | p. 151 |
| I-14 | Insurance consumption lacks complete bad-debt event coverage          | —                | INFORMATIONAL | RESOLVED     | p. 152 |
| I-15 | openTaker MarginTransferred reports pre-fee totalMargin               | Informational    | INFORMATIONAL | RESOLVED     | p. 153 |
| I-16 | Terminal MAX_TICK leaves no active maker liquidity for buy-side fills | Informational    | INFORMATIONAL | RESOLVED     | p. 154 |

## Findings continued.

| ID   | TITLE                                                                                | CATEGORY       | SEVERITY      | STATUS       | PAGE   |
|------|--------------------------------------------------------------------------------------|----------------|---------------|--------------|--------|
| I-17 | Taker liquidation eligibility ignores mandatory swap fees                            | Informational  | INFORMATIONAL | RESOLVED     | p. 155 |
| I-18 | Backstops can hand over positions that remain liquidatable after the liquidation fee | Informational  | INFORMATIONAL | RESOLVED     | p. 156 |
| I-19 | Unsigned Initial Price Enters TWAP                                                   | Best Practices | INFORMATIONAL | ACKNOWLEDGED | p. 157 |
| I-20 | Oracle Configurations Need Safety Checks                                             | Best Practices | INFORMATIONAL | ACKNOWLEDGED | p. 158 |

### REMEDIATION REVIEW 2

|      |                                                                         |               |          |              |        |
|------|-------------------------------------------------------------------------|---------------|----------|--------------|--------|
| C-01 | Exposure increases bypass the initial margin requirement                | Validation    | CRITICAL | ACKNOWLEDGED | p. 160 |
| C-02 | Same-block mark manipulation enables profitable atomic self-liquidation | Logical Error | CRITICAL | ACKNOWLEDGED | p. 162 |
| H-01 | Position Actions Lack Mark-Price Bounds                                 | Validation    | HIGH     | ACKNOWLEDGED | p. 164 |
| H-02 | Sandwiched reports poison index EMA and enable wrongful taker backstops | Logical Error | HIGH     | ACKNOWLEDGED | p. 165 |
| H-03 | Perp Markets Accept Stale Beacon Prices                                 | Oracle        | HIGH     | ACKNOWLEDGED | p. 166 |
| H-04 | Unsafe Low-Fee Configurations Enable Profitable Atomic Self-Liquidation | Validation    | HIGH     | ACKNOWLEDGED | p. 167 |
| H-05 | EMA-Anchored Price Bounds Enable Profitable Ratchet Liquidations        | Validation    | HIGH     | ACKNOWLEDGED | p. 169 |
| H-06 | Reported capacity does not guarantee executable close liquidity         | DoS           | HIGH     | ACKNOWLEDGED | p. 171 |
| H-07 | Transient Mark PnL Bypasses Initial Margin                              | Oracle        | HIGH     | ACKNOWLEDGED | p. 173 |

## Findings continued.

| ID     | TITLE                                                                                                 | CATEGORY      | SEVERITY | STATUS       | PAGE   |
|--------|-------------------------------------------------------------------------------------------------------|---------------|----------|--------------|--------|
| H-08   | One-wei mark floor enables undercollateralized shorts and bad debt                                    | Math          | HIGH     | ACKNOWLEDGED | p. 174 |
| H-09   | Mark-AMM divergence hides executable insolvency                                                       | Oracle        | HIGH     | ACKNOWLEDGED | p. 175 |
| M-01   | Deploy Script Still Uses Mock Contracts                                                               | Warning       | MEDIUM   | RESOLVED     | p. 176 |
| M-02   | Funding Cap Calc Uses Wrong Price                                                                     | Math          | MEDIUM   | RESOLVED     | p. 177 |
| M-02.2 | Out-of-domain beacon index lets liquidators drain shared collateral through mark-priced fees          | Validation    | MEDIUM   | RESOLVED     | p. 178 |
| M-03   | Taker Limits Exclude Dynamic Swap Fees                                                                | Validation    | MEDIUM   | ACKNOWLEDGED | p. 179 |
| M-04   | Dormant markets force makers to reopen at stale AMM price and expose collateral to funding extraction | Warning       | MEDIUM   | ACKNOWLEDGED | p. 180 |
| M-05   | Backstops can hand over positions that remain liquidatable after the liquidation fee                  | Logical Error | MEDIUM   | ACKNOWLEDGED | p. 182 |
| M-06   | Remote low-price maker ranges capture utilization rent with cheap capacity                            | Math          | MEDIUM   | ACKNOWLEDGED | p. 183 |
| M-07   | Margin operations lack bad-debt slippage protection                                                   | Validation    | MEDIUM   | ACKNOWLEDGED | p. 184 |
| M-08   | Split maker liquidations repeatedly charge unchanged residual notional                                | Gaming        | MEDIUM   | ACKNOWLEDGED | p. 185 |
| M-09   | Self-matched trades can modestly bias maker funding receipts                                          | Gaming        | MEDIUM   | ACKNOWLEDGED | p. 186 |

## Findings continued.

| ID   | TITLE                                                                                | CATEGORY      | SEVERITY | STATUS       | PAGE   |
|------|--------------------------------------------------------------------------------------|---------------|----------|--------------|--------|
| M-10 | Per-tick LP fee rounding slightly overcharges fragmented liquidity books             | Validation    | MEDIUM   | ACKNOWLEDGED | p. 187 |
| M-11 | Beacon Updates Do Not Checkpoint Perp Markets                                        | Compatibility | MEDIUM   | ACKNOWLEDGED | p. 188 |
| M-12 | A Reverting Beacon Prevents Beacon Migration                                         | DoS           | MEDIUM   | ACKNOWLEDGED | p. 189 |
| M-13 | Blacklisted Users Are Not Liquidatable                                               | DoS           | MEDIUM   | ACKNOWLEDGED | p. 190 |
| M-14 | Unnecessary Maker Position Close DoS                                                 | DoS           | MEDIUM   | RESOLVED     | p. 191 |
| M-15 | Users can avoid socialized losses                                                    | Gaming        | MEDIUM   | ACKNOWLEDGED | p. 192 |
| M-16 | Taker liquidation eligibility ignores mandatory swap fees                            | Logical Error | MEDIUM   | ACKNOWLEDGED | p. 194 |
| M-17 | Tightened PricImpact bounds temporarily block incremental risk-reducing swaps        | Validation    | MEDIUM   | ACKNOWLEDGED | p. 195 |
| M-18 | Partial liquidations can sandwich pending maker backstops                            | Validation    | MEDIUM   | ACKNOWLEDGED | p. 196 |
| M-19 | Unsigned margin write blocks net-equity maker recapitalization                       | Logical Error | MEDIUM   | ACKNOWLEDGED | p. 197 |
| M-20 | Opening collateral is omitted from same-transaction fee socialization                | Math          | MEDIUM   | ACKNOWLEDGED | p. 198 |
| M-21 | Dust liquidity lets directional maker residuals bypass taker OI and utilization fees | Logical Error | MEDIUM   | ACKNOWLEDGED | p. 199 |
| M-22 | adjustTaker absolute health gate blocks incremental deleveraging                     | Validation    | MEDIUM   | ACKNOWLEDGED | p. 200 |

## Findings continued.

| ID   | TITLE                                                                                 | CATEGORY            | SEVERITY | STATUS       | PAGE   |
|------|---------------------------------------------------------------------------------------|---------------------|----------|--------------|--------|
| L-01 | tokenURI returns metadata for nonexistent and burned position NFTs                    | Unexpected Behavior | LOW      | RESOLVED     | p. 201 |
| L-02 | Six-decimal utilization flooring slightly underprices utilization fees                | Rounding            | LOW      | RESOLVED     | p. 202 |
| L-03 | Swap-Fee Limit Excludes the Protocol Fee                                              | Validation          | LOW      | ACKNOWLEDGED | p. 203 |
| L-05 | Unnormalized Composite Weights                                                        | Validation          | LOW      | ACKNOWLEDGED | p. 204 |
| L-06 | Timelock defaults to zero, allowing immediate admin changes                           | Configuration       | LOW      | RESOLVED     | p. 205 |
| L-07 | V4 rounding dust can block final maker exit without opposite capacity                 | DoS                 | LOW      | ACKNOWLEDGED | p. 206 |
| L-08 | Module Setters Can Install Unusable Addresses                                         | Validation          | LOW      | ACKNOWLEDGED | p. 207 |
| L-09 | Stale timelocked beacon migrations let third parties liquidate healthy positions      | Validation          | LOW      | ACKNOWLEDGED | p. 208 |
| L-10 | Per-maker capacity remainders understate global capacity and inflate utilization fees | Rounding            | LOW      | ACKNOWLEDGED | p. 209 |
| L-11 | Swap fee ordering diverts protocol and creator fees into insurance                    | Rewards             | LOW      | ACKNOWLEDGED | p. 210 |
| L-12 | High liquidity permits stationary zero-quote exact-input shorts                       | Validation          | LOW      | ACKNOWLEDGED | p. 211 |
| L-13 | Long swaps undercharge LP fees due to floor-vs-ceil replay rounding                   | Rounding            | LOW      | ACKNOWLEDGED | p. 212 |

## Findings continued.

| ID   | TITLE                                                                                    | CATEGORY            | SEVERITY      | STATUS       | PAGE   |
|------|------------------------------------------------------------------------------------------|---------------------|---------------|--------------|--------|
| L-14 | Forced partial maker liquidations discard LP and utilization fee remainders              | Rounding            | LOW           | ACKNOWLEDGED | p. 213 |
| L-15 | Chunked maker liquidations reduce fees through repeated flooring                         | Rounding            | LOW           | ACKNOWLEDGED | p. 214 |
| L-16 | Partial liquidation can turn a pending full close into an opposite-side position         | Unexpected Behavior | LOW           | ACKNOWLEDGED | p. 215 |
| L-17 | Utilization-curve updates require each market to refresh its cached rates                | Rewards             | LOW           | ACKNOWLEDGED | p. 216 |
| L-18 | TWAP Accumulator Can Overflow                                                            | Math                | LOW           | ACKNOWLEDGED | p. 217 |
| L-19 | CGBM historical variance permits abrupt moves on confidence-regime changes               | Math                | LOW           | ACKNOWLEDGED | p. 218 |
| L-20 | TernaryToBinary floor rounding directionally biases CGBM and worsens dust-sequence jumps | Rounding            | LOW           | ACKNOWLEDGED | p. 219 |
| L-21 | CGBM over-corrects weak positive reports and lowers Unbounded prices                     | Math                | LOW           | ACKNOWLEDGED | p. 220 |
| L-22 | CGBM stationary low-confidence regime decouples oracle moves from classifier confidence  | Math                | LOW           | ACKNOWLEDGED | p. 221 |
| I-01 | Backstop deposits above int128.max contradict the uint128 API                            | Validation          | INFORMATIONAL | RESOLVED     | p. 222 |
| I-02 | setSwapFees emits an unchanged LP-configuration event                                    | Events              | INFORMATIONAL | RESOLVED     | p. 223 |
| I-03 | Self-recipient backstops clear token-specific ERC-721 approvals                          | Informational       | INFORMATIONAL | RESOLVED     | p. 224 |

## Findings continued.

| ID   | TITLE                                                                                     | CATEGORY          | SEVERITY      | STATUS       | PAGE   |
|------|-------------------------------------------------------------------------------------------|-------------------|---------------|--------------|--------|
| I-04 | Liquidation Fee Can Be Set to 100%                                                        | Trust Assumptions | INFORMATIONAL | ACKNOWLEDGED | p. 225 |
| I-05 | Reorg can capture GroupManager component bindings through the shared factory CREATE nonce | Informational     | INFORMATIONAL | ACKNOWLEDGED | p. 226 |
| I-06 | Aggregate capacity can wrap at uint128 boundary                                           | Warning           | INFORMATIONAL | ACKNOWLEDGED | p. 227 |
| I-07 | Bundled group reports create live-spot moves absent from wall-clock TWAP                  | Informational     | INFORMATIONAL | ACKNOWLEDGED | p. 228 |
| I-08 | TwAvg comment incorrectly describes timestamp deduplication as per-block                  | Informational     | INFORMATIONAL | ACKNOWLEDGED | p. 229 |
| I-09 | Zero Price-Impact Bound Freezes Swaps                                                     | Validation        | INFORMATIONAL | ACKNOWLEDGED | p. 230 |
| I-10 | Positive dust liquidity permits one-atom AMM spot displacement                            | Informational     | INFORMATIONAL | ACKNOWLEDGED | p. 231 |
| I-11 | Unchecked uint80 accumulators can erase protocol and creator fee claims                   | Informational     | INFORMATIONAL | ACKNOWLEDGED | p. 232 |
| I-12 | Bitmap-word segmentation leaves irreducible exact-fill residual                           | Rounding          | INFORMATIONAL | ACKNOWLEDGED | p. 233 |
| I-13 | Excessive permitted fees can block insolvent taker liquidations                           | Validation        | INFORMATIONAL | ACKNOWLEDGED | p. 234 |
| I-14 | Zero backstop margin ratio permanently disables backstop rescue                           | Validation        | INFORMATIONAL | ACKNOWLEDGED | p. 235 |
| I-15 | Unbounded symbol length enables metadata gas griefing                                     | Gas Griefing      | INFORMATIONAL | ACKNOWLEDGED | p. 236 |

## Findings continued.

| ID   | TITLE                                                                          | CATEGORY         | SEVERITY      | STATUS       | PAGE   |
|------|--------------------------------------------------------------------------------|------------------|---------------|--------------|--------|
| I-16 | setSwapFees emits an unchanged LP-configuration event                          | Events           | INFORMATIONAL | ACKNOWLEDGED | p. 237 |
| I-17 | The V4 donation callback is unreachable from production Perp flows             | Superfluous Code | INFORMATIONAL | ACKNOWLEDGED | p. 238 |
| I-18 | Unchecked insurance credits can wrap the uint80 reserve balance                | Informational    | INFORMATIONAL | ACKNOWLEDGED | p. 239 |
| I-19 | Unchecked insurance credits can wrap the uint80 reserve balance                | Rounding         | INFORMATIONAL | ACKNOWLEDGED | p. 240 |
| I-20 | sFullMulDiv reverts on the representable int256 minimum                        | Math             | INFORMATIONAL | ACKNOWLEDGED | p. 241 |
| I-21 | Utilization proration discards sub-atom maker earnings dust                    | Rounding         | INFORMATIONAL | ACKNOWLEDGED | p. 242 |
| I-22 | E6 health flooring rejects atom-scale maker liquidations within capacity slack | Rounding         | INFORMATIONAL | ACKNOWLEDGED | p. 243 |
| I-23 | Post-socialization fee split assigns rounding residuals to creator             | Rounding         | INFORMATIONAL | ACKNOWLEDGED | p. 244 |
| I-24 | Oversized maker margin bricks converted taker adjustments                      | Informational    | INFORMATIONAL | ACKNOWLEDGED | p. 245 |
| I-25 | Malformed UTF-8 market names can disrupt strict metadata consumers             | Informational    | INFORMATIONAL | ACKNOWLEDGED | p. 246 |
| I-26 | Clipped Bounded updates amplify CGBM reversal sensitivity                      | Informational    | INFORMATIONAL | ACKNOWLEDGED | p. 247 |
| I-27 | Reorg can substitute hostile DiscreteAllocation at a factory child address     | Informational    | INFORMATIONAL | ACKNOWLEDGED | p. 248 |

## Findings continued.

| ID   | TITLE                                                                                    | CATEGORY      | SEVERITY      | STATUS       | PAGE   |
|------|------------------------------------------------------------------------------------------|---------------|---------------|--------------|--------|
| I-28 | Dust pseudocount can round RelativeDominance ratio to zero and revert updates            | Rounding      | INFORMATIONAL | ACKNOWLEDGED | p. 249 |
| I-29 | z-clamping breaks exact martingale claim at finite Unbounded bounds                      | Informational | INFORMATIONAL | ACKNOWLEDGED | p. 250 |
| I-30 | CGBM zero-history fallback zeroes drift correction on nonzero branches                   | Informational | INFORMATIONAL | ACKNOWLEDGED | p. 251 |
| I-31 | DGBM+Bounded can bias indices under asymmetric high-volatility configurations            | Informational | INFORMATIONAL | ACKNOWLEDGED | p. 252 |
| I-32 | CGBM+Unbounded martingale claim is not satisfied by the second-order correction          | Informational | INFORMATIONAL | ACKNOWLEDGED | p. 253 |
| I-33 | BinaryDGBM sample scripts miswire parameters and revert on nonce zero                    | Informational | INFORMATIONAL | ACKNOWLEDGED | p. 254 |
| I-34 | Nested WeightedSum compositions are non-associative due to per-node flooring             | Rounding      | INFORMATIONAL | ACKNOWLEDGED | p. 255 |
| I-35 | GMNormalize compression factor theoretically truncates to zero at unreachable dispersion | —             | INFORMATIONAL | ACKNOWLEDGED | p. 256 |
| I-36 | DGBM quantizes sub-WAD branch probability to zero                                        | Informational | INFORMATIONAL | ACKNOWLEDGED | p. 257 |
| I-37 | Softmax allocation scaling incurs avoidable double rounding                              | Rounding      | INFORMATIONAL | ACKNOWLEDGED | p. 258 |
| I-38 | Shipped Argmax preprocessor has no compatible multiclass base function                   | Informational | INFORMATIONAL | ACKNOWLEDGED | p. 259 |

## Findings continued.

| ID   | TITLE                                                              | CATEGORY      | SEVERITY      | STATUS       | PAGE   |
|------|--------------------------------------------------------------------|---------------|---------------|--------------|--------|
| I-39 | Softmax member TWAPs can sum above INDEX_SCALE at one timestamp    | Informational | INFORMATIONAL | ACKNOWLEDGED | p. 260 |
| I-40 | GMNormalize compression creates capped equal-unit basket convexity | Informational | INFORMATIONAL | ACKNOWLEDGED | p. 261 |

HIGH **H-01**

RESOLVED

# TWAP accounting attributes elapsed time to the new index

CATEGORY Math  
LOCATION src/libraries/TwAvg.sol  
REVIEW Main Review

## DESCRIPTION

`TwAvg.calcObservation` accrues `currentVal * timePassed` for the entire interval since the last observation. The beacon callers mutate `_index` before calling `write`, so the value passed into `write` is the new index, not the value that was active during the elapsed interval.

For example, if an index is `100` for one day and then updates to `200`, the observation written at update time records that past day as if the value had been `200`.

`TwAvg.timeWeightedAvg` also returns `currentVal` when `delta == 0`, which is incorrect when the true cumulative delta is zero because the tracked value was zero over the window.

Impact: TWAP consumers can receive materially incorrect averages. A single update can rewrite the apparent value for the entire interval since the last observation, making TWAP-dependent integrations easy to mislead.

## RECOMMENDATION

Write the observation using the previous index before mutating `_index`. Treat `currentVal` only as the value from the latest observation timestamp to `block.timestamp`. In `timeWeightedAvg`, only fall back to `currentVal` when `endTimeStamp == startTimeStamp`; otherwise return `delta / (endTimeStamp - startTimeStamp)`.

## RESOLUTION

Status: Resolved.

HIGH **H-02**

RESOLVED

# CompositeBeacon TWAP Can Be Manipulated

CATEGORY Gaming  
LOCATION `src/core/composite/CompositeBeacon.sol:57`  
REVIEW Main Review

## DESCRIPTION

All beacons except Composite require a cryptographic proof verified by `VERIFIER`, so observations are only written when a valid measurement update is submitted. `CompositeBeacon.update()` has no such gate:

```
// src/core/composite/CompositeBeacon.sol:57
function update() external {
    uint256 _index = index();
    _twAvgState.write(uint32(block.timestamp), _index);
    emit IndexUpdated(_index);
}
```

`TwAvg` enforces at most one write per block (`TwAvg.sol:95`), but it does not enforce a minimum time gap between observations. The buffer is a circular queue whose oldest observation is overwritten once the active cardinality is full:

```
state.index = (state.index + 1) % state.cardinality;
state.observations[state.index] = calcObservation(recentObservation, blockTimestamp,
currentVal);
```

When `twAvg(secondsAgo)` is requested for a target timestamp older than the oldest stored observation, `TwAvg` silently caps the start of the calculation to the oldest available observation:

```
if (targetTimestamp <= oldest.timestamp) return (oldest, oldest);
```

Therefore, the effective lookback window is not guaranteed by the `secondsAgo` argument. It is bounded by the timestamp range currently retained in the circular buffer. Because anyone can call `CompositeBeacon.update()` every block, an attacker can compress the retained history to approximately:

```
cardinality * block_time
```

For example, if the active cardinality is 10 and blocks are approximately 12 seconds apart, an attacker can replace the retained history in about 120 seconds by calling `update()`

## H-02 CompositeBeacon TWAP Can Be Manipulated

CONTINUED

once per block. A later call to `twAvg(3600)` will not use one hour of history; it will use only the roughly two minutes still present in the buffer.

This issue remains even if the separate TWAP accounting bug (H-1) is fixed. The root cause here is that `CompositeBeacon` exposes a permissionless manual sampler for a derived value that may change independently of `CompositeBeacon` state. The caller cannot directly set the composite index, but they can force the observation buffer to contain only recent samples, removing the intended protection of a longer TWAP window.

Impact: Consumers may believe they are using a long-window TWAP, while the returned value is actually calculated over a much shorter attacker-compressed window. This weakens TWAP-based manipulation resistance and makes downstream pricing, collateral valuation, liquidation, or settlement logic more exposed to short-term composite index movement.

The smaller the active cardinality, the faster and cheaper the retained history can be overwritten. This is Medium severity in isolation because the attacker does not directly control the composite index value, but it can become High severity for integrations where the compressed TWAP is used for value-bearing decisions and the underlying reference indices can be economically moved over the shortened window.

### — RECOMMENDATION

Restrict `update()` to a trusted keeper role, or require a minimum time gap between calls so the full buffer cannot be populated by a single actor within a short window. Alternatively, auto-write an observation inside `index()` so every read passively populates the buffer, mirroring how Uniswap V3 observations are written on every state-changing interaction rather than by a separate permissionless call.

### — RESOLUTION

Status: Resolved.

HIGH **H-03**

RESOLVED

# refreshRatesAndEmas uses post-swap price

CATEGORY Logical Error

LOCATION `src/libraries/PerpLogic.sol:460` (`accrue`), `src/libraries/PerpLogic.sol:488` (`refreshRatesAndEmas`), `src/libraries/PerpLogic.sol:505` (`calcEmas`)

REVIEW Main Review

## DESCRIPTION

Every taker action runs two EMA computations over the same elapsed period `dt`, using two different price observations. The results diverge in a way that leaves the stored EMA materially wrong after any trade in an idle market.

**\*\*Step 1 — `accrue()` reads pre-swap price, computes correct intermediate EMA:\*\***

```

snap.spots.ammPrice = poolState(...); // P_old (pre-swap)
snap.emas = calcEmas(s.emas, snap.spots, env.emaWindow, dt);
// stored as snap.emas = old_ema * alpha + P_old * (1 - alpha)
// NOT written to s.emas

```

**\*\*Step 2 — swap executes, price moves `P_old` → `P_new` Step 3 — `refreshRatesAndEmas()` called with post-swap spots, recomputes EMA from scratch:\*\***

```

uint256 dt = block.timestamp - s.rates.lastTouch; // same full idle dt
s.emas = calcEmas(s.emas, newSpots, env.emaWindow, dt);
// s.emas = old_ema * alpha + P_new * (1 - alpha) ← wrong

```

The correctly computed `snap.emas` (which weighted `P_old` across the idle period) is discarded.

`refreshRatesAndEmas` recomputes from the same `s.emas` base with the same `dt` but substitutes `P_new` — attributing the post-swap price to the entire elapsed period as if the swap happened at `lastTouch`.

Why this matters in idle markets:

The EMA smoothing factor  $\alpha = e^{(-dt/emaWindow)}$ . As `dt` grows:

$dt \rightarrow \infty \Rightarrow \alpha \rightarrow 0 \Rightarrow (1 - \alpha) \rightarrow 1$

After a long idle period, a single swap causes the stored EMA to snap almost entirely to `P_new`:

```

Market idle 24 hours. EMA = $100. Swap moves AMM: $100 → $150.
Correct: s.emas = 100 * alpha + 100 * (1 - alpha) = $100 (P_old for 24h)
Actual: s.emas = 100 * alpha + 150 * (1 - alpha) >> $100 (P_new for 24h)

```

**H-03** refreshRatesAndEmas uses post-swap price

CONTINUED

Funding rate impact:

The distorted EMA immediately sets the next funding rate:

```
newRates.fundingPerDay = env.modules.funding.funding(newSpots, s.emas);
// ↑ wrong EMA
```

All positions opened or adjusted after this point accrue funding based on a premium/discount that reflects a fabricated price history. In the worst case (market dormant for `dt >> emaWindow`), a single swap fully controls the stored EMA and thus the funding rate for all subsequent interactions.

**— RECOMMENDATION**

Pass `snap.emas` into `refreshRatesAndEmas` and use it as the stored EMA directly, rather than recomputing:

```
// current – wrong
function refreshRatesAndEmas(PerpStorage storage s, Env memory env, PricePair memory newSpots)
internal {
  uint256 dt = block.timestamp - s.rates.lastTouch;
  s.emas = calcEmas(s.emas, newSpots, env.emaWindow, dt); // ← uses P_new for full dt
  ...
}

// fix – pass snap.emas from accrue, which already accounted for idle period at P_old
function refreshRatesAndEmas(PerpStorage storage s, Env memory env, PricePair memory newSpots,
PricePair memory snapEmas) internal {
  s.emas = snapEmas; // correct: idle period weighted by P_old, already computed in accrue
  ...
}
```

Since `calcEmas` with `dt = 0` returns the input unchanged (`alpha = 1`), the post-swap price observation at the instant of the action contributes nothing to the EMA — which is correct. The idle period has already been absorbed using the pre-action price.

**— RESOLUTION**

Status: Resolved.

HIGH **H-04**

RESOLVED

# Post-Swap Donation Misallocates LP Fees

CATEGORY Logical Error  
LOCATION src/Perp.sol:366-367  
REVIEW Main Review

## DESCRIPTION

The market pool is initialized with zero native Uniswap swap fee.

During a taker swap, the protocol executes the Uniswap swap first. Only after the swap completes does it compute the LP fee and donate it/

The callback then calls:

```
POOL_MANAGER.donate(poolKey(), 0, lpFeeAmt, "");
```

In Uniswap v4, `donate()` credits fee growth to the pool's current active liquidity:

```
uint128 liquidity = state.liquidity;  
state.feeGrowthGlobal1X128 += amount1 * Q128 / liquidity;
```

Since donation happens after the swap, `state.liquidity` and the current tick correspond to the post-swap price, not the liquidity that was active throughout the swap path.

Example

```
Maker A range: 100 -> 119  
Maker B range: 119 -> 120  
Taker swap moves AMM from 100 -> 120  
Swap volume: 1,000 USDC  
LP fee: 1% = 10 USDC
```

Expected behavior:

Maker A earns fees for the portion of swap volume executed through 100 -> 119.

Maker B earns fees for the portion of swap volume executed through 119 -> 120.

Current behavior:

1. Swap executes from 100 -> 120 with zero native LP fee.
2. Protocol computes 10 USDC LP fee on total volume.
3. Protocol donates 10 USDC after the swap.

**H-04** Post-Swap Donation Misallocates LP Fees

CONTINUED

4. The donation is credited to liquidity active at the final price near 120.

5. Maker B can receive the donated fee even if Maker A provided most of the liquidity used by the swap.

This means LP fees are distributed by final active liquidity, not by liquidity that serviced the taker trade.

JIT / Sandwich Amplification

This also amplifies a JIT-style fee capture vector.

A searcher can observe a large taker swap and front-run by adding liquidity around the expected final post-swap price. If the taker swap lands in that range, the searcher's liquidity is active when the post-swap donation occurs and can capture a disproportionate share of the LP fee calculated on the entire swap volume.

This is different from normal Uniswap JIT liquidity.

In Uniswap, JIT LP must provide liquidity that the swap actually trades through.

Fees are accrued during each swap step, proportional to active liquidity in that step.

**— RECOMMENDATION**

Do not distribute LP fees through a single post-swap donation.

**— RESOLUTION**

Status: Resolved.

HIGH **H-05**

ACKNOWLEDGED

## Validations can prevent AMM/index convergence

|          |                             |
|----------|-----------------------------|
| CATEGORY | Logical Error               |
| LOCATION | src/libraries/PerpLogic.sol |
| REVIEW   | Main Review                 |

### — DESCRIPTION

perp.city relies on taker swaps through the Uniswap V4 AMM to move the AMM price toward the external index price. Unlike a normal spot AMM, however, these convergence trades are not freely executable. They must pass protocol validation, including capacity/utilization checks.

The clearest example is directional capacity. A taker swap is only accepted if the resulting open interest remains within maker-supplied capacity:

```
function checkUtilization(OpenInterest memory oi, Capacity memory cap) internal pure {  
    if (oi.long > cap.long) revert LongUtilizationExceeded();  
    if (oi.short > cap.short) revert ShortUtilizationExceeded();  
}
```

`updateOpenInterest()` applies this check after every taker swap. Therefore, when the side needed to move the AMM toward the index is already at capacity, the corrective trade can revert even if the AMM price is materially stale.

Example:

```
AMM price = 100  
real/index = 120  
long OI = long capacity  
needed action = buy perp / increase long OI to push AMM upward  
result = new long trades revert with LongUtilizationExceeded
```

The AMM is not necessarily permanently frozen. Existing shorts may be able to close, makers may add capacity, or governance/keepers may intervene depending on deployment policy. The issue is that convergence is not guaranteed by arbitrage alone once the trades needed for convergence are rejected by protocol validation.

Capacity is not the only relevant validation. The same convergence or unwind trade can also fail because of other trade validations.

**H-05** Validations can prevent AMM/index convergence

CONTINUED

This is important because protocol accounting uses the AMM price and AMM EMA as inputs to mark price, funding, price-impact limits, utilization fees, and health checks.

If the AMM price remains stale while the index has moved, any module that gives meaningful weight to the stale AMM component can produce distorted risk checks. For example, if the index moves up while the AMM remains low, long positions may be valued too conservatively and short positions too generously relative to the current index. That can create false-liquidation risk for longs, delayed liquidation risk for shorts, distorted funding, and stale utilization signals.

**— RECOMMENDATION**

The core issue is that the protocol has no explicit escape path for AMM/index divergence when the trades needed to restore convergence are rejected by capacity, price-impact, liquidity. We are not sure how to resolve this as per perp-city's current design as of now, however this should be resolved as EMA of AMM itself is deciding factor for operations.

**— RESOLUTION**

Status: Acknowledged.

HIGH **H-06**

RESOLVED

# adjustTaker Allows Expo Below Init Margin

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/Perp.sol#L322-L327>

REVIEW Main Review

## DESCRIPTION

The `openTaker` function enforces taker initial margin, but later `adjustTaker` checks post-adjust health against the stored liquidation margin ratio. This allows an existing taker to withdraw margin down near liquidation level to get more leverage than the system allows.

So basically anyone can:

- Open a position at the initial margin ratio
- Call `adjustTaker` to bypass the initial margin ratio and exceed the allowed leverage

As anyone can create new markets, there might be markets referencing to assets with a very fluctuating price. In this case bypassing the maximum allowed leverage could be very dangerous for the system and lead to bad debt easily.

The same behavior occurs in the maker to taker position conversion, as well as in the normal maker flow.

## RECOMMENDATION

Consider to implement checks that revert in the `adjustTaker` flow if a user tries to decrease their margin ratio and the end result is not above the initial margin ratio.

## RESOLUTION

Status: Resolved.

HIGH **H-07**

RESOLVED

# Insurance Fund Charged For Solvent Positions

|          |                                                                                                                                                                                                                                                                                                                                   |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Logical Error                                                                                                                                                                                                                                                                                                                     |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/libraries/PerpLogic.sol#L303-L309">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/libraries/PerpLogic.sol#L303-L309</a> |
| REVIEW   | Main Review                                                                                                                                                                                                                                                                                                                       |

## DESCRIPTION

During full maker liquidation, the protocol determines liquidation eligibility using the maker's full marked equity, including residual perp exposure. However, after all maker liquidity is removed, bad-debt accounting only considers settled margin plus residual USDC exposure:

```
equity = netMargin + pos.delta.amount1();  
uint256 remainingEquity = accountBadDebt(s, equity);
```

This ignores `pos.delta.amount0()`, even though the residual perp exposure is preserved and converted into a taker position immediately afterward.

As a result, the protocol can treat a position as bad debt even when its total marked equity is positive. The insurance fund is debited for the realized USDC deficit, while the original position owner keeps the residual profitable perp exposure and can later close it for USDC.

This enables profitable self-liquidations: the user can socialize the realized loss on the USDC leg through the insurance fund while retaining the profitable residual perp exposure.

## RECOMMENDATION

Consider to not call `accountBadDebt` on maker full-liquidation residuals without including the marked value of `amount0`.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/main/test/guardianaudits/pocs/InsuranceFundUsedDespiteNoBadDebt.t.sol>

## RESOLUTION

Status: Resolved.

HIGH **H-08**

RESOLVED

# ContinuousAllocation Ignores Relevance

CATEGORY Math  
LOCATION src/core/group/groupfns/ContinuousAllocation.sol  
REVIEW Main Review

## DESCRIPTION

`ContinuousAllocation` removes the irrelevant class and normalizes the remaining prediction vector over relevant classes before computing the z-space delta. This preserves directional exposure, but it causes low-relevance and high-relevance observations with the same conditional class split to produce effectively the same update magnitude.

For example:

```
[0, 0.7e18, 0.3e18]
[0.99e18, 0.007e18, 0.003e18]
```

both normalize to approximately the same relevant-class distribution. The second prediction is almost entirely irrelevant, but it can still move the index with the same directional magnitude as the first.

Since markets rely on these indices, mostly irrelevant model outputs can materially affect market pricing, funding, collateral valuation, or liquidation logic instead of being proportionally dampened.

## RECOMMENDATION

Scale the directional delta by total relevant mass after computing the conditional relevant-class deviation.

For example:

```
uint256 relevantMass = predSum;
int256 delta = SCALED_SIGMA_BASE
.sMulWad(normPred.toInt256() - normClassProbs[i])
.sMulWad(relevantMass.toInt256());
```

If one-vs-all vectors are allowed and entries may not sum to `WAD`, explicitly define whether `relevantMass` should be `sum(prediction[1:])`, `WAD - prediction[0]`, or a clamped/normalized relevance score.

---

**H-08** ContinuousAllocation Ignores Relevance

CONTINUED

---

— RESOLUTION

Status: Resolved.

HIGH **H-09**

RESOLVED

# Dominance Double-Discounts Classes

|          |                                                                                     |
|----------|-------------------------------------------------------------------------------------|
| CATEGORY | Math                                                                                |
| LOCATION | src/core/group/groupfns/Dominance.sol src/core/group/groupfns/RelativeDominance.sol |
| REVIEW   | Main Review                                                                         |

## DESCRIPTION

`Dominance` and `RelativeDominance` treat group prediction vectors as if relevant class entries are conditional on relevance. They multiply each relevant class score by  $1 - \text{prediction}[0]$ .

However, the intended semantics are unconditional per-class probability scores, with class `0` reserved for irrelevant/outside. Under those semantics, each `prediction[i + 1]` already represents direct class probability mass. Multiplying it by total relevant mass discounts it a second time.

Example:

```
prediction = [0.80 irrelevant, 0.14 class A, 0.06 class B]
```

The class A score is already `0.14`. The implementation records:

```
0.14 * (1 - 0.80) = 0.028
```

This systematically suppresses relevant evidence whenever irrelevant probability is nonzero. Since markets rely on the resulting dominance indices, this can cause prices and downstream market mechanisms to track an incorrectly dampened index.

## RECOMMENDATION

Use the unconditional class score directly:

```
uint256 observation = prediction[i + 1];
```

If the protocol wants to support conditional vectors instead, encode that as a separate mode or component and validate the vector semantics explicitly at deployment.

## RESOLUTION

Status: Resolved.

HIGH **H-10**

RESOLVED

# Perp Markets Accept Stale Beacon Prices

|          |             |
|----------|-------------|
| CATEGORY | Oracle      |
| LOCATION | GLOBAL      |
| REVIEW   | Main Review |

## DESCRIPTION

In `PerpLogic accrue()`, the current index is read directly: `snap.spots.index = uint128(env.modules.beacon.index());`

That value then feeds mark price, funding, EMA refreshes, price-impact bounds, health checks, and liquidation logic. However, the beacon interface only exposes the `index()` and does not expose any freshness metadata and the beacon implementation keeps returning the last stored index until another valid update is submitted.

As a result, if the off-chain beacon signer/verifier/relayer pipeline stops producing updates, perp markets continue operating against the last published index indefinitely. This can happen accidentally due to infrastructure outages, signer downtime, verifier failures, or relayer failures. A malicious actor may also intentionally exploit this by DDoSing the off-chain signer, verifier, or relayer infrastructure, preventing fresh beacon updates while the perp market continues accepting the last published index indefinitely.

This is especially dangerous for volatile markets. If the real spot price moves materially while the beacon index remains frozen, the protocol can compute distorted mark prices, funding rates, price-impact limits, margin health, and liquidation eligibility. Positions may be incorrectly protected from liquidation, incorrectly liquidated, or allowed to trade against stale risk assumptions.

## RECOMMENDATION

Consider to add explicit freshness enforcement.

## RESOLUTION

Status: Resolved.

HIGH **H-11**

ACKNOWLEDGED

# Bad-Debt Socialization Uses Inflated totalMargin

|          |                                                                                                                                                                                                                                                                                                                                   |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Logical Error                                                                                                                                                                                                                                                                                                                     |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/d2f922c0cbf76a8d0742e902cb50bb4c155f33a0/src/libraries/PerpLogic.sol#L514-L515">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/d2f922c0cbf76a8d0742e902cb50bb4c155f33a0/src/libraries/PerpLogic.sol#L514-L515</a> |
| REVIEW   | Main Review                                                                                                                                                                                                                                                                                                                       |

## — DESCRIPTION

When an insolvent position is fully liquidated, PerpCity records the uncovered loss as bad debt but does not remove the closed position's previous margin from `s.solvencyState.totalMargin`.

`accountBadDebt()` increases `s.solvencyState.badDebt` when equity is negative, but it does not update `totalMargin`. The liquidation flow then calls `closeTaker()` with `remainingEquity == 0`, which deletes the position but does not decrease `totalMargin`. As a result, `totalMargin` can continue to include margin from positions that no longer exist and no longer represent withdrawable funds.

This stale value is later used as the denominator in `socializeLoss()`:

```
uint256 fee = debt >= margin ? originalAmt : originalAmt.fullMulDivUp(debt, margin);
```

Because the denominator is inflated, each future withdrawal or payout is charged too small a share of the outstanding bad debt. Remaining users can withdraw more than they should, leaving part of the bad debt unrepaid. That residual debt may then be pushed onto future insurance fees, donations, or later users instead of being fully socialized across the real remaining margin at the time of insolvency.

This was validated with a Foundry regression test. After an insolvent full taker liquidation, the deleted position had zero margin, but `solvencyState.totalMargin` still exceeded the sum of active position margins by the closed position's stale margin.

Separately, this same accounting inconsistency creates a payout-liveness risk. `transferMargin()` applies `socializeLoss()` to reduce the actual amount sent when bad debt exists, but still decrements `s.solvencyState.totalMargin` by the full requested claim amount. If `totalMargin` is stale or otherwise inconsistent and a valid payout claim exceeds the remaining tracked `totalMargin`, this full decrement can underflow and revert before the socialized payout completes. This could block otherwise valid closes, margin removals, or liquidation-fee payouts during insolvency.

---

**H-11** Bad-Debt Socialization Uses Inflated totalMargin

---

CONTINUED

**— RECOMMENDATION**

Ensure that insolvent full closures reduce `s.solvencyState.totalMargin` by the closed position's stale margin/equity contribution.

Preserve badDebt as a separate liability and do not additionally subtract the newly recorded bad debt from `totalMargin`.

**— RESOLUTION**

Status: Acknowledged.

HIGH **H-12**

RESOLVED

# DAlloc Amplifies Low-Relevance Markets

CATEGORY Math  
 LOCATION src/core/group/groupfns/DiscreteAllocation.sol  
 REVIEW Main Review

## — DESCRIPTION

`DiscreteAllocation` is intended to apply a winner-take-all update where rare classes receive larger z-space movement than common classes. The economically coherent form is to inverse-weight the winning class by its baseline probability.

However, the implementation divides the update magnitude by both the winning class probability and the total relevant probability mass:

```
updateMagnitudes[i] = scaledSigmaBase.divWad(classProbs[i + 1]).divWad(probSum);
```

where:

```
probSum = 1 - P(irrelevant)
```

So the implemented update is:

```
update_j = sigma / p_j / P(relevant)
```

This means relevant-class wins move more when the market has higher irrelevant baseline mass.

Example:

```
[irrelevant=0.8, A=0.1, B=0.1]
```

Since  $P(\text{relevant}) = 0.2$ , an A win is scaled by an extra:

```
1 / 0.2 = 5x
```

Economically, low relevance should usually dampen update confidence or volatility. Instead, the current formula amplifies it.

This also changes the expected baseline z-drift:

```
E[delta z_i] = p_i * sigma / p_i / P(relevant)
= sigma / P(relevant)
```

## H-12 DAlloc Amplifies Low-Relevance Markets

CONTINUED

Although `Softmax` cancels common drift for perfectly baseline-proportional sequences, finite market-facing sequences become more volatile as relevant mass decreases.

### — RECOMMENDATION

If `classProbs[i + 1]` is the unconditional baseline probability, remove the extra division by `probSum`:

```
updateMagnitudes[i] = scaledSigmaBase.divWad(classProbs[i + 1]);
```

If conditional relevant probabilities are intended, compute the conditional probability explicitly and inverse-weight by it:

```
P(class_i | relevant) = classProbs[i + 1] / probSum
```

Then use:

```
update_i = sigma / P(class_i | relevant)
```

Do not unintentionally scale by:

```
1 / (p_i * P(relevant))
```

### — RESOLUTION

Status: Resolved.

MEDIUM **M-01**

RESOLVED

# CGBM reverts on exact neutral predictions

|          |                                                                                                                                                                                 |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Math                                                                                                                                                                            |
| LOCATION | - `src/core/standalone/basefns/CGBM.sol:65` - `src/core/standalone/basefns/CGBM.sol:66` - `src/core/standalone/basefns/CGBM.sol:70` - `src/core/standalone/basefns/CGBM.sol:72` |
| REVIEW   | Main Review                                                                                                                                                                     |

## DESCRIPTION

`CGBM.zDelta` only rejects predictions greater than `WAD`, so `prediction == 0.5e18` is within the accepted input range. The code comment also says this value is treated as negative.

However, at `prediction == HALF_WAD`, `normP` becomes zero:

```
uint256 normP = pos ? (2 * prediction - WAD) : (WAD - 2 * prediction);  
uint256 m = normP.toInt256().powWad(ALPHA).toUint256();
```

For normal positive `ALPHA`, `powWad(0, ALPHA)` reverts because it attempts to evaluate `ln(0)`.

Impact: A valid neutral model output can block beacon updates. This is especially relevant when upstream predictors naturally emit `0.5` for uncertain binary predictions.

## RECOMMENDATION

Explicitly handle `prediction == HALF_WAD`. Depending on intended semantics, either:

- return `0` as a no-op z-space delta;
- reject it with a clear custom error; or
- add a small epsilon floor before exponentiation.

## RESOLUTION

Status: Resolved.

MEDIUM **M-02**

RESOLVED

# CGBM reverts for one-sided prediction streak

CATEGORY Math

LOCATION - `src/core/standalone/basefns/CGBM.sol:75` - `src/core/standalone/basefns/CGBM.sol:88` - `src/core/standalone/basefns/CGBM.sol:96` - `src/core/standalone/basefns/CGBM.sol:100` - `src/core/standalone/basefns/CGBM.sol:101`

REVIEW Main Review

## DESCRIPTION

CGBM computes an adaptive variance factor:

```
uint256 varianceFactor =  
(oneMinusPHat.mulWad(oneMinusPHat).mulWad(fSqUp) +  
pHat.mulWad(pHat).mulWad(fSqDown)).divWad(effectiveN);  
uint256 sqrtVariance = varianceFactor.sqrtWad();  
uint256 sigma = SCALED_SIGMA_BASE.divWad(sqrtVariance);
```

After a long streak of same-side predictions, fixed-point decay can drive the opposite side's EMA and squared-magnitude state to zero. At the same time, `pHat` can saturate to `0` or `1`. When the weighted variance term rounds to zero, `sqrtVariance` is zero and `divWad` reverts.

Impact: Valid repeated predictions can permanently or intermittently block updates depending on the current state. A model that produces a sustained one-sided run can push the component into a reverting state.

## RECOMMENDATION

Add a variance floor before division, for example:

```
if (varianceFactor < MIN_VARIANCE) varianceFactor = MIN_VARIANCE;
```

Alternatively, preserve nonzero priors in both variance sides or clamp `pHat` away from exact `0` and `1`.

## RESOLUTION

Status: Resolved.

MEDIUM **M-03**

RESOLVED

# ECDSA replay protection is signature-based and nonces are not enforced

CATEGORY Signatures  
LOCATION src/verifiers/ECDSA/ECDSAVerifier.sol  
REVIEW Main Review

## DESCRIPTION

`ECDSAVerifier.verify` stores replay state by canonical signature hash. The signed nonce is decoded but never checked against verifier state.

As a result, any unused old signature remains valid forever and can be submitted out of order. Additionally, if the signer produces two different valid signatures over the same digest, both can be accepted because replay state is keyed by the signature, not the signed message or nonce.

Impact: Path-dependent beacons can be updated with stale measurements, and signed messages do not have strict single-use or ordering guarantees.

## RECOMMENDATION

Track used message digests or used nonces instead of used signature hashes. If update ordering matters, store `lastNonce` and require strictly increasing nonces. Consider adding signed deadlines or timestamps for expiry.

## RESOLUTION

Status: Resolved.

MEDIUM **M-04**

RESOLVED

# Public one-time binding lets attackers permanently grief unbound mutable components

CATEGORY Logical Error

LOCATION - `src/core/base/CallerBound.sol:12` - `src/libraries/BindingLib.sol:21` - `src/verifiers/ECDSA/ECDSAVerifierFactory.sol:12` - `src/core/standalone/basefns/factories/CGBMFactory.sol:16`

REVIEW Main Review

## DESCRIPTION

`CallerBound.bind` is public and allows any address to bind an unbound component to any nonzero caller. Binding is permanent. Several factories deploy components in an unbound state, so users who deploy components first and compose them later can be front-run or grieved.

An attacker can call `bind(attacker)` on a newly created verifier/base function/group function before the intended beacon or manager constructor calls `BindingLib.bindComponent`. The later construction then reverts with `InvalidComponentBinding`, or the component remains unusable by the intended parent.

Impact: Any deployment flow that creates reusable components before atomically wiring them into a beacon can be permanently denied service.

## RECOMMENDATION

Bind components atomically during factory creation, or add an immutable intended binder/deployer and restrict `bind` to that address. Avoid exposing unbound mutable components from public factories unless the binder is also fixed at creation time.

## RESOLUTION

Status: Resolved.

MEDIUM **M-05**

RESOLVED

# Missing touch On Module Updates

CATEGORY Configuration

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/main/src/Perp.sol>

REVIEW Main Review

## DESCRIPTION

Some config functions influence params that are not immediately updated and instead are updated with the next `touch`, like for example `setFundingModule`.

This is unexpected behavior and could have bad consequences if a fast update is needed as the market is for example in a dangerous state.

## RECOMMENDATION

Consider to call `touch` in functions that will influence params that are updated on the next `touch` like for example `setFundingModule`.

## RESOLUTION

Status: Resolved.

MEDIUM **M-06**

RESOLVED

# Missing Check In Liquidation Flow

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/main/src/Perp.sol>

REVIEW Main Review

## DESCRIPTION

There are no checks at the end of liquidation flows that the health of the given position increased.

## RECOMMENDATION

Consider to add these best practice checks to prevent potential exploits in the liquidation path.

## RESOLUTION

Status: Resolved.

MEDIUM **M-07**

ACKNOWLEDGED

# Fee withdrawals bypass bad-debt socialization

CATEGORY Logical Error  
LOCATION `src/libraries/PerpLogic.sol:1105`  
REVIEW Main Review

## — DESCRIPTION

Margin payouts are routed through `transferMargin()`, which applies `socializeLoss()` when `badDebt > 0`. This haircuts outgoing payouts and uses the haircut to reduce outstanding bad debt.

However, `collectCreatorFees()` and `collectProtocolFees()` transfer the full fee buckets directly:

```
SafeTransferLib.safeTransfer(USDC, recipient, creatorFees);  
SafeTransferLib.safeTransfer(USDC, recipient, protocolFees);
```

These paths do not call `socializeLoss()`. As a result, creator/protocol fees can be withdrawn in full while ordinary margin payouts are being socialized.

This allows fee recipients to avoid contributing to bad-debt recapitalization and shifts more loss to remaining margin holders.

## — RECOMMENDATION

Apply bad-debt socialization to creator and protocol fee withdrawals, or use fee buckets to repay outstanding `badDebt` before transferring any remainder.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

RESOLVED

# Liquidation Fee Excluded From Eligibility

CATEGORY Rewards

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/interfaces/modules/IFees.sol#L22-L24>

REVIEW Main Review

## DESCRIPTION

`IFees.sol` states that the liquidation fee "contributes to liquidation eligibility", which is a good / more conservative approach. However, `liquidateTaker` checks whether a position is liquidatable before calculating or subtracting the liquidation fee.

## RECOMMENDATION

Calculate the liquidation fee before the eligibility check and include it in the health evaluation.

## RESOLUTION

Status: Resolved.

MEDIUM **M-09**

RESOLVED

# Irrelevant CAlloc Reverts

CATEGORY Math  
LOCATION src/core/group/groupfns/ContinuousAllocation.sol  
REVIEW Main Review

## DESCRIPTION

When a `ContinuousAllocation` prediction places all mass on the irrelevant class, there is no relevant-class allocation signal. The expected behavior is decay-only, or a no-op when `DECAY == WAD`.

Currently, a fully irrelevant prediction such as:

```
[1e18, 0, 0]
```

makes `predSum == 0`, and the function attempts to divide by zero while normalizing relevant classes. This causes an unintended math revert instead of explicit decay-only handling.

If fully irrelevant predictions are valid model outputs, this can block index updates and leave markets using stale index state.

## RECOMMENDATION

Add an explicit `predSum == 0` branch before normalization.

If the intended behavior is decay-only:

```
if (predSum == 0) {  
  for (uint256 i; i < NUM_RELEVANT_CLASSES; i++) {  
    oldZSpaceIndices[i] = DECAY.sMulWad(oldZSpaceIndices[i]);  
  }  
  return oldZSpaceIndices;  
}
```

When `DECAY == WAD`, this naturally becomes a no-op.

## RESOLUTION

Status: Resolved.

MEDIUM **M-10**

RESOLVED

# GMNormalize Clamp Breaks GM Invariant

CATEGORY Math  
LOCATION src/core/group/transforms/GMNormalize.sol  
REVIEW Main Review

## DESCRIPTION

`GMNormalize` is intended to map z-space vectors into relative dominance indices whose geometric mean equals `INDEX_SCALE`.

It does this by subtracting the mean z-value, then exponentiating each centered value:

```
index_i = INDEX_SCALE * exp(z_i - mean(z))
```

This works because the centered values sum to zero:

```
sum(z_i - mean(z)) = 0
```

therefore:

```
product(exp(z_i - mean(z))) = exp(0) = 1
```

However, the implementation independently clamps each centered value before exponentiation:

```
int256 centered = zSpaceIndices[i] - zMean;  
centered = centered.clamp(EXP_UNDERFLOW, EXP_OVERFLOW);  
indices[i] = centered.expWad().toUint256().mulDiv(INDEX_SCALE, WAD);
```

Once any centered value exceeds the clamp range, the clamped centered values may no longer sum to zero. As a result, the geometric mean of the returned indices no longer equals `INDEX_SCALE`.

Example:

```
centered before clamp = [-30, 10, 20]  
sum = 0
```

After clamping to `[-20, 20]`:

```
centered after clamp = [-20, 10, 20]  
sum = 10
```

The output product becomes:

**M-10** GMNormalize Clamp Breaks GM Invariant

CONTINUED

```
exp(-20) * exp(10) * exp(20) = exp(10)
```

so the geometric mean is:

```
exp(10 / 3)
```

instead of **1**.

This is not a small fixed-point rounding issue. It changes the transform's core invariant once z-space dispersion exceeds the clamp range.

The condition is reachable through normal group functions. For example, `DiscreteAllocation` can repeatedly increment the winning class z-space value when `DECAY == WAD`, and `ContinuousAllocation` can accumulate directional deltas over time with high decay. Once the z-space spread exceeds `20e18`, `GMNormalize` can return indices outside the intended constant-geometric-mean surface.

Since markets rely on these dominance indices, a broken normalization invariant can cause downstream pricing, funding, collateral, or liquidation logic to consume mathematically invalid group index values.

**— RECOMMENDATION**

Consider using a normalization routine that preserves the zero-sum centered condition after any bounding operation.

**— RESOLUTION**

Status: Resolved.

MEDIUM **M-11**

RESOLVED

# Blacklisted Users Are Not Liquidatable

|          |                                                                                                                                                                                                         |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | DoS                                                                                                                                                                                                     |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/main/src/Perp.sol">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/main/src/Perp.sol</a> |
| REVIEW   | Main Review                                                                                                                                                                                             |

## — DESCRIPTION

The liquidation flows will transfer remaining USDC margin to the owner of the position.

If this owner is blacklisted by USDC the transfer will fail and therefore the whole liquidation transaction reverts.

This could lead to a position accruing more and more bad debt without anyone being able to liquidate it. A dangerous state for the market.

## — RECOMMENDATION

Consider to put the USDC transfer in a try catch block and in case of a revert, escrow the USDC margin so that the owner can claim it later.

## — RESOLUTION

Status: Resolved.

MEDIUM **M-12**

RESOLVED

# Predicted UniV4 Pools Can Be Pre-Initialized

|          |                                                                                                                                                                                                                                                                                                                 |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | DoS                                                                                                                                                                                                                                                                                                             |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/PerpFactory.sol#L43-L110">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/PerpFactory.sol#L43-L110</a> |
| REVIEW   | Main Review                                                                                                                                                                                                                                                                                                     |

## DESCRIPTION

`createPerp` deterministically derives accounting-token clone addresses from the caller and salt, then uses those addresses to construct the UniV4 `PoolKey`. Because the caller and salt are visible in a pending `createPerp` transaction, an attacker can compute the same future `PoolKey` before the transaction executes.

The attacker can then call `POOL_MANAGER.initialize(predictedPoolKey, attackerSqrtPrice)` directly and once the pool is initialized, the legitimate factory call later reverts at `POOL_MANAGER.initialize(...)` with `PoolAlreadyInitialized`.

This permanently blocks that specific (creator, salt) market. The creator can choose a new salt, but a mempool attacker can repeat the grieving attack against public launch attempts.

## RECOMMENDATION

Add UniV4's `BEFORE_INITIALIZE_FLAG` to the required hook flags and implement `beforeInitialize` in `PerpGuardHook` to only allow initialization from `PERP_FACTORY`.

## RESOLUTION

Status: Resolved.

MEDIUM **M-13**

RESOLVED

# Same-Block Beacon EMA Desync

CATEGORY Oracle

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/libraries/PerpLogic.sol#L580>

REVIEW Main Review

## DESCRIPTION

`PerpLogic accrue()` always reads the current beacon index, but `calcEma()` returns the previous EMA unchanged when `dt == 0`.

As a result, if a beacon update and a Perp action happen in the same block, the action can consume the new spot index while still using the old index EMA.

A malicious actor can intentionally create this state around a large beacon update:

- first front-run the beacon update with `touch()`, setting `lastTouch` to the current block while the old beacon index is still active
- then let the beacon update execute
- then back-run the beacon update with a trade, margin adjustment, or liquidation. Because `dt == 0`, the back-run action uses the new spot index while the EMA remains based on the old index

This can create inconsistent pricing and liquidation inputs for that block.

## RECOMMENDATION

Be aware about this and consider making spot index and EMA consumption consistent for same-block updates. For example, by updating the EMA consistently when a new beacon index is consumed in a `dt == 0` block.

## RESOLUTION

Status: Resolved.

MEDIUM **M-14**

RESOLVED

# DAlloc Irrelevant Wins Mutate State

CATEGORY Math  
LOCATION src/core/group/groupfns/DiscreteAllocation.sol  
REVIEW Main Review

## DESCRIPTION

When the irrelevant class wins in `DiscreteAllocation`, there is no relevant allocation signal. The whitepaper states that no update is performed for irrelevant observations.

The implementation applies decay before checking whether the irrelevant class won:

```
if (DECAY != INT_WAD) {  
  for (uint256 i; i < NUM_RELEVANT_CLASSES; i++) {  
    oldZSpaceIndices[i] = DECAY.sMulWad(oldZSpaceIndices[i]);  
  }  
}  
  
if (maxIdx == 0) {  
  if (DECAY == INT_WAD) revert IrrelevantPrediction();  
}
```

As a result:

- if `DECAY < WAD`, irrelevant observations mutate all z-space indices through decay;
- if `DECAY == WAD`, irrelevant observations revert.

If irrelevant observations are intended to be true no-ops, the current behavior is incorrect. Even without a directional class winner, irrelevant observations can move market-facing `Softmax` allocations back toward neutral through decay.

## RECOMMENDATION

Define the intended irrelevant-observation semantics explicitly.

If irrelevant observations should be true no-ops, check `maxIdx == 0` before applying decay and return unchanged z-space:

```
if (maxIdx == 0) {  
  return oldZSpaceIndices;  
}
```

**M-14** DAlloc Irrelevant Wins Mutate State

CONTINUED

If decay-only is intended, document that irrelevant observations are not no-ops and should decay latent state. In that case, avoid reverting when `DECAY == WAD` ; simply return unchanged state.

**— RESOLUTION**

Status: Resolved.

MEDIUM **M-15**

RESOLVED

# setBeacon ( ) Does Not Accrue Before Switching

CATEGORY Oracle

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/Perp.sol#L230-L234>

REVIEW Main Review

## DESCRIPTION

`Perp.setBeacon()` replaces `modules.beacon` without first accruing market state against the old beacon. `accrue()` later computes elapsed time from `s.rates.lastTouch` and reads the current beacon through `env.modules.beacon.index()`. If the market was idle before the beacon switch, the next accrual applies the new beacon index over the entire elapsed period since the previous touch, even though the old beacon was active during that time.

This can retroactively distort EMA, mark price, funding accrual, utilization fee accrual, and liquidation state.

## RECOMMENDATION

Consider to call `touch()` before replacing the `beacon`.

## RESOLUTION

Status: Resolved.

MEDIUM **M-16**

RESOLVED

# Liquidation Fees Use Post-Liquidation Price

CATEGORY Math

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/Perp.sol#L329-L334>

REVIEW Main Review

## DESCRIPTION

`liquidateTaker()` uses two different mark prices during the same liquidation flow.

First, the function accrues state and checks whether the taker position is liquidatable using the pre-liquidation snapshot mark price. This determines whether the liquidation is allowed to proceed.

After that check passes, the function executes the liquidation swap. This swap changes the AMM price, updates open interest, refreshes EMAs and rates, and then recomputes the mark price. The liquidation fee is calculated from this new post-swap mark price.

As a result, liquidation eligibility is based on the pre-swap mark price, while the liquidation fee is charged using a post-swap mark price that was moved by the liquidation itself.

This makes the liquidation action affect its own fee basis. For long liquidations, the liquidation swap sells perp into the AMM and can lower the post-swap mark, reducing the liquidation fee. For short liquidations, the liquidation swap buys perp from the AMM and can raise the post-swap mark, increasing the liquidation fee.

## RECOMMENDATION

Consider to use a consistent price basis for both liquidation eligibility and liquidation fee calculation.

## RESOLUTION

Status: Resolved.

MEDIUM **M-17**

RESOLVED

# Unnecessary Maker Position Close DoS

|          |                                                                                                                                                                                                                                                                                                                         |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | DoS                                                                                                                                                                                                                                                                                                                     |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/libraries/PerpLogic.sol#L209">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/libraries/PerpLogic.sol#L209</a> |
| REVIEW   | Main Review                                                                                                                                                                                                                                                                                                             |

## — DESCRIPTION

When a maker removes all liquidity, `adjustMaker()` first settles accrued maker funding/utilization fees directly against `pos.margin`:

```
pos.margin = (pos.margin.toInt256() + p.marginDelta + lpFees.toInt256() -  
timeFees.total()).toUInt256().toUInt128();
```

If accrued funding owed by the maker exceeds the position's settled margin, this conversion to `uint256` reverts unless the maker supplies additional margin in the same call. Later, in the full close/conversion branch, the protocol only nets `pos.margin + pos.delta.amount1()` before converting residual perp exposure into a taker position. Residual perp PnL is not usable to cover the funding debt during the maker close step.

As a result, a maker position can be blocked from closing when its cash margin is insufficient, even though the remaining perp exposure could potentially be converted to a taker position and closed immediately to settle the debt. The maker may be forced to acquire and deposit extra USDC only to unblock the close, paying external swap/bridge/transaction costs and taking on extra operational friction.

## — RECOMMENDATION

Consider to allow full maker exits to account for the complete close-equity path instead of requiring nonnegative settled margin before conversion.

One approach is to support an atomic "remove maker liquidity and close residual taker exposure" flow, where funding debt, residual USD balance, residual perp close proceeds, and swap fees are netted before enforcing final nonnegative equity.

## — RESOLUTION

Status: Resolved.

MEDIUM **M-18**

RESOLVED

## Users can avoid socialized losses

CATEGORY Gaming  
LOCATION src/libraries/PerpLogic.sol:834  
REVIEW Main Review

### — DESCRIPTION

Bad debt is created only when a full liquidation/close resolves with negative equity:

```
uint256 remainingEquity = accountBadDebt(s, equity);
```

If insurance is insufficient, the uncovered loss is added to `solvencyState.badDebt`.

Future margin withdrawals then call:

```
socializeLoss(s.solvencyState, marginDelta.abs())
```

and receive only:

```
withdrawal - socialized haircut
```

However, before the bad-debt transaction executes, `badDebt == 0`, so withdrawals are not haircut. A user who observes a pending liquidation expected to create bad debt can attempt to close/withdraw first and avoid the loss.

Example

1. Market has 100 USDC total margin and 0 badDebt.
2. Position A is liquidatable and will create 10 USDC bad debt after insurance is exhausted.
3. User B sees the pending liquidation and closes/withdraws 20 USDC first.
4. Since badDebt is still 0, User B receives the full 20 USDC.
5. Position A liquidation executes and records 10 USDC badDebt.
6. Remaining users' future withdrawals are haircut to repay the 10 USDC loss.

The loss is therefore not socialized across all users exposed at the time the bad debt was economically inevitable. It is socialized only across users who remain after the bad-debt transaction lands.

### — RECOMMENDATION

TBA

---

**M-18** Users can avoid socialized losses

CONTINUED

---

— RESOLUTION

Status: Resolved.

MEDIUM **M-19**

ACKNOWLEDGED

# Maker liquidation can get stuck

CATEGORY Validation

LOCATION – `src/libraries/PerpLogic.sol:272` – `src/libraries/PerpLogic.sol:291` – `src/libraries/PerpLogic.sol:303` – `src/libraries/PerpLogic.sol:1088`

REVIEW Main Review

## DESCRIPTION

`liquidateMaker` can reject both full and partial liquidation for the same unsafe maker:

- Full liquidation can revert if removing all maker capacity makes OI exceed remaining capacity.
- Partial liquidation can revert if the remaining-maker branch leaves `netMargin < 0`.

This can leave a liquidatable maker position open with no executable liquidation amount.

Affected Code

Capacity removal enforces utilization:

```
updateCapacity(s, cap, false);
```

Partial maker liquidation rejects negative residual margin:

```
if (maker.liquidity != 0) {  
  if (netMargin < 0) revert NegativeMargin();  
}
```

Full liquidation handles bad debt instead:

```
uint256 remainingEquity = accountBadDebt(s, equity);
```

Impact: Unsafe maker exposure may remain in the system, delaying bad-debt resolution and allowing losses to grow.

## RECOMMENDATION

Preferred fixes:

- Cap partial liquidation fees so they cannot push residual maker margin below zero.
- Require accepted partial liquidations to improve maker health or reduce risk.
- For unrecoverable makers, allow a path that accounts bad debt instead of reverting indefinitely.

---

**M-19** Maker liquidation can get stuck

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

MEDIUM **M-20**

RESOLVED

# Non-Standard EIP-712 Array Hash

|          |                                                                                                                                                                                                                                                                                                                     |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Signatures                                                                                                                                                                                                                                                                                                          |
| LOCATION | <a href="https://github.com/GuardianOrg/beacons-team1-1779719143808/blob/d61aca0a3b41ee61676208eeae7710c71a73058b/src/verifiers/ECDSA/ECDSAVerifier.sol#L49">https://github.com/GuardianOrg/beacons-team1-1779719143808/blob/d61aca0a3b41ee61676208eeae7710c71a73058b/src/verifiers/ECDSA/ECDSAVerifier.sol#L49</a> |
| REVIEW   | Main Review                                                                                                                                                                                                                                                                                                         |

## DESCRIPTION

`ECDSAVerifier` defines the signed payload as an EIP-712-style struct with a dynamic array: `Measurement(uint256[] measurement, uint256 nonce)`.

However, the verifier hashes the array field directly instead of first hashing the encoded array contents according to EIP-712 array rules.

Under EIP-712, array values must be encoded as the `keccak256` hash of the concatenated encoded array contents before being included in the struct hash. Therefore, standard `eth_signTypedData` tooling may compute a different digest than the contract expects.

This does not directly allow signature forgery, but it can break oracle liveness if off-chain signing infrastructure uses standard EIP-712 encoding while the verifier expects this custom encoding.

## RECOMMENDATION

Encode the `uint256[] measurement` field according to EIP-712 array rules before hashing the struct, or clearly document that the verifier uses a custom digest scheme and require signers to use `ECDSAVerifier.digest()` exactly.

## RESOLUTION

Status: Resolved.

LOW **L-01**

RESOLVED

# CGBM can accept degenerate parameter sets

|          |                                      |
|----------|--------------------------------------|
| CATEGORY | Validation                           |
| LOCATION | src/core/standalone/basefns/CGBM.sol |
| REVIEW   | Main Review                          |

## DESCRIPTION

CGBM validates raw constructor inputs, but it does not validate the derived fixed-point state used by the variance calculation.

Examples of accepted degenerate configurations include:

- `decay == 0`, which clears prior state on each update;
- very small nonzero `decay`, which can behave like zero after WAD rounding;
- very small nonzero `initialSigmaRatio`, where `initialSigmaRatio.mulWad(initialSigmaRatio)` rounds to zero;
- very small `sigmaBase * scalingFactor`, where `SCALED_SIGMA_BASE` can round to zero.

These configurations can make the first non-neutral update revert because `varianceFactor` becomes zero and `SCALED_SIGMA_BASE.divWad(sqrtVariance)` divides by zero.

Fuzzing found accepted constructor configurations that reverted on the first `zDelta(WAD)` with `DivWadFailed()`.

## RECOMMENDATION

Validate derived values, not only raw inputs. Reject zero or dust `decay`, reject zero derived variance priors, reject zero `SCALED_SIGMA_BASE`, and apply a minimum variance floor before computing `sigma`.

## RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

# DGBM can accept degenerate parameter sets

|          |                                      |
|----------|--------------------------------------|
| CATEGORY | Validation                           |
| LOCATION | src/core/standalone/basefns/DGBM.sol |
| REVIEW   | Main Review                          |

## DESCRIPTION

DGBM accepts parameter combinations that make every valid update return zero z-space delta.

Examples include:

- `decay == 0`, which clears prior counts before the current observation is added;
- very small nonzero `decay`, where `DECAY.mulWad(nUp)` and `DECAY.mulWad(nDown)` round to zero;
- nonzero `sigmaBase` and `scalingFactor` whose WAD product rounds `SCALED_SIGMA_BASE` to zero;
- dust parameter combinations where the final `SCALED_SIGMA_BASE.mulWad(...)` rounds down to zero even though the mathematical delta is nonzero.

The result is an apparently functional base function that accepts binary predictions but never moves the beacon index.

Impact: A beacon can continue accepting updates while its index remains unchanged. This is a silent failure mode rather than an immediate revert.

## RECOMMENDATION

Validate derived values after WAD arithmetic. Reject zero or dust decay values, reject `SCALED_SIGMA_BASE == 0`, and enforce a minimum update magnitude if zero movement is not intended. Alternatively, document that dust parameter choices intentionally produce inert behavior.

## RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

# Hardcoded chain-specific addresses create deployment risk

CATEGORY Configuration  
LOCATION src/libraries/Constants.sol  
REVIEW Main Review

## DESCRIPTION

src/libraries/Constants.sol hardcodes Arbitrum Sepolia addresses:

```
IPoolManager constant POOL_MANAGER =  
IPoolManager(0xFB3e0C6F74eB1a21CC1Da29aeC80D2Dfe6C9a317);  
address constant USDC = 0xBEF280BefeE2Cb28c20D1E4Cc1da999B4DA0f1fD;
```

Mainnet values are commented out, while the deployment docs mention multiple chains. Deploying unchanged to any other chain can cause the system to use incorrect addresses or fail entirely.

## RECOMMENDATION

Make POOL\_MANAGER and USDC constructor/configuration parameters, or introduce explicit per-chain build profiles.

## RESOLUTION

Status: Resolved.

LOW **L-05**

RESOLVED

# Timelock defaults to zero, allowing immediate admin changes

**CATEGORY** Configuration**LOCATION** <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/Perp.sol#L77-L78>**REVIEW** Main Review

## DESCRIPTION

`Perp.timelock` defaults to `0`. The owner can call `submit(data)` and then execute the timelocked function immediately because `executableAt[data] == block.timestamp`.

This affects risk-critical admin actions such as replacing pricing, funding, fees, margin ratio, price impact, and beacon modules.

## RECOMMENDATION

Set a non-zero initial timelock in the constructor or factory.

## RESOLUTION

Status: Resolved.

LOW **L-06**

RESOLVED

# Module Registry Is Not Enforced

CATEGORY Configuration  
LOCATION src/PerpFactory.sol:48  
REVIEW Main Review

## DESCRIPTION

ModuleRegistry allows the owner to register approved modules, but `PerpFactory.createPerp()` accepts arbitrary module addresses and does not check them against the registry.

The registry comment explicitly states that it is informational only:

perps may still be created with modules that are not registered here

Since modules control pricing, funding, fees, margin ratios, and price-impact behavior, users may incorrectly assume factory-created markets use approved modules.

## RECOMMENDATION

If registered modules are intended to be trusted/canonical, enforce registry checks in `PerpFactory.createPerp()` for each module type.

If markets are intentionally permissionless, document clearly that the registry is informational and that users/integrators must verify each market's module set independently.

## RESOLUTION

Status: Resolved.

LOW **L-07**

RESOLVED

# Signed Fixed-Point Rounds Toward Zero

CATEGORY Math  
LOCATION src/libraries/SignedFixedPointMathLib.sol:18  
REVIEW Main Review

## DESCRIPTION

sFullMulDiv() computes the absolute unsigned result first, then reapplies the sign:

```
int256 absResult = SafeCastLib.toInt256(  
    FixedPointMathLib.fullMulDiv(unsignedA, unsignedB, denominator)  
);  
result = negative ? -absResult : absResult;
```

For negative values, this rounds toward zero, not toward negative infinity.

As a result, roundUp=false does not provide floor-style rounding for negative results.

Impact appears limited to small accounting bias.

## RECOMMENDATION

Either document round-toward-zero semantics clearly, or implement true floor rounding for negative values when roundUp=false.

## RESOLUTION

Status: Resolved.

LOW **L-08**

RESOLVED

# Health Check Adds One To Position Value

CATEGORY Math  
LOCATION src/libraries/PerpLogic.sol:726  
REVIEW Main Review

## DESCRIPTION

`isHealthy()` divides by `posVal + 1`:

```
return (equity <= 0 ? 0 : equity.toUint256().fullMulDiv(E6, posVal + 1)) >=
```

This avoids division by zero, but it slightly biases margin ratio calculations downward, especially for very small positions.

## RECOMMENDATION

Use an explicit `posVal == 0` branch instead of adding one to every denominator.

## RESOLUTION

Status: Resolved.

LOW **L-09**

RESOLVED

# Liquidation Fees Are Socialized During Bad Debt

CATEGORY Rewards  
LOCATION src/libraries/PerpLogic.sol:1109  
REVIEW Main Review

## — DESCRIPTION

Liquidation fees are paid through `transferMargin()`:

```
transferMargin(s, -liqFeeAmt.toInt128(), liquidationFeeRecipient);
```

When `badDebt > 0`, `transferMargin()` applies `socializeLoss()` to outgoing transfers.

This means liquidator compensation can be haircut during insolvency events.

This may be intentional as part of global recapitalization, but it weakens liquidation incentives precisely when liquidations are most important.

## — RECOMMENDATION

Consider exempting liquidation fees from socialization, paying them from insurance first, or explicitly documenting that liquidator rewards are subject to bad-debt haircuts.

## — RESOLUTION

Status: Resolved.

LOW **L-10**

RESOLVED

# WeightedSum accumulates rounding error per term

|          |                                                 |
|----------|-------------------------------------------------|
| CATEGORY | Math                                            |
| LOCATION | src/core/composite/composers/WeightedSum.sol:30 |
| REVIEW   | Main Review                                     |

## DESCRIPTION

WeightedSum.compose() applies mulWad inside the accumulation loop, dividing by WAD on every iteration. Each division truncates independently, producing up to one wei of rounding error per term. The total error grows linearly with the number of reference beacons.

```
for (uint256 i = 0; i < WEIGHT_COUNT; i++) {  
    index += weights[i].mulWad(indices[i]); // ← divides by WAD each iteration  
}
```

mulWad(a, b) computes  $a * b / \text{WAD}$ , discarding the remainder. Across N terms this gives:

$$\text{result} = \sum \text{floor}(\text{weights}[i] * \text{indices}[i] / \text{WAD})$$

The exact result is:

$$\text{exact} = \text{floor}(\sum (\text{weights}[i] * \text{indices}[i]) / \text{WAD})$$

The difference is up to  $N - 1$  wei, where N is WEIGHT\_COUNT.

## RECOMMENDATION

Accumulate the unscaled numerator and divide once after the loop:

```
function compose(uint256[] memory indices) external view returns (uint256 index) {  
    if (indices.length != WEIGHT_COUNT) revert InvalidIndexLength();  
    uint256 numerator;  
    for (uint256 i = 0; i < WEIGHT_COUNT; i++) {  
        numerator += weights[i] * indices[i];  
    }  
    index = numerator / WAD;  
}
```

**L-10** WeightedSum accumulates rounding error per term

CONTINUED

This reduces the rounding error to at most one wei regardless of `WEIGHT_COUNT`. Overflow is not a concern — with weights and indices both bounded by WAD (1e18), each product is at most 1e36, and uint256 accommodates up to ~1.15e77.

**— RESOLUTION**

Status: Resolved.

LOW **L-11**

RESOLVED

# DAlloc Ties Bias Lower Classes

|          |                                                |
|----------|------------------------------------------------|
| CATEGORY | Math                                           |
| LOCATION | src/core/group/groupfns/DiscreteAllocation.sol |
| REVIEW   | Main Review                                    |

## — DESCRIPTION

`DiscreteAllocation` selects the winning class using a strict greater-than comparison:

```
if (prediction[i] > maxVal) {  
    maxVal = prediction[i];  
    maxIdx = i;  
}
```

The initial winner is class `0`, the irrelevant class:

```
(uint256 maxVal, uint256 maxIdx) = (prediction[0], 0);
```

Therefore, exact ties always keep the earlier class.

Examples:

```
[0.5e18, 0.5e18, 0]
```

selects irrelevant.

```
[0, 0.5e18, 0.5e18]
```

selects class 1, not class 2.

Repeated ambiguous predictions can therefore bias `Softmax` allocation toward lower-index relevant classes even when the model assigns equal probability to tied classes.

## — RECOMMENDATION

Define explicit tie semantics.

Possible approaches:

- revert on exact ties;
- treat ties as no-op / irrelevant;
- split the update across tied relevant classes;
- use deterministic tie-breaking only if documented and accepted as an intentional bias.

---

**L-11** DAlloc Ties Bias Lower Classes

---

CONTINUED

---

**— RESOLUTION**

Status: Resolved.

LOW **L-12**

RESOLVED

# Beacon Index Is Downcast Without Bounds Check

|          |                                                                                                                                                                                                                                                                                                                         |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Validation                                                                                                                                                                                                                                                                                                              |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/libraries/PerpLogic.sol#L571">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/libraries/PerpLogic.sol#L571</a> |
| REVIEW   | Main Review                                                                                                                                                                                                                                                                                                             |

## DESCRIPTION

`IBeacon.index()` returns a `uint256`, but `PerpLogic accrue()` immediately converts it to `uint128`:  
`snap.spots.index = uint128(env.modules.beacon.index());`

If a beacon would ever returns a value above `type(uint128).max`, the value is silently truncated instead of reverting.

## RECOMMENDATION

Consider to add a bounds check before converting the beacon index to `uint128`.

## RESOLUTION

Status: Resolved.

LOW **L-13**

RESOLVED

# Unbounded Transform Silently Saturates

CATEGORY Math  
LOCATION `src/core/standalone/transforms/Unbounded.sol`  
REVIEW Main Review

## DESCRIPTION

Unbounded` is intended to expose GBM-style index behavior:

```
index = INITIAL_INDEX * exp(z)
```

However, the implementation clamps `zSpaceIndex` before exponentiation:

```
zSpaceIndex = zSpaceIndex.clamp(EXP_UNDERFLOW, EXP_OVERFLOW);  
return INITIAL_INDEX.mulWad(zSpaceIndex.expWad().toUint256());
```

Since `EXP_UNDERFLOW = -20e18` and `EXP_OVERFLOW = 20e18`, the transform is not actually unbounded. Once z-space exceeds the clamp range, additional z-space movement no longer changes the public index.

For example:

```
z = 20e18 -> index = INITIAL_INDEX * exp(20)  
z = 30e18 -> index = INITIAL_INDEX * exp(20)  
z = 100e18 -> index = INITIAL_INDEX * exp(20)
```

The same applies in the negative direction below `-20e18`.

This can cause the market-facing index to saturate while the base function continues accumulating latent z-space movement. The issue is low severity because the clamp range is very wide, but it can become relevant for aggressive parameters or long-lived one-sided markets.

## RECOMMENDATION

Avoid silently saturating an "unbounded" transform.

Either:

---

**L-13** Unbounded Transform Silently SaturatesCONTINUED

---

- revert when `zSpaceIndex` is outside the safe exponentiation range;
- document the transform as saturating rather than unbounded; or
- use an exponentiation/scaling strategy that supports the intended valid z-space range without silently clamping market-facing outputs.

**— RESOLUTION**

Status: Resolved.

INFORMATIONAL **I-01**

RESOLVED

# Unused Measurement Scale

CATEGORY Superfluous Code  
LOCATION GLOBAL  
REVIEW Main Review

## DESCRIPTION

`Threshold` and `Argmax` accept `measurementScale` in their constructors via `BasePreprocessor`, but the value is unused by their logic.

## RECOMMENDATION

Consider to remove unused code to follow best practices.

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-02

RESOLVED

# Mixed Composite Beacon Updates

|          |                                                                                                                                                                                                                                       |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Warning                                                                                                                                                                                                                               |
| LOCATION | <a href="https://github.com/GuardianOrg/beacons-team1-1779719143808/blob/main/src/core/composite/CompositeBeacon.sol">https://github.com/GuardianOrg/beacons-team1-1779719143808/blob/main/src/core/composite/CompositeBeacon.sol</a> |
| REVIEW   | Main Review                                                                                                                                                                                                                           |

## DESCRIPTION

`CompositeBeacon.index()` computes the composite value by reading each child beacon's current `index()` live. Since child beacons update independently, the composite can temporarily reflect a mix of updated and non-updated constituents.

For example, if a composite tracks two child beacons and only one child has received a new oracle update, the composite value will include the new value for one child and the old value for the other. A malicious actor may be able to front-run or back-run child oracle updates and perform trades, liquidations, or other market actions while the composite is in this mixed-update state.

## RECOMMENDATION

Be aware that `CompositeBeacon` does not provide atomic constituent snapshots and consider to document this behavior.

## RESOLUTION

Status: Resolved.

## CRITICAL C-01

RESOLVED

# Zero-For-One Tick Replay Can Skip Current Boundary

CATEGORY Logical Error  
LOCATION src/libraries/PerpTickLogic.sol  
REVIEW Remediation Review 1

## DESCRIPTION

PerpCity implements custom LP-fee and funding accounting by replaying tick crossings after a Uniswap V4 swap.

For zero-for-one swaps, the replay starts from `snap.tick - 1`:

```
int24 tick = zeroForOne ? snap.tick - 1 : snap.tick;
```

If the current tick is itself an initialized boundary, Uniswap can cross that boundary first during the real swap. Starting the custom replay from `snap.tick - 1` skips that boundary in PerpCity's accounting.

When a tick is crossed during replay, PerpCity updates custom outside-growth values:

```
tickInfo.cumlFunding0ppX96 = s.cumls.fundingX96 - tickInfo.cumlFunding0ppX96;  
tickInfo.cumlFundingDivSqrtP0ppX96 = s.cumls.fundingDivSqrtPX96 -  
tickInfo.cumlFundingDivSqrtP0ppX96;  
tickInfo.lpFeeGrowthOutsideX128 = s.cumls.lpFeeGrowthGlobalX128 -  
tickInfo.lpFeeGrowthOutsideX128;
```

Skipping a boundary therefore leaves `lpFeeGrowthOutsideX128` and funding-outside values stale. Later fee-growth-inside calculations can become inconsistent with maker checkpoints.

Impact: This can misallocate LP fees across adjacent ranges and break maker settlement.

One representative flow:

1. Maker A provides liquidity below an initialized boundary.
2. Maker B provides liquidity above or across that boundary.
3. Price moves upward onto the boundary.
4. A zero-for-one swap moves price back downward through the same boundary.
5. Uniswap crosses the current initialized tick.
6. PerpCity's replay starts from `snap.tick - 1` and skips that crossing.
7. Custom `lpFeeGrowthOutsideX128` remains stale.

**C-01** Zero-For-One Tick Replay Can Skip Current Boundary

CONTINUED

8. Maker fee growth can be credited to the wrong range.

9. An honest maker's `lpFeeGrowthInside` can move below its stored checkpoint.

`makerLpFeesAccrued` subtracts the stored checkpoint from current inside growth:

```
return (lpFeeGrowthInsideX128 - maker.lastLpFeeGrowthInsideX128)
.fullMulDiv(maker.liquidity, Q128);
```

If current inside growth moves backward, this unchecked subtraction can underflow into a huge value.

Maker settlement paths such as `adjustMaker`, `liquidateMaker`, and `backstopMaker` can then revert or produce incorrect accounting.

An attacker can use boundary liquidity and round-trip price movement to extract LP-fee margin that belongs to adjacent makers while leaving honest makers difficult or impossible to settle until future fee growth repairs the checkpoint gap.

**— RECOMMENDATION**

Replay zero-for-one tick crossings from the current tick when the current initialized boundary may be crossed, so PerpCity's custom accounting matches Uniswap's tick-crossing semantics..

**— RESOLUTION**

Status: Resolved.

## CRITICAL C-02

RESOLVED

# Dust maker liquidity hides residual exposure from OI and margin checks

CATEGORY Logical Error  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

When a maker removes nearly all liquidity but leaves nonzero dust, `adjustMaker()` remains in the maker branch. `valPnl()` uses only the remaining liquidity value as `posVal`, while residual `delta.amount0()` affects PnL but is not included in the margin denominator, and `updateOpenInterest()` is skipped until full conversion. This lets directional residual inventory remain off taker OI and avoid taker-style liquidation while collateral is withdrawn.

Impact: A maker can hide residual directional exposure behind dust liquidity, withdraw most collateral, and delay OI/liquidation accounting. The updated PoC demonstrates the hidden short surviving off short OI, withdrawal to near the minimum margin, liquidation blocked while dust remains, then adverse mark/funding moves leading to conversion, insurance consumption, and protocol bad debt.

## RECOMMENDATION

If residual `delta.amount0()` is nonzero after a liquidity reduction, force maker-to-taker conversion and call `updateOpenInterest()`, or include the residual taker notional in maker health valuation. Do not allow nonzero dust liquidity to suppress conversion for positions with directional inventory.

## RESOLUTION

Status: Resolved.

HIGH **H-01**

ACKNOWLEDGED

# Spot-Mark Backstops Can Seize Healthy Positions

CATEGORY Gaming  
LOCATION src/Perp.sol, src/libraries/PerpLogic.sol  
REVIEW Remediation Review 1

## DESCRIPTION

The backstop flows use the current AMM-derived mark price to decide whether a position is below the backstop threshold. If the check passes, the protocol transfers the entire position NFT to the backstopper.

For takers, `backstopTaker` computes health using:

```
(uint256 val, int256 pnl) = valPnl(pos.delta, markPrice(env.modules.pricing, snap.spots, snap.emas));  
if (isHealthy(netMargin + pnl, val, pos.backstopMarginRatio)) revert NotLiquidatable();
```

For makers, `backstopMaker` similarly computes maker health using `markPrice(...)`.

After the library call succeeds, `Perp` transfers the victim's NFT:

```
PerpLogic.backstopTaker(s, env(), posId, marginIn, positionRecipient);  
_transfer(ownerOf(posId), positionRecipient, posId);
```

The issue is that `markPrice` can be influenced by the live AMM spot price. An attacker can first move AMM spot with a taker trade, then immediately call `backstopTaker` or `backstopMaker` while the distorted spot is still live.

Unlike liquidation swaps, the backstop path does not execute a swap and therefore does not re-run `PriceImpactTooHigh`. It only reads the already-manipulated spot through `accrue()`.

## Attack Scenario

1. A victim has a long position that is healthy under the honest mark.
2. The attacker opens a short trade to push AMM spot downward within the allowed price-impact bound.
3. The lower AMM spot reduces the mark price used by backstop health checks.
4. The victim now appears below the backstop threshold.
5. The attacker calls `backstopTaker` with a small `marginIn`.

## H-01 Spot-Mark Backstops Can Seize Healthy Positions

CONTINUED

6. The protocol transfers the victim's position NFT to the attacker.
7. The attacker unwinds the AMM manipulation.
8. The seized NFT is now healthy again and belongs to the attacker.

The same pattern applies to maker positions through `backstopMaker`.

Impact: This enables permissionless theft of healthy or near-healthy position NFTs.

The impact is worse than ordinary liquidation-fee extraction because backstop transfers ownership of the full position, including its remaining margin and future PnL. A short-lived AMM spot manipulation can therefore become a direct position-seizure primitive.

The attack can be especially cheap when the attacker also supplies dominant liquidity in the affected AMM range, because the manipulation trade largely interacts with self-owned liquidity.

### — RECOMMENDATION

Do not allow instantaneous AMM spot to determine backstop seizure eligibility.

### — RESOLUTION

Status: Acknowledged.

HIGH **H-02**

RESOLVED

# Insurance Spend Skews Solvency

CATEGORY Logical Error  
LOCATION src/libraries/PerpLogic.sol  
REVIEW Remediation Review 1

## DESCRIPTION

Insurance fees are removed from `solvencyState.totalMargin` when they are charged on swaps, because those amounts are moved into `feeFund.insurance`.

When a later insolvent close or liquidation is covered by insurance, `accountBadDebt()` only reduces `feeFund.insurance`. It does not credit the covered amount back into `solvencyState.totalMargin` or otherwise reconcile the global solvency buckets.

This can make the tracked accounting buckets diverge from the actual USDC held by the market. The issue is lower confidence than a direct theft bug because the intended accounting model may treat insurance as a separate reserve rather than active margin, but the code should make that invariant explicit and keep aggregate collateral/liability accounting synchronized.

After insurance absorbs bad debt, later bad-debt socialization may use a distorted `totalMargin` base. This can lead to inaccurate haircuts, orphaned collateral accounting, or unexpected close/liquidation failures during stressed states.

## RECOMMENDATION

Define the solvency invariant for insurance-covered losses and enforce it in code. If insurance coverage restores a user/maker liability that should remain claimable in the margin pool, credit the covered amount back into `solvencyState.totalMargin`.

If insurance is intentionally outside margin accounting, the code should still keep `totalMargin`, `feeFund.insurance`, `badDebt`, and actual USDC balance consistent after insurance-covered liquidations.

## RESOLUTION

Status: Resolved.

HIGH **H-03**

ACKNOWLEDGED

# Taker Liquidation Sandwich

CATEGORY MEV  
LOCATION src/Perp.sol:349  
REVIEW Remediation Review 1

## DESCRIPTION

Voluntary taker opens and adjustments execute an AMM swap and then enforce `checkTakerAmountLimits()` against the user-provided quote-side limit. This gives users slippage protection over the realized `amount1`.

Taker liquidation uses the same AMM swap machinery to close or partially close the position, but the liquidation path does not accept or enforce a quote-side bound for the victim. It only requires the requested perp amount to be filled and relies on the configured post-swap price-impact band.

Because the AMM price can be moved before the liquidation transaction, a liquidator or searcher can manipulate the close quote, execute the liquidation while the victim accepts any realized `amount1`, and then unwind the manipulation. The post-swap price-impact check may not protect the victim from adverse execution inside the accepted band.

This can confiscate residual liquidation equity from the liquidated account, increase bad debt, and transfer value to the liquidator/searcher. The issue is High because liquidation is permissionless, the victim cannot provide slippage controls, and the forced trade directly determines the user's remaining equity.

## RECOMMENDATION

Add quote-side protection for forced taker closes.

The liquidation path should derive an acceptable `amount1` bound from a pre-liquidation mark/index reference, oracle-aware execution price, or liquidation-specific slippage rule, and revert if the realized AMM quote falls outside that bound. The bound should apply separately from the general post-swap price-impact band, because price-impact validity does not imply fair liquidation execution for the victim.

## RESOLUTION

Status: Acknowledged.

HIGH **H-04**

ACKNOWLEDGED

# Transient-price maker cycles under-account capacity below executable liquidity

CATEGORY Logical Error  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

`adjustMaker()` removes capacity by prorating the maker's existing stored capacity, but re-adds capacity by freshly calling `calcCapacity()` at the then-current AMM price. After a transient AMM price move, these operations are not inverse. A maker can push the AMM price, repeatedly remove/re-add liquidity to reduce stored long capacity, and then unwind the price-moving taker position; final V4 liquidity is unchanged but `s.cap.long` can remain below a fresh capacity recomputation at the restored price.

Impact: The updated PoC shows existing OI remains below both stored and freshly recomputed capacity, but stored capacity is artificially below recomputed executable capacity. A new long that would fit under fresh capacity accounting reverts with `LongUtilizationExceeded`.

## RECOMMENDATION

Make capacity debits and credits symmetric across maker adjustments. For example, recompute maker/global capacity from current liquidity and AMM price after each liquidity change, track capacity tranches by the price basis used on add, or otherwise ensure remove/re-add cycles cannot ratchet stored capacity below the capacity represented by unchanged pool liquidity.

## RESOLUTION

Status: Acknowledged.

HIGH **H-05**

RESOLVED

# Uninsured bad debt omitted from socialization claim base

CATEGORY Logical Error  
LOCATION —  
REVIEW Remediation Review 1

## — DESCRIPTION

When a liquidation or full close realizes negative equity that exceeds available insurance, `accountBadDebt()` records only `solvencyState.badDebt`. It does not add the realized unpaid winning claim to `solvencyState.totalMargin`, while later `socializeLossInternal()` haircuts outgoing claims against `badDebt / totalMargin` and `transferMargin()` debits the full nominal outgoing amount from `totalMargin`.

Impact: After uninsured bad debt is realized, subsequent profitable exits are over-haircutted relative to the full nominal claim base, and the accounting state can leave bad debt plus USDC stranded above recorded `totalMargin`. The updated PoC uses the actual `LossSocialized` close-equity amount after accrual/swap/fees and shows a 40,472,389-atom close claim charged 4,562,809 atoms versus 4,108,296 atoms under a `totalMargin + badDebt` claim base, with bad debt still remaining afterward. The effect scales with insolvency severity and can materially misallocate remaining collateral among creditors.

## — RECOMMENDATION

When booking uninsured bad debt, include the realized unpaid claim in the socialization claim base (for example by increasing `solvencyState.totalMargin` by the uninsured shortfall, or by computing haircuts against `totalMargin + badDebt` and debiting `totalMargin` consistently). Add invariants for post-bad-debt withdrawals conserving nominal claims and collateral.

## — RESOLUTION

Status: Resolved.

HIGH **H-06**

RESOLVED

# Full maker liquidation cannot book bad debt when residual perp remains

CATEGORY Logical Error  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

When full maker liquidation removes all liquidity, `liquidateMaker()` converts positions with residual `amount0` to takers but first requires `netMargin >= 0`. The bad-debt path is only reached when residual `amount0 == 0`, so a fee-insolvent maker with residual perp inventory reverts before `accountBadDebt()` can run. This is distinct from the prior dust-liquidity issue and from expected partial-liquidation negative-margin reverts.

Impact: Default fee/funding conditions can leave a liquidatable maker unable to be fully liquidated: the maker liquidity and residual exposure remain, and `solvencyState.badDebt` is not updated. Backstop can sometimes recapitalize later, but it is voluntary, has a stricter eligibility threshold, and does not provide the intended permissionless liquidation/bad-debt recognition path.

## RECOMMENDATION

For full maker removal with nonzero residual perp and insolvency, settle total equity through `accountBadDebt()` or an equivalent bad-debt path before conversion/closure instead of reverting solely because fee margin is negative.

## RESOLUTION

Status: Resolved.

HIGH **H-07**

ACKNOWLEDGED

# Stripping in-range liquidity while OOR capacity remains freezes underwater taker liquidations

CATEGORY Logical Error  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

Perp counts above-range maker liquidity as long capacity and only enforces aggregate `oi.long <= cap.long`. A maker can therefore provide OOR long capacity, let a taker open against in-range liquidity, then remove almost all in-range executable liquidity. `adjustMaker()` subtracts only the removed maker's stored capacity and `checkUtilization()` still passes due to the OOR capacity, even though the remaining book cannot execute full-size long exits or liquidations in the zero-for-one direction. This is distinguishable from the prior rejected OOR-capacity note because the updated PoC demonstrates an underwater taker liquidation path remaining utilization-valid but unenforceable.

Impact: Voluntary full closes and full `liquidateTaker()` calls can revert with `InsufficientLiquidityToFill`, leaving underwater long OI open and preventing bad-debt realization through the protocol's AMM liquidation path. Failed attempts also burn substantial gas traversing empty V4 tick words before Perp's exact-fill check reverts.

## RECOMMENDATION

Tie capacity used for utilization to liquidity executable for the relevant close/liquidation direction, prevent in-range removals that leave outstanding OI without executable depth, or add a bounded/fallback liquidation path for exact-fill failures.

## RESOLUTION

Status: Acknowledged.

HIGH **H-08**

ACKNOWLEDGED

# Bounded Drift Can Point Outward

CATEGORY Math  
LOCATION `src/core/standalone/transforms/Bounded.sol:43-50`  
REVIEW Remediation Review 1

## — DESCRIPTION

The whitepaper claims sigmoid curvature makes expected bounded movement point toward the midpoint when latent movement has zero conditional drift. Using the actual state-dependent branch probabilities and deltas, fuzzing finds reachable states where expected movement points outward instead.

For example:

```
bounds: [1, 101]
midpoint: 51
current index: 51.023447
expected next index: 51.023452
```

The index is already above its baseline but moves farther upward in expectation under the modeled baseline outcome distribution. The implemented composition does not universally satisfy the lemma's zero-latent-drift premise, and the sigmoid proof also requires care when possible updates cross its inflection point.

Repeated baseline-consistent observations can compound the wrong-way movement into material mark-price divergence. Since the beacon is the market's price source, this bias propagates directly into funding and liquidation eligibility without requiring forged oracle reports.

## — RECOMMENDATION

Calculate branch returns from one shared pre-observation state and explicitly enforce probability-weighted latent neutrality. If midpoint mean reversion is intended as a global protocol guarantee, also verify the final transformed branches directly:

```
z > 0: E[Bounded(z + deltaZ)] <= Bounded(z)
z < 0: E[Bounded(z + deltaZ)] >= Bounded(z)
```

**H-08** Bounded Drift Can Point Outward

CONTINUED

Apply state-dependent price-space calibration or constrain step sizes and parameters so the lemma's assumptions hold.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/beacons-fuzz-1779719148928/blob/main/test/standalone-findings/WhitepaperEconomicSearch.t.sol>

**— RESOLUTION**

Status: Acknowledged.

HIGH **H-09**

ACKNOWLEDGED

# Relative Dominance Stationary Bias

|          |                                                                                                                                                                                                                                                                       |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Math                                                                                                                                                                                                                                                                  |
| LOCATION | <a href="https://github.com/GuardianOrg/beacons-fuzz-1779719148928/blob/main/test/standalone-findings/WhitepaperEconomicSearch.t.sol">https://github.com/GuardianOrg/beacons-fuzz-1779719148928/blob/main/test/standalone-findings/WhitepaperEconomicSearch.t.sol</a> |
| REVIEW   | Remediation Review 1                                                                                                                                                                                                                                                  |

## DESCRIPTION

`RelativeDominance` compares fast and slow EMA levels after adding the same absolute pseudocount, `ALPHA`, to every class:

$$(mFast\_i + ALPHA) / (mSlow\_i + ALPHA)$$

When the fast and slow EMAs encode the same stationary class distribution, their absolute levels differ because the horizons differ. Adding the same absolute pseudocount changes each class's ratio by a different amount, especially for asymmetric class probabilities.

In the reproduction, both EMAs encode the same 80%/20% stationary distribution. The normalized member indices become approximately 98.3818% and 101.6448%, a spread of approximately 3.2631%, despite there being no relative distribution change.

## RECOMMENDATION

Normalize EMA levels by horizon before comparison:

$$normalizedFast\_i = (1 - decayFast) * mFast\_i$$
$$normalizedSlow\_i = (1 - decaySlow) * mSlow\_i$$

Apply pseudocounts in the same normalized probability space, or scale each pseudocount by its EMA horizon. Matching stationary distributions must produce equal relative indices for every supported class distribution.

## RESOLUTION

Status: Acknowledged.

HIGH **H-10**

ACKNOWLEDGED

# CGBM Misses Its Volatility Budget

|          |                                             |
|----------|---------------------------------------------|
| CATEGORY | Math                                        |
| LOCATION | src/core/standalone/basefns/CGBM.sol:97-133 |
| REVIEW   | Remediation Review 1                        |

## DESCRIPTION

CGBM exposes `sigmaBase` as its configured volatility budget. Appendix A of the whitepaper states that variance scaling should make indices consume this budget uniformly despite differences in the model's prediction-confidence distribution.

The intended conditional property is approximately:

$$E[(\Delta Z)^2 \mid \text{current state}] = \text{sigmaBase}^2$$

CGBM first converts a prediction into a confidence magnitude:

$$m = |2 * \text{prediction} - 1|^\alpha$$

It then calculates branch returns using:

$$\text{positive } \Delta Z = m * \sigma * (1 - \hat{p})$$

$$\text{negative } \Delta Z = -m * \sigma * \hat{p}$$

where:

$$\sigma = \text{sigmaBase} / \sqrt{\text{varianceFactor}}$$

The implementation does not consistently achieve the intended second moment. It incorporates the current observation into the directional masses, squared-magnitude accumulators, and effective sample size before calculating that observation's return. Positive and negative counterfactual branches can therefore use different `pHat`, `varianceFactor`, and `sigma` values.

Additionally, the whitepaper notes that its variance-scaling derivation must be redone for the symmetric return formula used by the implementation. The current normalization therefore does not algebraically guarantee:

$$\begin{aligned} &P(\text{positive}) * \Delta Z_{\text{Positive}}^2 \\ &+ P(\text{negative}) * \Delta Z_{\text{Negative}}^2 \\ &= \text{sigmaBase}^2 \end{aligned}$$

## H-10 CGBM Misses Its Volatility Budget

CONTINUED

Fuzzing reaches states where the conditional second moment differs materially from the configured target, including cases where the effective second moment collapses to zero despite a nonzero `sigmaBase`.

Impact: Market makers and protocol risk modules may use the configured oracle volatility to choose spreads, liquidity depth, funding parameters, and liquidation buffers. If realized index volatility is materially different from `sigmaBase`, these controls are calibrated against a property the implementation does not preserve.

Underestimated effective volatility can leave makers undercompensated for oracle jumps. Overestimated volatility can produce unnecessarily wide markets and weak price discovery. Branch-dependent volatility can also create directional asymmetry between long and short exposure.

### — RECOMMENDATION

e-derive variance normalization for the symmetric return formula actually used by CGBM.

Calculate the current observation's return from one immutable pre-observation snapshot:

1. Read the existing directional masses and squared-magnitude accumulators.
2. Derive a shared pre-observation probability and volatility scale.
3. Calculate both counterfactual branch returns from that shared state.
4. Verify their probability-weighted second moment against `sigmaBase^2`.
5. Apply the realized return.
6. Incorporate the observation into the accumulators used for the next report.

The implementation should enforce, within a documented fixed-point tolerance:

```
abs(  
  P(positive) * deltaZPositive^2  
  + P(negative) * deltaZNegative^2  
  - sigmaBase^2  
) <= tolerance
```

If exact normalization is not intended, rename or document `sigmaBase` as an internal scaling parameter and publish the actual supported bounds on conditional realized volatility.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/beacons-fuzz-1779719148928/blob/main/test/economic-invariant/StandaloneUnbounded.economic.invariant.t.sol>

---

**H-10** CGBM Misses Its Volatility Budget

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

HIGH **H-11**

ACKNOWLEDGED

# Ternary Gate Differs From Specification

|          |                                                             |
|----------|-------------------------------------------------------------|
| CATEGORY | Logical Error                                               |
| LOCATION | src/core/standalone/preprocessors/TernaryToBinary.sol:41-46 |
| REVIEW   | Remediation Review 1                                        |

## — DESCRIPTION

The whitepaper classifies an observation as relevant only when:

```
positive + negative > irrelevant + threshold
```

The implementation instead rejects only when:

```
irrelevant > positive + negative + threshold
```

These conditions are not equivalent. Increasing the configured threshold can make the implementation accept more observations, while the whitepaper's rule requires stronger relevance. Observations classified as irrelevant by the specification can consequently update the index.

## — RECOMMENDATION

Implement the stated relevance-margin condition directly and define equality behavior explicitly.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

# Winning Takers Can Exit Before Maker Losses Settle

CATEGORY Logical Error  
LOCATION src/libraries/PerpLogic.sol  
REVIEW Remediation Review 1

## DESCRIPTION

Taker full closes can pay realized AMM PnL immediately even when the corresponding maker-side losses have not yet been settled into maker margins or bad debt.

In `adjustTaker`, when a taker fully closes, the protocol computes:

```
equity = netMargin + netDelta.amount1();  
closeTaker(s, msg.sender, p.posId, equity.toUint256());
```

`closeTaker` then pays the taker through `transferMargin`. If `s.solvencyState.badDebt == 0`, `transferMargin` pays the full amount with no haircut.

The issue is that maker losses are lazily realized. A maker's funding/utilization/LP accounting is only crystallized when that maker position is touched. Therefore, a taker can withdraw profit from the shared USDC pool before the losing maker side has been settled and before any bad debt has been recorded.

This creates a first-come-first-served payout race among profitable takers.

Impact: Early winning takers can exit at par while later takers or remaining users absorb the shortfall once maker losses are eventually realized.

This does not require existing bad debt. The problem happens before bad debt is recorded:

1. Takers become profitable against the AMM.
2. Makers are economically losing, but their losses remain lazy/unsettled.
3. `badDebt` is still zero.
4. The first winning taker fully closes and receives full USDC payout.
5. Remaining obligations are now underfunded.
6. Later profitable closes may revert, receive worse treatment, or trigger later socialization.

The result is a solvency and fairness failure: payout ordering determines who receives full value.

**M-01** Winning Takers Can Exit Before Maker Losses Settle

CONTINUED

**— RECOMMENDATION**

Before paying realized taker PnL on a full close, the protocol should ensure the corresponding maker-side loss is crystallized.

**— RESOLUTION**

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

# Bad Debt Ignored In Health Checks

CATEGORY Logical Error  
LOCATION src/libraries/PerpLogic.sol:1010  
REVIEW Remediation Review 1

## DESCRIPTION

During bad-debt periods, outgoing margin transfers are haircut through `socializeLoss`, but position health checks continue to use nominal equity.

`transferMargin` applies bad-debt socialization when USDC leaves the market. However, `isHealthy` only receives nominal `equity` and `posVal`; it does not account for the fact that part of the position's withdrawable margin would be socialized while `badDebt > 0`.

This matters for maker positions because makers provide the capacity that takers trade against. A maker can appear healthy under nominal margin even though its realizable margin after socialization would be below the relevant liquidation threshold.

The same nominal health check is used after maker adjustments. If a maker is undercollateralized on a realizable basis but still passes nominal health, it can add liquidity and expand market capacity during an insolvency period. This can admit new taker exposure against already-impaired collateral and spread insolvency losses to users who enter after the bad-debt event.

## RECOMMENDATION

Health checks should use a realizable-collateral model while `badDebt > 0`.

Possible fixes:

- Apply the same bad-debt haircut inside health and liquidation checks.
- Forbid maker liquidity expansion while the market has bad debt unless the maker remains healthy after applying the haircut.
- Ensure the implementation explicitly handles states where `badDebt > 0`, nominal maker health passes, but post-socialization maker health would fail.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

# Insolvent Liquidations Pay Fees

CATEGORY Logical Error  
LOCATION src/libraries/PerpLogic.sol  
REVIEW Remediation Review 1

## — DESCRIPTION

Full liquidation paths subtract the liquidation fee from the position before bad-debt accounting, but then pay the liquidation fee as a separate outgoing margin transfer.

In `liquidateTaker`, the fee is included in `netMargin` before final equity is calculated and passed into `accountBadDebt`. After the bad-debt accounting step, the same flow pays the liquidation fee recipient through `transferMargin`.

The same pattern exists in `liquidateMaker`: the fee is removed from `netMargin`, insolvency is accounted through `accountBadDebt`, and then the fee is paid through a separate `transferMargin`.

This means an insolvent liquidation can record bad debt while still attempting to pay a liquidation fee from the shared margin pool. If the position owner controls the liquidation call or recipient, they can route the fee to themselves while externalizing the remaining loss to insurance, bad debt, or other users.

Liquidation fees can therefore be extracted even when the liquidated position does not have enough equity to cover them. The payout may be reduced if bad-debt socialization is already active, but the fee path still attempts to pay from shared margin after the shortfall has been recorded.

## — RECOMMENDATION

Cap liquidation fees by recoverable equity.

If a full liquidation has zero remaining equity after bad-debt accounting, the liquidation fee should be zero or paid only from a specifically funded liquidation reserve. The liquidation fee and owner payout should be reconciled in a single accounting step so an insolvent close cannot both create bad debt and pay a full nominal liquidation reward.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

# Module Registry Approvals Are Append-Only

|          |                                                                                                                                                                                                                                                                                                                     |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Configuration                                                                                                                                                                                                                                                                                                       |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/36f08736d2ccf0cf9546e53fe87f3006b93bb35b/src/ModuleRegistry.sol#L18-L20">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/36f08736d2ccf0cf9546e53fe87f3006b93bb35b/src/ModuleRegistry.sol#L18-L20</a> |
| REVIEW   | Remediation Review 1                                                                                                                                                                                                                                                                                                |

## DESCRIPTION

`ModuleRegistry` is intended to track modules approved by Perp City, but registered modules are append-only. `registerModule()` can only set `modules[moduleType][module] = true` and emit `ModuleRegistered`; the interface does not expose any way to unregister, revoke, deprecate, or mark a module as unsafe.

If an approved module is later found to be compromised, incorrectly configured, obsolete, or unsafe for new markets, the registry cannot remove that approval. Integrators, users, or deployment tooling that rely on `ModuleRegistry.modules()` may continue treating the module as approved even after the protocol no longer wants to recommend it.

## RECOMMENDATION

Consider to add an `unregisterModule()` function, or similar functionality.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

RESOLVED

# Taker Fees Wrap Total Margin

|          |                             |
|----------|-----------------------------|
| CATEGORY | Logical Error               |
| LOCATION | src/libraries/PerpLogic.sol |
| REVIEW   | Remediation Review 1        |

## DESCRIPTION

When a taker swap occurs, protocol, creator, and insurance fees are first processed through `accrueSwapFees()`.

If bad debt already exists, part or all of those fees may be used to repay bad debt instead of becoming live creator, protocol, or insurance claims. However, the taker adjustment and liquidation paths later call `removeSwapFeesFromMargin()`, which subtracts the full nominal fee amount from `solvencyState.totalMargin`.

That subtraction is unchecked. If the market is already impaired and `totalMargin` has fallen below the nominal fee amount, `totalMargin` can wrap to a very large `uint128` value.

Once `totalMargin` wraps, future bad-debt socialization uses an inflated denominator. This can cause withdrawals and payouts to receive little or no insolvency haircut even though bad debt remains, shifting losses to later participants or leaving the market accounting permanently distorted.

This is classified as Medium because it depends on the market already being in a bad-debt state with very low tracked `totalMargin`, and on a surviving taker action still being executable. Severity can increase if these preconditions are reachable in normal market operation.

## RECOMMENDATION

Debit `solvencyState.totalMargin` only by the portion of fees that actually becomes a live claim after bad-debt repayment.

Avoid unchecked subtraction for solvency-critical accounting, especially when bad debt exists, `totalMargin` is low, and taker swaps or partial liquidations charge fees.

## RESOLUTION

Status: Resolved.

MEDIUM **M-06**

RESOLVED

# Lazy Funding Wraps Total Margin

CATEGORY Logical Error  
LOCATION src/libraries/PerpLogic.sol:432  
REVIEW Remediation Review 1

## DESCRIPTION

Taker funding credits are settled lazily when a position is adjusted. If accrued funding is negative for the taker, the credit increases the position's apparent spendable margin inside `adjustTaker()`.

During bad-debt periods, outgoing transfers are socialized at payout time, but `transferMargin()` still decrements `solvencyState.totalMargin` by the full requested withdrawal amount. This can allow lazy funding credits to reduce tracked `totalMargin` even though the corresponding counterparty liability has not necessarily been settled into the same accounting base.

If tracked `totalMargin` becomes very small while bad debt exists, a later fee-paying taker action can reach `removeSwapFeesFromMargin()` with insufficient tracked margin. Because that fee debit is unchecked, `totalMargin` can wrap to a very large value. Future bad-debt socialization then uses the inflated denominator and may apply little or no haircut despite outstanding bad debt.

This is classified as Medium because it depends on an already impaired market, lazy funding credits, and a later surviving taker action. It is still security-relevant because all of these are normal protocol states/actions rather than privileged behavior.

## RECOMMENDATION

Do not let lazy funding or utilization credits become withdrawable unless the matching liability is reflected consistently in `solvencyState.totalMargin`.

When bad debt exists, decrement `totalMargin` by the amount actually paid or by a clearly defined claim-extinguishment amount, not by an inconsistent pre-haircut value. Replace unchecked solvency-critical fee debits with checked accounting that cannot wrap when tracked margin is insufficient.

---

**M-06** Lazy Funding Wraps Total Margin

CONTINUED

---

— RESOLUTION

Status: Resolved.

MEDIUM **M-07**

RESOLVED

# Dust Signals Escape Variance Tracking

CATEGORY Math  
LOCATION src/core/standalone/basefns/CGBM.sol:91-116  
REVIEW Remediation Review 1

## DESCRIPTION

A near-neutral prediction can produce a nonzero magnitude `m` while `m.mulWad(m)` rounds to zero. CGBM then adds the signal to `mUp` or `mDown` and increments `effectiveN`, but records no corresponding squared contribution in `fSqUp` or `fSqDown`. Repeated dust reports can therefore lower the measured variance and amplify a later ordinary report. Existing reproductions show material divergence in the resulting beacon index, which can propagate into downstream pricing, funding, and liquidation calculations.

## RECOMMENDATION

Use enough internal precision to preserve the squared contribution of every accepted nonzero magnitude. Alternatively, reject magnitudes below the minimum representable square or consistently treat them as zero across directional mass, squared mass, and effective sample size.

## RESOLUTION

Status: Resolved.

MEDIUM **M-08**

ACKNOWLEDGED

# Softmax Creates Allocation Drift

CATEGORY Math  
LOCATION src/core/group/transforms/Softmax.sol:31  
REVIEW Remediation Review 1

## DESCRIPTION

`DiscreteAllocation` attempts to remove baseline class-frequency bias by giving rare classes larger latent-state updates than common classes:

```
winner update for class i = scaledSigma / baselineProbability_i
```

This makes every class receive the same expected update in `z`-space:

```
baselineProbability_i × winnerUpdate_i = scaledSigma
```

The whitepaper relies on Softmax's translation invariance to argue that this common expected movement cancels and therefore does not change the expected allocation.

However, equal expected latent-state movements do not imply equal expected prices after applying Softmax. Softmax is nonlinear:

```
E[softmax(z + deltaZ)] != softmax(z + E[deltaZ])
```

Consider two relevant classes starting from equal latent states, so both member indices initially represent 50% of the allocation. Assume their baseline probabilities and winner updates are:

```
common class probability = 80%
```

```
rare class probability = 20%
```

```
common winner update = 0.625
```

```
rare winner update = 2.5
```

The expected latent update is identical for both classes:

```
common: 80% × 0.625 = 0.5
```

```
rare: 20% × 2.5 = 0.5
```

But the common class's Softmax allocation after each possible outcome is:

```
common class wins: approximately 65.1%
```

```
rare class wins: approximately 7.6%
```

## M-08 Softmax Creates Allocation Drift

CONTINUED

Its expected next allocation is therefore:

$$80\% \times 65.1\% + 20\% \times 7.6\% = \text{approximately } 53.6\%$$

The common class begins at 50% but has an expected next value of approximately 53.6%, even though the prediction distribution exactly matches the configured baseline probabilities.

Translation invariance only proves that adding the same realized constant to all latent states leaves Softmax unchanged:

$$\text{softmax}(z + c \times 1) = \text{softmax}(z)$$

It does not prove that different random updates with the same expected value cancel after Softmax. The unequal jump distributions introduced by inverse probability weighting create transformed-price drift.

As a result, a `DiscreteAllocation + Softmax` beacon family can systematically favor common classes and move away from its intended baseline under an honest, perfectly calibrated prediction stream. Downstream perpetual markets may then use biased member indices for pricing, funding, and liquidation decisions.

### — RECOMMENDATION

Calibrate the update magnitudes against expected post-Softmax allocations instead of only equalizing expected latent-state changes.

For each class and current state, the update rule should satisfy a price-space condition such as:

$$\begin{aligned} &\text{sum over outcomes} ( \\ &\text{outcomeProbability} \times \text{softmax}(z \text{ after outcome}) \\ &) = \text{softmax}(z \text{ before update}) \end{aligned}$$

If an exact state-dependent solution is too expensive on-chain, derive and validate an approximation that compensates for Softmax curvature, restrict the supported probability and volatility ranges to regions where the drift is negligible, or use an update construction that is a martingale directly in allocation space.

### — RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

# CGBM Price Drift After Exponentiation

CATEGORY Math  
LOCATION src/core/standalone/basefns/CGBM.sol:107  
REVIEW Remediation Review 1

## DESCRIPTION

CGBM attempts to balance positive and negative updates in its internal `z`-space. A `StandaloneBeacon` can then pass this accumulated value through the `Unbounded` transform:

$$\text{index} = \text{initialIndex} \times \exp(z)$$

Balancing movements in `z`-space does not balance the resulting index prices. Exponentiation makes an upward movement increase the price by more than an equal downward movement decreases it.

For example, starting from an index of `100`, equal log-space movements of `+0.05` and `-0.05` produce:

$$\text{positive outcome: } 100 \times \exp(0.05) = 105.13$$

$$\text{negative outcome: } 100 \times \exp(-0.05) = 95.12$$

$$\text{average outcome: } = 100.125$$

The average is above the original index even though the two internal movements are equal and opposite.

More generally, the implementation targets:

$$E[\text{deltaZ}] = 0$$

but an unbounded price is neutral only when:

$$E[\exp(\text{deltaZ})] = 1$$

These conditions are not equivalent. For any nonconstant zero-mean `deltaZ`, exponentiation causes:

$$E[\exp(\text{deltaZ})] > 1$$

**M-09** CGBM Price Drift After Exponentiation

CONTINUED

Consequently, a CGBM beacon configured with the generic Unbounded transform can exhibit systematic upward index drift even when its positive and negative signals are calibrated to be neutral in latent space. This can bias funding, mark prices, and liquidation thresholds in favor of long exposure over time.

**— RECOMMENDATION**

Calibrate CGBM returns against the final transformed index rather than only against the internal `z` value.

Before applying an update, derive a correction that makes the probability-weighted next prices equal the current price:

$$\begin{aligned} &P(\text{positive}) \times \exp(\text{returnPositive}) \\ &+ P(\text{negative}) \times \exp(\text{returnNegative}) \\ &= 1 \end{aligned}$$

**— RESOLUTION**

Status: Acknowledged.

MEDIUM **M-10**

ACKNOWLEDGED

# Neutral Ticks Amplify CGBM Moves

CATEGORY Math

LOCATION `src/core/standalone/basefns/CGBM.sol:91-127`

REVIEW Remediation Review 1

## DESCRIPTION

An exact-neutral CGBM prediction ( `prediction == 0.5e18` ) produces zero movement:

```
magnitude = |2 × prediction - 1|^alpha = 0  
zDelta = 0
```

However, `CGBM.zDelta()` still mutates all of its adaptive state before returning that zero delta. Each neutral update:

```
mUp = DECAY.mulWad(mUp);  
mDown = DECAY.mulWad(mDown);  
fSqUp = DECAY.mulWad(fSqUp);  
fSqDown = DECAY.mulWad(fSqDown);  
effectiveN = DECAY.mulWad(effectiveN) + WAD;
```

Because the neutral sample contributes no magnitude or squared magnitude, `mUp`, `mDown`, `fSqUp`, and `fSqDown` decay while `effectiveN` continues to receive a full sample increment.

This reduces the estimated variance:

```
varianceFactor =  
((1 - pHat)^2 × fSqUp + pHat^2 × fSqDown) / effectiveN
```

CGBM then derives its volatility multiplier inversely from that variance:

```
sigma = scaledSigmaBase / sqrt(varianceFactor)
```

As a result, a sequence of updates that individually produce no index movement can silently increase the size of a later directional update.

With representative parameters:

```
sigmaBase = 0.1  
scalingFactor = 1  
alpha = 1
```

**M-10** Neutral Ticks Amplify CGBM Moves

CONTINUED

```
decay = 0.99
initialSigmaRatio = 1
minVariance = 1e-12
```

the same full-positive prediction produces approximately:

```
without neutral history: zDelta = 0.0576
after 122 neutral predictions: zDelta = 0.3185
```

The later update is approximately **5.5×** larger even though every preceding neutral update returned zero.

For an index starting at **50**, this corresponds approximately to:

```
normal update after neutral priming
bounded [0, 100] 51.44 57.90
unbounded 52.97 68.75
```

This behavior can occur during an honest period of model uncertainty if the feed emits exact-neutral predictions. A later ordinary directional report can then unexpectedly create a large oracle jump, affecting market prices, funding, and liquidation thresholds.

This issue is distinct from nonzero dust-prediction priming: it does not rely on fixed-point squaring rounding a nonzero magnitude to zero. Every primer is an exact, intentionally supported zero-magnitude prediction.

**— RECOMMENDATION**

Treat exact-zero-magnitude predictions as true no-signal updates and do not mutate CGBM's directional or variance-estimation state:

```
if (m == 0) return 0;
```

This check should occur before decaying or incrementing any accumulators.

**— RESOLUTION**

Status: Acknowledged.

MEDIUM **M-11**

RESOLVED

# Confidence Ordering Can Invert

CATEGORY Math  
LOCATION `src/core/standalone/basefns/CGBM.sol:91-133`  
REVIEW Remediation Review 1

## DESCRIPTION

CGBM incorporates the current prediction into its directional masses, squared-magnitude accumulators, and effective sample size before calculating that prediction's return. A stronger prediction can therefore increase the estimated variance enough to reduce `sigma` by more than its larger magnitude increases the return. Reachable histories exist where a stronger same-direction prediction produces a smaller immediate price movement than a weaker prediction. This breaks the expected monotonic relationship between confidence and price impact and can cause materially incorrect oracle marks under otherwise valid signed data.

## RECOMMENDATION

Calculate the current prediction's return from an immutable snapshot of the estimator state, then update the stored statistics after determining the return. Independently test and enforce that, for every supported state and parameter set, same-direction return magnitude is monotonic in prediction confidence.

## RESOLUTION

Status: Resolved.

MEDIUM **M-12**

ACKNOWLEDGED

# Zero-liquidity gap poisons AMM EMA to force liquidation

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 1

## — DESCRIPTION

Perp reads Uniswap V4 slot0 as the AMM oracle without checking active liquidity. In a sparse market, an attacker-controlled maker can withdraw in-range liquidity, leave one-sided depth past a zero-liquidity gap, and use dust swaps plus touch() to ratchet slot0/emaAmm away from an unchanged beacon index under the default modules.

Impact: The PoC's matched control stays NotLiquidatable after the same accrual window, while the poisoned market's victim becomes liquidatable from excess funding/mark effects and is force-closed by a permissionless liquidator. This is concrete victim collateral loss from economically unbacked slot0 movement; Medium due to requiring sparse/attacker-shaped liquidity and time.

## — RECOMMENDATION

Do not use slot0 observations for EMA/funding/liquidation when active liquidity is zero or below a minimum. Require price moves to consume meaningful liquidity/fees before blending them into AMM oracle state, or fall back to beacon/TWAP values for zero-liquidity states.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-13**

RESOLVED

# Maker conversion checks quote debt before valuing residual perps

CATEGORY DoS  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

When a maker removes all liquidity and has residual perp exposure, `adjustMaker()` first folds the quote leg into margin with `equity = pos.margin + pos.delta.amount1()` and reverts `NegativeMargin` if that cash/quote value is negative. The code only then zeroes `amount1` and values the residual `amount0` through `valPnl()/isHealthy()`. A residual long can therefore have positive net equity and satisfy taker initial margin, while conversion is rejected before the taker health check sees the perp value.

Impact: A maker can be unable to complete a full exit/convert-to-taker unless they deposit enough USDC to cover the raw negative quote leg, even when the residual position is economically solvent at taker initial margin. This is a reversible exit DoS/excess-collateral requirement that leaves the LP stuck as a maker and exposed to further market/funding moves; no direct theft or bad debt is shown.

## RECOMMENDATION

Align the conversion gate with taker health accounting. Run solvency/initial-margin checks on the full residual delta including `amount0` value before rejecting, and either preserve the quote leg in the converted taker delta or otherwise settle/realize `amount0` and `amount1` consistently without requiring unnecessary quote prepayment.

## RESOLUTION

Status: Resolved.

LOW **L-01**

RESOLVED

# Bad Debt Can Wrap Around uint128

CATEGORY Math  
LOCATION src/libraries/PerpLogic.sol  
REVIEW Remediation Review 1

## DESCRIPTION

`SolvencyState.badDebt` is stored as a `uint128`:

```
struct SolvencyState {  
    uint128 badDebt;  
    uint128 totalMargin;  
}
```

When a liquidation or close realizes negative equity, `accountBadDebt` adds the uncovered loss to this accumulator:

```
s.solvencyState.badDebt += (badDebt - insurance).toUint128();
```

This addition is performed inside an `unchecked` block. If cumulative bad debt exceeds `type(uint128).max`, the value wraps around to a smaller number.

The wrapped value is then used by `socializeLossInternal` to calculate withdrawal haircuts:

```
uint256 debt = solvency.badDebt;  
uint256 fee = debt >= margin ? originalAmt : originalAmt.fullMulDivUp(debt, margin);
```

As a result, an insolvent market with very large real bad debt could appear to have only a small amount of recorded bad debt after wraparound.

Impact: If reachable, this can cause withdrawals and fee payouts to be under-haircutted after insolvency. Users could withdraw close to full value even though the market still has large outstanding losses.

This is marked Low here because practical reachability depends on whether production market parameters and modules can ever create cumulative bad debt near `uint128.max`. However, the arithmetic invariant is not enforced by the code itself.

---

**L-01** Bad Debt Can Wrap Around uint128

---

CONTINUED

---

**RECOMMENDATION**

Use checked or wider bad-debt accounting.

**RESOLUTION**

Status: Resolved.

LOW **L-02**

ACKNOWLEDGED

# Missing Settled Position Preview View

|          |                                                                                                                                                                                                                                                                                           |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Informational                                                                                                                                                                                                                                                                             |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/36f08736d2ccf0cf9546e53fe87f3006b93bb35b/src/Perp.sol#L124">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/36f08736d2ccf0cf9546e53fe87f3006b93bb35b/src/Perp.sol#L124</a> |
| REVIEW   | Remediation Review 1                                                                                                                                                                                                                                                                      |

## DESCRIPTION

`positions()`, `makerDetails()`, and related public getters expose raw stored state only. For maker positions, this can under-report or otherwise misrepresent current economic state because LP fees, funding, utilization fees, unrealized PnL, and health are settled or evaluated lazily inside write paths such as `adjustMaker()`, `liquidateMaker()`, and `backstopMaker()`.

The internal code already computes these values when needed, but there is no canonical public view that returns a settled preview of a position's current margin, pending fees, equity, value, and health status. As a result, UIs, indexers, liquidators, and monitoring systems must reconstruct this logic off-chain from multiple raw getters and module calls, increasing integration complexity and the risk of stale or inconsistent displays.

## RECOMMENDATION

Consider adding an official read-only settled-preview function for maker and taker positions.

## RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

# Split Reports Alter CGBM Movement

CATEGORY Math  
LOCATION src/core/standalone/basefns/CGBM.sol:97-133  
REVIEW Remediation Review 1

## DESCRIPTION

Splitting a fixed amount of directional information across multiple lower-confidence reports changes the final CGBM state and cumulative movement. Every report independently applies decay, increments `effectiveN` by a full sample, and updates variance statistics, so several reports are not economically equivalent to one aggregated report. A relayer can exploit this only when it has discretion over the granularity or cadence of valid signed observations. Because the trust and reporting pipeline must provide that discretion, and because no direct profit or liquidation path is established, the issue is Low severity.

## RECOMMENDATION

Define and enforce canonical reporting intervals and aggregation rules in the signing pipeline. If split and aggregated reports are intended to be equivalent, weight `effectiveN` and decay by signal information or elapsed time rather than by transaction count.

## RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

# Unsafe Minimum Variance Bounds

|          |                                            |
|----------|--------------------------------------------|
| CATEGORY | Math                                       |
| LOCATION | src/core/standalone/basefns/CGBM.sol:51-79 |
| REVIEW   | Remediation Review 1                       |

## DESCRIPTION

The CGBM constructor rejects only `minVariance == 0`. Extremely small accepted values permit a very small variance denominator and correspondingly large `sigma`, potentially causing extreme returns or arithmetic reverts after valid histories. Extremely large values can instead round `sigma` to zero and freeze all movement. Exploitation requires a beacon to be deployed with unsafe creator-selected parameters, so this is primarily a deployment-validation risk rather than an attacker-controlled runtime vulnerability.

## RECOMMENDATION

Derive minimum and maximum safe `minVariance` values from `SCALED_SIGMA_BASE`, supported prediction magnitudes, transform bounds, and downstream price domains. Enforce these bounds in CGBM and its factories, and cap the maximum permitted per-report latent delta.

## RESOLUTION

Status: Resolved.

LOW **L-05**

RESOLVED

# Valid History Can Brick Unbounded

CATEGORY Math  
LOCATION `src/core/standalone/StandaloneBeacon.sol:83-93`  
REVIEW Remediation Review 1

## DESCRIPTION

`StandaloneBeacon` accumulates every valid base-function delta into `_zSpaceIndex`, while `Unbounded` accepts only the finite exponentiation interval supported by `expWad`. A sequence of individually valid signed reports can move the accumulated z value outside that interval, causing `toIndex` to revert. The revert also rolls back verifier nonce advancement, so the same report continues to fail and later sequential nonces cannot be submitted. A live feed can consequently freeze at a stale price until the beacon is replaced.

## RECOMMENDATION

Enforce transform-aware headroom before committing each update. Cap the resulting cumulative z value, constrain base-function deltas to the remaining transform range, or use a transform whose valid domain covers every state reachable under supported parameters. Recovery should not depend solely on replacing the beacon.

## RESOLUTION

Status: Resolved.

LOW **L-06**

RESOLVED

# Bounded Accumulates Hidden Windup

CATEGORY Math  
LOCATION `src/core/standalone/StandaloneBeacon.sol:83-93`  
REVIEW Remediation Review 1

## DESCRIPTION

`Bounded.toIndex` clamps only the temporary exponent input used to calculate the published index, while `StandaloneBeacon` continues storing the unconstrained `_zSpaceIndex`. Once the public price saturates near a bound, further same-direction reports accumulate invisible latent movement. After the signal reverses, opposite reports must first unwind that hidden state before the published price responds. This can leave a Perp market using a stale boundary price during a genuine reversal, affecting funding, trading, and liquidation decisions.

## RECOMMENDATION

Clamp the stored z-space state to the same effective bounds used by the transform, or discard the portion of a delta that only increases saturation. The first meaningful opposite report should be able to move the published index away from the boundary.

## RESOLUTION

Status: Resolved.

LOW **L-07**

RESOLVED

# Stale taker margin snapshots can bypass dynamic risk tiers

CATEGORY Configuration  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

Taker init/liquidation/backstop ratios are sampled into `Position` at open/conversion, but `adjustTaker()` does not reload them when exposure changes after a dynamic margin module's live tier changes. `liquidateTaker()` then checks liquidation eligibility against the stored `pos.liqMarginRatio` rather than the active module's current liquidation ratio. The updated PoC uses a skew-aware `IMarginRatios` module, opens a small low-tier taker, crowds long OI so the module returns higher 20%/18% tiers, then expands the original position while retaining 5%/2.5% snapshots; the position is below the current liquidation tier but full liquidation reverts `NotLiquidatable` under the stale snapshot.

Impact: If margin-ratio modules are intended to enforce live/dynamic risk tiers on exposure changes and liquidation, traders can retain obsolete low-tier thresholds for newly enlarged positions. This leaves open interest that should be liquidatable under the active risk module protected from liquidation, widening the loss/bad-debt window. The demonstrated impact is delayed/prevented liquidation rather than realized loss, so Medium is appropriate.

## RECOMMENDATION

Clarify margin-ratio semantics. If active module outputs should govern adjusted taker exposure and liquidation, reload taker init/liq/backstop ratios after OI-changing adjustments and use the current liquidation/backstop thresholds in liquidation/backstop checks. If ratios are intentionally grandfathered at open/conversion, document that invariant and avoid or explicitly gate dynamic/skew-based margin modules that expect live enforcement.

## RESOLUTION

Status: Resolved.

LOW **L-08**

ACKNOWLEDGED

# Same-liquidation swap fees accrue before insolvency bad debt is recorded

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 1

## — DESCRIPTION

During a full taker liquidation, `liquidateTaker()` calls `PerpAmmLogic.swap()/accrueSwapFees()` before the full-close branch computes final equity and records fresh bad debt via `accountBadDebt()`. Because `accrueSwapFees()` only haircuts non-insurance fees against pre-existing `badDebt`, creator/protocol fees from the same insolvent liquidation are credited in full while `badDebt` is still zero, then the unpaid deficit is recorded afterward. This is distinct from the prior collection-priority issue because the root cause is intra-liquidation accrual ordering; the fresh debt does not yet exist when the fees are booked.

Impact: For an insolvent full liquidation with no legacy bad debt, creator and protocol swap fees generated by that liquidation can be recorded in `feeFund` and later withdrawn through `collectCreatorFees()/collectProtocolFees()` while the corresponding deficit remains as `badDebt` to be absorbed by insurance or other margin. The loss is bounded by configured swap-fee rates on liquidation volume, so Medium is appropriate rather than High.

## — RECOMMENDATION

Defer creator/protocol swap-fee accrual for liquidation swaps until final solvency is known, or retroactively haircut same-liquidation fees against any bad debt created by that liquidation. Collection endpoint fixes alone do not address the accrual-ordering issue.

## — RESOLUTION

Status: Acknowledged.

LOW **L-09**

RESOLVED

# Rounded-up maker capacity proration causes dust-scale cap drift

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 1

## — DESCRIPTION

On partial maker liquidity removals, `proratedCapacity()` rounds the removed long and short capacity up, while re-adding the same liquidity restores capacity through `calcCapacity()`, which floors/truncates the proportional capacity. Repeated remove/add round trips can therefore reduce the maker's stored capacity and global `s.cap` despite unchanged liquidity.

Impact: The drift is bounded to at most one 6-decimal capacity atom per side per round trip, so this is dust-scale rather than a practical market-wide DoS. It can nonetheless make utilization checks reject trades a few atoms earlier than the pool liquidity would otherwise support.

## — RECOMMENDATION

Make remove/add capacity accounting symmetric: track and carry rounding remainders, use matching rounding, or recompute removed capacity from the removed liquidity so round trips conserve capacity.

## — RESOLUTION

Status: Resolved.

LOW **L-10**

RESOLVED

## V4 rounding dust can block final maker exit without opposite capacity

CATEGORY DoS  
LOCATION —  
REVIEW Remediation Review 1

### — DESCRIPTION

Uniswap V4 amount0 deltas round up on liquidity add and down on removal, so an add/remove cycle can leave a 1-atom residual perp balance. On final maker removal, PerpLogic removes the maker capacity and then converts any nonzero residual amount0 into taker open interest; with no opposite capacity, updateOpenInterest reverts.

Impact: A maker's voluntary full exit, and final liquidation in the same residual path, can be temporarily blocked with margin/liquidity left in the NFT. The effect is dust-scale and permissionlessly remediable by adding small opposite-side capacity, so impact is liveness friction rather than material loss.

### — RECOMMENDATION

Treat de minimis rounding residuals as zero or absorb them into margin/accounting on final maker close, or defer/adjust capacity removal so dust conversion cannot fail solely because the removed maker was the last capacity.

### — RESOLUTION

Status: Resolved.

LOW **L-11**

RESOLVED

# Bounded round-trip can brick perps on first zero-delta update

CATEGORY DoS  
LOCATION src/core/standalone/transforms/Bounded.sol  
REVIEW Remediation Review 1

## DESCRIPTION

`StandaloneBeacon` stores the constructor `_initialIndex` verbatim while separately initializing `_zSpaceIndex = transform.toZSpace(_initialIndex)`. `Bounded.toZSpace()` accepts ratios outside the clamped forward image used by `Bounded.toIndex()`: the inverse floors `(index - MIN_INDEX).divWad(INDEX_RANGE)`, but the forward path clamps `z * STEEPNESS` to `[EXP_UNDERFLOW, EXP_OVERFLOW]` before flooring the bounded output. A valid no-op update with `BASE_FN.zDelta(prediction) == 0` therefore rewrites `_index` to `TRANSFORM.toIndex(_zSpaceIndex)`, which can differ by orders of magnitude from the deployment index for a constructor-valid wide range.

Impact: A market can be created while the beacon reports a `PerpFactory`-acceptable deployment index, then any relayer with a valid neutral update can snap the beacon above `uint128.max`. All normal market entry points that call `PerpLogic accrue()` then revert on `env.modules.beacon.index().toUint128()`, blocking touch, trading, maker adjustment, and liquidation for users with deposited margin. Timelocked recovery through `setBeacon()` also calls `accrue()` before replacing the beacon, so the affected market has no normal in-contract recovery path once bricked.

## RECOMMENDATION

Make the bounded inverse and forward domains consistent. For example, reject initial/current indices whose `toZSpace()` result lies outside the forward clamp image, initialize `_index` from the canonical round-trip value, or store z-space as the only canonical state and ensure zero-delta updates are idempotent. Also consider enforcing `Perp`-safe beacon output bounds for markets that will be used by `PerpFactory`.

## RESOLUTION

Status: Resolved.

LOW **L-12**

ACKNOWLEDGED

# Alpha Zero Removes Confidence

CATEGORY Configuration  
LOCATION src/core/standalone/basefns/CGBM.sol:54-72  
REVIEW Remediation Review 1

## DESCRIPTION

CGBM calculates signal magnitude as  $\text{normP}^\alpha$ . When  $\alpha$  is zero, every nonzero normalized prediction has magnitude one. A prediction one wei away from neutral therefore produces the same initial update as a full-confidence prediction in the same direction.

This removes confidence information from all non-neutral reports. If  $\alpha = 0$  is an accepted configuration, minimally directional reports can move the beacon as strongly as maximally confident reports.

## RECOMMENDATION

Reject  $\alpha = 0$  unless binary-sign behavior is explicitly intended. If it is supported, document it as a separate operating mode rather than presenting the configuration as confidence-sensitive CGBM.

## RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

# Unbounded Can Publish Zero

|          |                                                    |
|----------|----------------------------------------------------|
| CATEGORY | Configuration                                      |
| LOCATION | src/core/standalone/transforms/Unbounded.sol:25-43 |
| REVIEW   | Remediation Review 1                               |

## DESCRIPTION

`Unbounded.toIndex` multiplies the initial index by `exp(z)` using WAD arithmetic. For a sufficiently small initial index and negative in-range `z`, the multiplication rounds to zero even though the mathematical result is strictly positive.

The transform accepts the configuration and publishes zero, while `toZSpace(0)` explicitly reverts. This breaks transform invertibility and can publish a price that downstream systems treat as invalid or catastrophic.

## RECOMMENDATION

Require a minimum initial index that remains nonzero throughout the supported negative-`z` range, or revert whenever the computed output is zero. Validate the complete reachable output range during deployment.

## RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

# Unbounded Exceeds Consumer Domains

|          |                                                    |
|----------|----------------------------------------------------|
| CATEGORY | Math                                               |
| LOCATION | src/core/standalone/transforms/Unbounded.sol:22-37 |
| REVIEW   | Remediation Review 1                               |

## DESCRIPTION

`Unbounded` returns a `uint256` without enforcing a protocol-wide maximum. It can produce values above narrower domains used by integrations, storage layouts, casts, or arithmetic assumptions. The fuzz detector demonstrates outputs above `uint128`.

The standalone interface itself uses `uint256`, so impact depends on an actual consumer narrowing or otherwise bounding the value.

## RECOMMENDATION

Define and enforce the supported index domain at the transform or factory boundary. Consumers using narrower types must validate before casting, and deployment tooling should reject configurations whose reachable output range exceeds a consumer's domain.

## RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

# Bounded Round Trips Lose Precision

CATEGORY Math  
LOCATION `src/core/standalone/transforms/Bounded.sol:33-61`  
REVIEW Remediation Review 1

## DESCRIPTION

For low-scale ranges or steepness values, converting an index to z-space and back can lose a large fraction of the original value. Integer division in the normalized sigmoid and logarithmic inverse removes information that the forward transform cannot recover.

Beacon initialization or transform replacement can consequently move the published value materially despite there being no economic update.

## RECOMMENDATION

Enforce minimum range and steepness constraints that guarantee acceptable round-trip error. Deployment tooling should calculate maximum absolute and relative round-trip error before accepting a configuration.

## RESOLUTION

Status: Acknowledged.

LOW **L-16**

ACKNOWLEDGED

# TWAP Accumulator Can Overflow

CATEGORY Math  
LOCATION src/libraries/TwAvg.sol:71-80  
REVIEW Remediation Review 1

## DESCRIPTION

TWAP observations store cumulative value in `uint216`, while beacon indices use `uint256`. A value and elapsed-time pair accepted by the beacon can exceed the cumulative storage domain:

```
cumulativeValue + currentValue * elapsedTime > type(uint216).max
```

The next observation write reverts, preventing an otherwise valid oracle update.

## RECOMMENDATION

Enforce a maximum index based on the maximum observation interval, use a wider cumulative type, or define overflow-safe modular cumulative arithmetic with formally bounded deltas.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

# Oracle Reports Lack Expiry

|          |                                       |
|----------|---------------------------------------|
| CATEGORY | Warning                               |
| LOCATION | src/verifiers/ECDSA/ECDSAVerifier.sol |
| REVIEW   | Remediation Review 1                  |

## DESCRIPTION

`ECDSAVerifier` authenticates signed measurements by signer and strict sequential nonce, but the signed payload does not include a data timestamp, expiry, or maximum delay. A valid next nonce can therefore be submitted at any later time as long as it has not already been consumed.

This appears to be expected behavior for the current verifier design, where sequencing is nonce-based rather than freshness-based. The operational tradeoff is that freshness guarantees must come from signer/relayer policy or downstream consumers, not from the verifier itself.

## RECOMMENDATION

If freshness is required, include timestamp or expiry metadata in the signed payload and enforce max-age checks. Otherwise, explicitly document that signed reports do not expire on-chain and that freshness is handled operationally.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-02

ACKNOWLEDGED

# Composite Reads Mixed Rounds

|          |                                        |
|----------|----------------------------------------|
| CATEGORY | Warning                                |
| LOCATION | src/core/composite/CompositeBeacon.sol |
| REVIEW   | Remediation Review 1                   |

## DESCRIPTION

`CompositeBeacon.index()` reads each reference beacon live and composes the values immediately. Since reference beacons update independently, the composite can temporarily reflect a mix of old and new constituent rounds.

This is not necessarily a contract bug if the protocol intentionally chose live composition over synchronized basket snapshots. The tradeoff is that `index()` can expose transient mixed-round values, and consumers must decide whether those values are acceptable for their use case.

## RECOMMENDATION

Document that `CompositeBeacon.index()` is a live, unsynchronized read. Consumers that require synchronized basket pricing should use keeper-sampled snapshots, TWAPs, or round/timestamp-aware composition instead of raw live reads.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-03

RESOLVED

# Beacon Switch Keeps Old EMA

|          |                      |
|----------|----------------------|
| CATEGORY | Configuration        |
| LOCATION | src/Perp.sol:236     |
| REVIEW   | Remediation Review 1 |

## DESCRIPTION

`setBeacon` switches the market to a new beacon, updates the spot index in the local snapshot, and then refreshes rates using the existing EMA values.

`refreshRatesAndEmas` stores the EMA values it receives, so a beacon migration can immediately combine the new beacon spot with the old beacon's index EMA. Any pricing or funding module that relies on both spot index and index EMA can temporarily observe a mixed oracle state.

This can create temporary pricing and funding discontinuities after a beacon migration. Depending on module behavior and market state, positions may appear healthier or less healthy than they would under a freshly seeded beacon EMA.

This is best treated as an informational/operational risk if beacon replacement is an admin-controlled, rare migration event. It becomes more severe only if beacon migrations are expected to happen frequently or during active stressed markets.

## RECOMMENDATION

Use an explicit beacon-migration routine that defines how beacon-derived EMA state should be initialized.

Reasonable options:

- Reset `snap.emas.index` to the new beacon spot during migration.
- Reinitialize all beacon-dependent EMA/rate state.
- Pause sensitive liquidation/backstop actions around beacon migrations.
- Document that beacon switches preserve historical EMA state and should only be used when the old and new oracle histories are compatible.

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-04

RESOLVED

# Funding Uses Cached Rate

|          |                                 |
|----------|---------------------------------|
| CATEGORY | Warning                         |
| LOCATION | src/libraries/PerpLogic.sol:586 |
| REVIEW   | Remediation Review 1            |

## DESCRIPTION

`accrue` charges funding for the full elapsed interval using the previously cached funding rate. Only after accrual does the protocol refresh rates.

This means the first market interaction after a beacon/index update settles the entire idle period at the old cached funding rate. The refreshed funding rate only applies going forward.

This is especially relevant because beacon updates occur outside the Perp market. If the beacon moves while the Perp market is idle, the Perp contract does not know the exact time of the oracle change unless someone calls `touch` or another market action.

Funding around oracle updates can therefore be path-dependent. The first user or keeper to touch the market after a large oracle move determines when the old rate stops applying. This can overcharge or undercharge positions for the idle interval between the last Perp touch and the oracle update, and in extreme cases can affect liquidation or backstop timing.

This is best treated as an informational design limitation if the protocol accepts lazy market accrual and keeper-driven synchronization. It becomes more serious if funding is expected to precisely track oracle-update timing.

## RECOMMENDATION

Document the lazy funding accrual model and its limitations.

If more precise accounting is desired:

- Sync Perp markets whenever beacon updates occur.
- Record oracle update boundaries and split funding accrual across pre-update and post-update rates.
- Use beacon-provided historical/TWAP data when accruing over idle intervals.
- Consider limiting liquidation/backstop sensitivity immediately after long idle periods.

---

**I-04** Funding Uses Cached Rate

CONTINUED

---

— **RESOLUTION**

Status: Resolved.

## INFORMATIONAL I-05

RESOLVED

# Timelock Survives Owner Transfer

CATEGORY Warning  
LOCATION src/Perp.sol:215  
REVIEW Remediation Review 1

## DESCRIPTION

Queued timelock actions are stored by raw calldata, and execution only checks whether the current `msg.data` has a matured timestamp.

The queued action is not bound to the owner who submitted it, and execution does not require the current owner. Ownership transfer is inherited from Solady `Ownable`, and `Perp` does not override ownership transfer or handover paths to clear pending timelock entries.

As a result, a timelock action queued before an ownership transfer can remain executable after the new owner takes control.

After a market ownership handover, previously queued module changes or selector abdications can still be executed. This can surprise a new owner and may preserve stale governance intent from the previous owner.

This is best treated as informational if ownership transfers are rare, trusted, and accompanied by off-chain review of pending actions. It becomes more serious if markets are expected to be sold, transferred, or handed over without a reliable pending-action audit.

## RECOMMENDATION

Document that pending timelock actions survive ownership changes, or bind queued actions to an ownership epoch.

Possible mitigations:

- Invalidate pending actions on ownership transfer, ownership handover, and renounce ownership.
- Bind `executableAt` entries to an owner epoch or proposer nonce.
- Require the current owner or a designated executor to execute matured actions.
- Provide enumerability or a cancellation sweep so a new owner can review and clear inherited pending actions.

---

**I-05** Timelock Survives Owner Transfer

CONTINUED

---

— **RESOLUTION**

Status: Resolved.

## INFORMATIONAL I-06

RESOLVED

# Util Fees Use Latest Beacon Spot

|          |                                 |
|----------|---------------------------------|
| CATEGORY | Warning                         |
| LOCATION | src/libraries/PerpLogic.sol:586 |
| REVIEW   | Remediation Review 1            |

## DESCRIPTION

`accrue()` snapshots the current beacon index only when settlement occurs, derives the current mark price, and then applies that mark price to the entire elapsed interval since `lastTouch`.

This means utilization fees are not charged against a historical path of index/mark prices over the elapsed period. If the beacon index moves materially after a long idle interval, the next `touch()` or position action applies the latest mark to the whole interval.

Utilization fee settlement can differ from a time-weighted historical result. Depending on the direction of the beacon move, takers may pay more or less than they would under path-aware accounting, and makers receive the corresponding distorted utilization fees.

This is best treated as an accounting/design limitation unless the protocol promises time-weighted historical utilization fee accrual. It may be acceptable if the team intentionally uses latest-observation settlement for simplicity.

## RECOMMENDATION

Document that utilization fees are settled using the latest mark over the elapsed interval, or switch to path-aware accounting. A stronger design would checkpoint oracle/mark changes, accrue at each oracle update, or use a time-weighted historical mark from the beacon layer when computing elapsed utilization fees.

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-07

ACKNOWLEDGED

# Complementary Probability Rounding

CATEGORY Warning  
LOCATION src/core/standalone/basefns/CGBM.sol:102-105  
REVIEW Remediation Review 1

## DESCRIPTION

CGBM independently calculates  $pHat = mUp / total$  and  $oneMinusPHat = mDown / total$ . Because both fixed-point divisions round down, their sum can equal  $WAD - 1$  rather than exactly  $WAD$ . The resulting one-wei probability deficit enters the variance and return calculations. This violates exact probability conservation, but no material price deviation or exploitable accumulation has been demonstrated, so the issue does not currently justify Low severity.

## RECOMMENDATION

If exact conservation is required, calculate both probabilities from one shared full-precision division and explicitly allocate the remainder. Do not simply derive one side as  $WAD - pHat$  without testing directional rounding bias, because assigning every remainder to one branch can introduce a larger systematic asymmetry than the current one-wei deficit.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-08

ACKNOWLEDGED

# One-Sided Updates Can Vanish

CATEGORY Informational  
LOCATION src/core/standalone/basefns/CGBM.sol:97-105  
REVIEW Remediation Review 1

## DESCRIPTION

During a sufficiently long one-sided sequence, the opposite directional mass decays until fixed-point division rounds `pHat` or `oneMinusPHat` to zero. Further reports in the dominant direction are then multiplied by zero and produce no latent movement until an opposite report arrives. The behavior is real, but production-style parameters require thousands of consecutive maximum-confidence reports and the effect is the terminal rounding point of CGBM's expected diminishing same-direction response. Without a practical manipulation or loss scenario, Informational severity is appropriate.

## RECOMMENDATION

Prevent directional probabilities from rounding exactly to zero or one while residual mass remains. This can be done with higher-precision probability calculations, a calibrated residual prior, or a small probability floor that preserves nonzero same-direction responsiveness without creating excessive drift.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-09**

ACKNOWLEDGED

# Tick spam raises taker swap gas along crossed path

CATEGORY Gas Griefing  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

Makers can permissionlessly initialize dense single-tick positions with tiny liquidity and the minimum opening margin. Taker opens/closes, adjusts, and liquidations execute Uniswap V4 Pool.swap and then PerpTickLogic.crossTicksAndAccrueLpFees, so gas grows with each initialized tick crossed and there is no per-swap crossing cap.

Impact: An attacker can materially increase gas for swaps and liquidations traversing a spammed price region, causing keeper/bot transactions with fixed gas budgets to revert. Under default PriceImpact bounds this is bounded gas griefing rather than full block-limit DoS, so Medium severity is appropriate.

## RECOMMENDATION

Add an initialized-tick crossing cap, increase the economic cost of initializing narrow/dust ranges, enforce minimum range width/liquidity, and/or redesign Perp tick accounting to avoid a second full traversal.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

# Dust ranges can add zero-output exact-output liquidation tolls

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 1

## — DESCRIPTION

Exact-output V4 swap steps can cross an initialized dust-liquidity range when rounded amountOut is zero but rounded amountIn is nonzero. Perpcity liquidateTaker uses an exact-output swap for short liquidation fills and has no victim-side amt1Limit, so dust ranges on the path can add a small input toll while providing no output for those steps.

Impact: The revised PoC shows a real but bounded effect: 20 one-tick dust ranges increased an identical short liquidation victim cost from 9,859,812 to 9,914,456 USDC atoms, an extra 54,644 atoms (~\$0.055). Default PriceImpact bounds and tickSpacing cap the number of crossed ranges to dozens, and seeding the ranges requires about 100 USDC of maker margin, so this is low-severity economic griefing rather than profitable extraction or bad-debt drain.

## — RECOMMENDATION

Avoid advancing exact-output swap steps to a target tick when rounded amountOut is zero, or avoid charging amountIn for zero-output steps. At the Perp layer, consider a max-USD budget for permissionless liquidation swaps.

## — RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

RESOLVED

# Maker-to-taker conversion skips MIN\_OPENING\_MARGIN floor

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 1

## — DESCRIPTION

During full maker liquidity removal, `adjustMaker()` only applies the `MIN_OPENING_MARGIN` floor before residual amount1 is folded into margin. If the position converts to a taker, the branch enforces taker initial-margin health but does not re-apply the 5 USDC opening floor that `openTaker()` checks before and after fees.

Impact: A maker can convert into an active taker position with settled margin below the direct `openTaker` floor. This can consume taker open-interest capacity and leave uneconomic dust positions that are costly to liquidate/backstop relative to their notional. The leverage-escalation aspect is excluded as duplicate of the accepted `adjustTaker` liquidation-ratio issue; no direct fund loss or bad debt is shown here.

## — RECOMMENDATION

After folding residual amount1 into margin in the maker-to-taker conversion branch, enforce `MIN_OPENING_MARGIN` before storing a nonzero taker, or close/reject positions whose residual margin is below the floor.

## — RESOLUTION

Status: Resolved.

## INFORMATIONAL I-12

RESOLVED

# Zero-capacity shadow liquidity dilutes honest LP fees

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

Maker capacity is calculated and added per liquidity delta using floored amount0 values. Sub-threshold narrow-range additions can therefore add active Uniswap liquidity while contributing zero maker.capacity and zero global s.cap. LP fee accrual later pays raw active liquidity via `makerLpFeesAccrued/lpFeeGrowthInside`, so zero-recorded-capacity shadow liquidity can collect a small share of taker-paid LP fees.

Impact: Honest makers that provide the recorded capacity admitting taker flow can be slightly diluted by active shadow liquidity that is not counted in capacity/utilization accounting. The impact is dust-bounded per zero-capacity slice and the updated PoC shows only very small default-module fee diversion, so this is Low severity rather than Medium.

## RECOMMENDATION

Reject positive liquidity additions whose computed capacity increment is zero, or accumulate capacity without per-delta flooring loss. If the intended reward basis is risk capacity, distribute LP fees using recorded/capacity-weighted liquidity rather than raw active liquidity alone.

## RESOLUTION

Status: Resolved.

INFORMATIONAL **I-13**

RESOLVED

# openTaker utilization failures occur after swap execution

CATEGORY Gas Optimization  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

openTaker() performs the PoolManager swap and tick-crossing fee replay before applyTakerSwapState() updates open interest and checks utilization. For new taker opens, the post-trade OI from p.perpDelta is knowable before the swap, so capacity-exceeding orders can be rejected earlier.

Impact: Caller-paid gas griefing/UX impact only. If capacity is consumed before a pending openTaker executes, the transaction can spend the swap path gas before reverting with ShortUtilizationExceeded/LongUtilizationExceeded. The PoC measured about 119k gas for the late revert versus about 53k for a pre-swap margin revert; no protocol state survives and no protocol funds are lost.

## RECOMMENDATION

Precompute the post-open OI from p.perpDelta and s.oi, then call the utilization check before PerpAmmLogic.swap() for openTaker(). Keep the post-swap check as the authoritative state update.

## RESOLUTION

Status: Resolved.

INFORMATIONAL **I-14**

RESOLVED

# Insurance consumption lacks complete bad-debt event coverage

CATEGORY —

LOCATION —

REVIEW Remediation Review 1

## — DESCRIPTION

accountBadDebt() reduces feeFund.insurance when negative equity is covered by insurance, but the fully insured branch returns without emitting BadDebtAccounted. In the underinsured branch it emits only the original loss, post-depletion insurance balance, and cumulative badDebt, without an explicit insuranceConsumed value.

Impact: On-chain accounting is unchanged, but event-only insurance and risk monitors can miss fully insured loss events and must infer partial insurance consumption from external/pre-state context.

## — RECOMMENDATION

Emit an insurance-consumption/bad-debt accounting event on both branches, and include explicit insuranceConsumed plus insuranceAfter fields.

## — RESOLUTION

Status: Resolved.

INFORMATIONAL **I-15**

RESOLVED

# openTaker MarginTransferred reports pre-fee totalMargin

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 1

## — DESCRIPTION

openTaker() calls transferMargin() with the gross posted margin, causing MarginTransferred to emit totalMargin after the deposit. It then calls removeSwapFeesFromMargin(), which subtracts protocol, creator, and insurance fees from solvencyState.totalMargin without emitting a follow-up totalMargin update.

Impact: Event consumers that treat MarginTransferred.totalMargin as the final solvency total for the transaction overstate totalMargin by the non-LP fee amounts. TakerOpened exposes fee amounts, so this is an off-chain reconstruction issue rather than an on-chain accounting flaw.

## — RECOMMENDATION

Either move fee removal before the margin-transfer event for openTaker, or emit a dedicated fee-reclassification/totalMargin update event after removeSwapFeesFromMargin().

## — RESOLUTION

Status: Resolved.

## INFORMATIONAL I-16

RESOLVED

# Terminal MAX\_TICK leaves no active maker liquidity for buy-side fills

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

PerpFactory allows initialization at the inclusive upper sqrt-price bound while Perp maker ranges are capped at MAX\_TICK. Because V4 liquidity is active only for  $\text{currentTick} < \text{tickUpper}$ , once the pool is moved to  $\text{currentTick} == \text{MAX\_TICK}$  all allowed maker ranges have  $\text{tickUpper} \leq \text{currentTick}$  and active pool liquidity is zero. No new allowed maker range can be active at that exact terminal tick.

Impact: At the terminal price, taker adjustments or closes that require buying perp cannot fill the exact requested delta and revert until a sell moves the pool back below MAX\_TICK. This is a terminal-price liveness/consistency flaw; bad debt or material liquidation loss was not demonstrated.

## RECOMMENDATION

Treat the upper bound as exclusive or otherwise ensure the pool cannot remain at  $\text{currentTick} == \text{MAX\_TICK}$  with all allowed maker ranges inactive; alternatively allow a maker upper tick above the maximum reachable active tick.

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-17

RESOLVED

# Taker liquidation eligibility ignores mandatory swap fees

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 1

## DESCRIPTION

`liquidateTaker()` determines eligibility using `isHealthy(equityBefore - liqFeeAmt, valBefore, pos.liqMarginRatio)` before executing the forced AMM close. The liquidation execution path then always charges LP/protocol/creator/insurance swap fees via `PerpAmmLogic.swap()` and debits `sr.totalFeeAmt` from the same position margin (`netMargin = settledMargin - sr.totalFeeAmt - liqFeeAmt`). As a result, a taker can be above the pre-check after only the liquidation fee but below maintenance after the mandatory close swap fees that liquidation itself will charge.

## RECOMMENDATION

Consider to include all mandatory closing costs in liquidation safety.

## RESOLUTION

Status: Resolved.

INFORMATIONAL **I-18**

RESOLVED

## Backstops can hand over positions that remain liquidatable after the liquidation fee

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 1

### — DESCRIPTION

backstopTaker() and backstopMaker() only require post-backstop equity to satisfy the liquidation margin ratio using full equity, while liquidateTaker() and liquidateMaker() decide liquidatability after subtracting the explicit liquidation fee. A minimum accepted backstop can therefore leave the transferred NFT immediately liquidatable in the next transaction.

### — RECOMMENDATION

After adding the backstop margin, require the position to pass the same fee-inclusive health calculation used to determine liquidation eligibility. This check should reserve the full prospective liquidation fee and any mandatory closing-cost buffer before transferring the NFT. Prefer a shared health helper for liquidation and backstop checks so an accepted backstop cannot be immediately liquidated under the same price snapshot.

### — RESOLUTION

Status: Resolved.

INFORMATIONAL **I-19**

ACKNOWLEDGED

# Unsigned Initial Price Enters TWAP

CATEGORY Best Practices  
LOCATION src/libraries/TwAvg.sol:42-49  
REVIEW Remediation Review 1

## — DESCRIPTION

The constructor sets `_index` from an unsigned deployment parameter and immediately initializes TWAP history. On the first signed update, the beacon records that initial index for the entire elapsed interval before applying the authenticated measurement.

TWAP queries can therefore remain dominated by a value that was never authenticated by the configured verifier. A stale or malicious initial value can influence pricing, funding, or liquidation systems during bootstrap.

## — RECOMMENDATION

Do not start TWAP accumulation until the first verified update. Alternatively, include the initial index in a signed initialization payload and initialize spot and TWAP state from that authenticated observation.

## — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-20

ACKNOWLEDGED

# Oracle Configurations Need Safety Checks

|          |                      |
|----------|----------------------|
| CATEGORY | Best Practices       |
| LOCATION | Global               |
| REVIEW   | Remediation Review 1 |

## DESCRIPTION

The beacon factories and component constructors primarily validate local type and nonzero constraints. They do not define or enforce a complete economically safe parameter envelope for each supported component composition.

As a result, constructor-valid configurations can produce behavior that is mathematically consistent with the supplied parameters but unsuitable for a live perpetual market. Depending on the selected components and values, a beacon can:

- round valid non-neutral signals to zero;
- remove confidence sensitivity through zero or extreme exponents;
- make one directional branch effectively unresponsive;
- produce zero or unexpectedly large transformed indices;
- exceed a downstream consumer's narrower numeric domain;
- lose material precision when converting between index and latent space;
- overflow intermediate arithmetic before a later clamp is applied;
- exhaust a transform's supported latent range after valid updates;
- revert because rehydrated EMA values are outside a numerically safe ratio;
- overflow a TWAP cumulative value for a sufficiently large index and elapsed interval; or
- deploy component combinations whose output and input domains are incompatible.

These cases generally require pathological parameter choices, extreme initial state, or assumptions about downstream consumer types. They do not establish a permissionless attack against a correctly configured market. However, the contracts accept many such configurations without warning, and users cannot determine from successful deployment alone whether a beacon is safe for its intended lifetime and consumer.

Impact: A creator can accidentally deploy an oracle that is inert, excessively sensitive, non-idempotent, or unable to process otherwise valid reports. If a

**I-20** Oracle Configurations Need Safety Checks

CONTINUED

market is created against that oracle, the resulting stale or out-of-domain index may disrupt pricing, accrual, trading, or liquidation until governance or the creator replaces the configuration.

The impact depends on creator choices and market integration. For this reason, the broader issue is best treated as deployment hardening rather than assigning independent security severity to every constructor-valid parameter edge.

**— RECOMMENDATION**

Either enforce safe configuration envelopes on-chain or clearly make configuration safety an explicit deployer responsibility supported by tooling and documentation.

**— RESOLUTION**

Status: Acknowledged.

## CRITICAL C-01

ACKNOWLEDGED

# Exposure increases bypass the initial margin requirement

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/libraries/PerpLogic.sol#L335-L391>

REVIEW Remediation Review 2

## DESCRIPTION

adjustTaker (and adjustMaker) select the post-action health threshold solely from the sign of marginDelta: withdrawals use the snapshotted initMarginRatio, while zero-margin or positive-margin adjustments—including exposure increases—use liqMarginRatio.

The check never inspects whether abs(perpDelta) increased or the position flipped side.

openTaker gates new exposure at initMr, but adjustTaker allows ratcheting to ~2x notional (2.2 vs 1.2 perp in the PoC) without posting incremental init-tier collateral, leaving health between maintenance and initial tiers (e.g. ~6-9% vs 10% init / 5% liq with default MarginRatios).

Consequently, an attacker can open a small valid position and increase it with `marginDelta == 0`, requiring only the 5% liquidation margin instead of the 10% initial margin.

The PoC grows a long to 5.4072% health, opens an opposing short, liquidates the undercollateralized long, and closes the short atomically. It turns \$27 into \$29.757090 without collecting the liquidation reward and leaves \$4.734436 of bad debt. Open interest and the AMM price return to their starting state. A malicious actor can scale this attack with more capital and in a loop to drain the market.

## RECOMMENDATION

Select the required margin tier based on the adjustment's economic effect, not `marginDelta`:

- Require the initial margin ratio whenever `adjustTaker` increases absolute exposure or flips the position to the opposite side.
- Apply the same rule to maker liquidity additions or other adjustments that increase maker exposure or capacity.
- Allow maintenance-margin checks only for genuinely risk-reducing adjustments that do not worsen position health.

**C-01** Exposure increases bypass the initial margin requirement

CONTINUED

Perform these checks using fully settled post-action equity and position value, including funding, utilization fees, swap fees, and the post-swap mark price.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/962e209bcf10e72f464e6aaf9f63a0b5fbad1dba/test/guardianaudits/pocs/P0CExposureIncreaseInitialMarginBypass.t.sol>

**— RESOLUTION**

Status: Acknowledged.

## CRITICAL C-02

ACKNOWLEDGED

# Same-block mark manipulation enables profitable atomic self-liquidation

|          |                      |
|----------|----------------------|
| CATEGORY | Logical Error        |
| LOCATION | GLOBAL               |
| REVIEW   | Remediation Review 2 |

## DESCRIPTION

The mark price gives the live AMM price 50% weight, while same-block accrual leaves both EMAs unchanged. An attacker can therefore manipulate the AMM, withdraw collateral against an inflated mark, reverse the manipulation, and self-liquidate the resulting undercollateralized position.

The PoC performs the complete attack atomically with two position NFTs:

1. Open a 250-perp long and a 330-perp helper short, both above 10% initial margin.
2. Confirm that a `$908.780490` withdrawal fails before manipulation.
3. Flip the helper from short to long, increasing the live AMM mark while `dt == 0` preserves the EMAs.
4. Execute the same withdrawal successfully.
5. Restore the helper short. The stripped long now has 4.7945% health and is liquidatable.
6. Self-liquidate the long, claim the liquidation reward, and close the profitable short.

The result is:

- Attacker capital: `$4,000`
- Final balance: `$4,064.011402`
- Profit: `$64.011402` before gas
- Gross liquidation deficit: `$77.427951`
- Final bad debt: `$73.553304`
- Final open interest: zero
- Final AMM price: restored

A malicious actor can scale this attack with more capital and in a loop to drain the market.

**C-02** Same-block mark manipulation enables profitable atomic self-liquidation

CONTINUED

The attack does not rely on the separate `adjustTaker` margin-tier bypass as shown in C-01. Instead the profit depends on the acknowledged "M-03 Insolvent Liquidations Pay Fees" behavior that pays liquidation rewards even when the liquidated position is insolvent. Without the `$123.749262` self-liquidation reward, this route loses `$59.737860`.

**— RECOMMENDATION**

Consider to cap liquidation fees by recoverable equity.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/f94e957b07658a86f3f9962daa46b5357a256f88/test/guardianaudits/pocs/P0CL19AtomicSelfLiquidation.t.sol>

**— RESOLUTION**

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

# Position Actions Lack Mark-Price Bounds

|          |                      |
|----------|----------------------|
| CATEGORY | Validation           |
| LOCATION | GLOBAL               |
| REVIEW   | Remediation Review 2 |

## — DESCRIPTION

The protocol uses two relevant prices: the AMM price determines exchanged token amounts, while the beacon-derived mark price determines position value and health. User parameters only constrain the AMM amounts and do not specify an acceptable beacon/mark-price range or transaction deadline.

Consequently, a pending position action can execute after a substantial beacon update while still satisfying its AMM slippage limits. The resulting position may have materially different leverage, equity, or liquidation risk than the user expected, potentially opening or modifying an immediately unsafe position.

This is particularly important because Perp City intends to support highly leveraged markets and permissionless oracles for a broad range of underlying assets. Some of these markets may have volatile mark prices. Combined with high leverage, even a relatively short execution delay or moderate mark-price movement could materially change a position's health.

## — RECOMMENDATION

Consider to add a caller-specified minimum/maximum beacon index or mark price to economically sensitive position actions.

## — RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

# Sandwiched reports poison index EMA and enable wrongful taker backstops

CATEGORY Logical Error  
LOCATION GLOBAL  
REVIEW Remediation Review 2

## — DESCRIPTION

After market idleness, an adversarial relayer can publish consecutive valid signed reports by first relaying an extreme report and calling `touch()`, which applies that new spot over the full elapsed EMA interval. Relaying the next normal report and touching again in the same timestamp leaves `emaIndex` poisoned because `calcEma()` preserves it at `dt==0`, even though the public beacon has returned to normal. Pricing then produces a mark below both live oracle and AMM prices.

Impact: A healthy long can be made backstoppable at a normal final beacon value. The attacker supplies temporary backstop margin, seizes the NFT, waits for EMA convergence, and closes at the normal prices. The production-verifier PoC demonstrates net USDC profit exceeding half of the victim's settled margin.

## — RECOMMENDATION

Consider to reconcile EMA state with beacon observation timing instead of applying a newly observed spot across the full idle interval. At minimum, prevent same-timestamp report/touch sequences from preserving an intermediate report's poisoned EMA for seizure-sensitive risk checks.

## — RESOLUTION

Status: Acknowledged.

HIGH **H-03**

ACKNOWLEDGED

# Perp Markets Accept Stale Beacon Prices

|          |                      |
|----------|----------------------|
| CATEGORY | Oracle               |
| LOCATION | GLOBAL               |
| REVIEW   | Remediation Review 2 |

## DESCRIPTION

In `PerpLogic accrue()`, the current index is read directly: `snap.spots.index = uint128(env.modules.beacon.index());`

That value then feeds mark price, funding, EMA refreshes, price-impact bounds, health checks, and liquidation logic. However, the beacon interface only exposes the `index()` and does not expose any freshness metadata and the beacon implementation keeps returning the last stored index until another valid update is submitted.

As a result, if the off-chain beacon signer/verifier/relayer pipeline stops producing updates, perp markets continue operating against the last published index indefinitely. This can happen accidentally due to infrastructure outages, signer downtime, verifier failures, or relayer failures. A malicious actor may also intentionally exploit this by DDoSing the off-chain signer, verifier, or relayer infrastructure, preventing fresh beacon updates while the perp market continues accepting the last published index indefinitely.

This is especially dangerous for volatile markets. If the real spot price moves materially while the beacon index remains frozen, the protocol can compute distorted mark prices, funding rates, price-impact limits, margin health, and liquidation eligibility. Positions may be incorrectly protected from liquidation, incorrectly liquidated, or allowed to trade against stale risk assumptions.

## RECOMMENDATION

Consider to add explicit freshness enforcement.

## RESOLUTION

Status: Acknowledged.

HIGH **H-04**

ACKNOWLEDGED

# Unsafe Low-Fee Configurations Enable Profitable Atomic Self-Liquidation

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/962e209bcf10e72f464e6aaf9f63a0b5fbad1dba/src/modules/Fees.sol>

REVIEW Remediation Review 2

## DESCRIPTION

The fee module enforces maximum fees but no minimum effective fee or joint solvency constraint. A market can therefore be configured with trading fees too low to offset manipulation and liquidation incentives.

The PoC shows a similar attack vector than “C-01 Exposure increases bypass the initial margin requirement” without the need to bypass the initial margin requirement if fees are too low:

- Opens a long that satisfies the 10% initial-margin requirement.
- Opens a larger short that moves the live-AMM-weighted mark and makes the long liquidatable.
- Self-liquidates the long and closes the short in the same transaction.

With the default 0.01% protocol fee still enabled and only the market-selected creator, insurance, and LP fees set to zero, the attacker earns \$3.928228 before gas and leaves \$10.101563 of final bad debt. A malicious actor can scale this attack with more capital and in a loop to drain the market.

## RECOMMENDATION

- Enforce a protocol-level minimum effective trading fee or, preferably, validate the complete market configuration against a worst-case solvency invariant. This must consider all fee components, liquidity, margin ratios, liquidation fee, and the price-impact bound—not an isolated static fee floor.
- As defense in depth, prevent positions from being opened or risk-increased and then closed or liquidated until a later block. The restriction must be position/block based rather than `msg.sender` based because multiple accounts can bypass sender restrictions.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/962e209bcf10e72f464e6aaf9f63a0b5fbad1dba/test/guardianaudits/pocs/P0CLowFeeAtomicSelfLiquidation.t.sol>

---

**H-04**     Unsafe Low-Fee Configurations Enable Profitable Atomic Self-Liquidation

---

CONTINUED

---

— RESOLUTION

Status: Acknowledged.

HIGH **H-05**

ACKNOWLEDGED

# EMA-Anchored Price Bounds Enable Profitable Ratchet Liquidations

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/68093f9c48dc05dc917ed94cc458f06893579a6f/src/modules/PriceImpact.sol#L43-L44>

REVIEW Remediation Review 2

## DESCRIPTION

`PriceImpact` only bounds each post-trade AMM price against the AMM's own EMA and ignores the Beacon price. An attacker can repeatedly move the AMM to the permitted boundary, wait for its EMA to follow, and repeat. Every trade passes validation while the AMM, its EMA, and consequently the mark price move progressively farther from the unchanged Beacon without any Beacon-relative ceiling.

This allows an attacker to make healthy positions appear insolvent, liquidate them against the manipulated AMM, and subsequently reverse the manipulation. Victims can lose collateral and the unfavorable forced execution can leave protocol bad debt.

The production module does not hardcode a 10% bound: it is owner-configurable and permits values up to 100%. Nevertheless, the repository consistently treats 10% as its reference configuration in tests.

At those reference parameters, one permitted 10% AMM movement can immediately move the mark by approximately 5% while the EMAs remain unchanged. This is comparable to the entire buffer between the reference 10% initial margin and 5% liquidation threshold.

Therefore, the reference bound can already permit liquidation-sized instantaneous manipulation before considering the larger cumulative ratchet. Lowering the bound would reduce the immediate effect but would not eliminate cumulative drift.

## RECOMMENDATION

Consider to add a cumulative, directional Beacon constraint, for example: movements toward the Beacon could remain permitted, while movements away could be stopped at a configured maximum divergence. An alternative approach would be to consider to update the used price for liquidations.

---

**H-05** EMA-Anchored Price Bounds Enable Profitable Ratchet Liquidations

---

CONTINUED

— RESOLUTION

Status: Acknowledged.

HIGH **H-06**

ACKNOWLEDGED

## Reported capacity does not guarantee executable close liquidity

CATEGORY DoS  
LOCATION GLOBAL  
REVIEW Remediation Review 2

### — DESCRIPTION

The Original issue: "H-07 — \*Stripping in-range liquidity while out-of-range capacity remains freezes underwater taker liquidations" was acknowledged as t\*he client stated that Uniswap can traverse empty ticks, that liquidation is primarily limited by the price-impact bound, and that replacement liquidity or backstopping resolves otherwise unclosable positions.

But as the provided POC showcases closing a long moves the AMM price downward—away from that range. Uniswap therefore cannot traverse toward and use the liquidity counted as capacity. Consequently, both the full close and liquidation revert with `InsufficientLiquidityToFill`.

This isolates unavailable swap-direction liquidity as the cause, rather than the configured price-impact bound.

Replacement liquidity and backstopping are mitigations, but do not refute H-07:

- Adding new in-range liquidity restores liquidation, but requires external capital.
- Before the position reaches the backstop threshold, liquidation can fail while `backstopTaker` still reverts with `NotLiquidatable`.
- Once backstopping becomes available, it recapitalizes and transfers the position; it does not restore executable close liquidity. The new owner's full close still fails.
- Bad debt is not recorded while liquidation remains blocked. Once executable liquidity is restored, the same liquidation succeeds and realizes the bad debt.

### — RECOMMENDATION

Tie capacity used for utilization to liquidity executable for the relevant close/liquidation direction and prevent in-range removals that leave outstanding OI without executable depth.

**H-06** Reported capacity does not guarantee executable close liquidity

CONTINUED

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/main/test/guardianaudits/pocs/POCH07WrongSideCapacity.t.sol>

**— RESOLUTION**

Status: Acknowledged.

HIGH **H-07**

ACKNOWLEDGED

# Transient Mark PnL Bypasses Initial Margin

|          |                                                                                                                                                                                                                                                                                                                                   |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Oracle                                                                                                                                                                                                                                                                                                                            |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/3dbdc0ad268e9d9e2acb81e6b7db1945a6c93321/src/libraries/PerpLogic.sol#L298-L333">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/3dbdc0ad268e9d9e2acb81e6b7db1945a6c93321/src/libraries/PerpLogic.sol#L298-L333</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                                                                                                                                              |

## DESCRIPTION

After a valid Beacon update, the mark immediately uses the new index while subtracting the lagging index EMA. `openTaker()` counts the resulting temporary mark PnL as equity, allowing an attacker to open more exposure than their real collateral supports. When the EMA converges, the paper profit disappears; the attacker can self-liquidate, collect the liquidation fee, and leave bad debt.

## RECOMMENDATION

Consider to not count favorable PnL created by the opening transaction toward initial margin. Require net deposited collateral, after fees, to independently satisfy initial margin using a conservative execution-aware position value.

## RESOLUTION

Status: Acknowledged.

HIGH **H-08**

ACKNOWLEDGED

# One-wei mark floor enables undercollateralized shorts and bad debt

CATEGORY Math

LOCATION <https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/modules/Pricing.sol#L19-L29>

REVIEW Remediation Review 2

## DESCRIPTION

Pricing floors a nonpositive premium-adjusted mark numerator to the integer 1, which is only one Q96 wei. After a sufficiently large but honest index spike and retrace, the stale index EMA can cancel a still-positive live index and AMM price. Taker position value then rounds to zero, and `isHealthy` substitutes denominator 1 for zero position value, allowing meaningful short exposure to open with minimal collateral.

## RECOMMENDATION

The `emaAmm - emaIndex` premium should only move the mark by a configured percentage of the live index/AMM anchor. It must never cancel almost the entire live price. Think about ways how to handle this case in a way that preserves a fair mark price.

## RESOLUTION

Status: Acknowledged.

HIGH **H-09**

ACKNOWLEDGED

# Mark-AMM divergence hides executable insolvency

CATEGORY Oracle  
LOCATION GLOBAL  
REVIEW Remediation Review 2

## — DESCRIPTION

Taker health and liquidation eligibility use the blended mark price, while positions ultimately settle against the AMM. During a sufficiently large mark-AMM divergence, a position can appear healthy at the mark even though its equity at the current AMM price is already negative. Liquidation is therefore blocked precisely while the position is insolvent at its executable price. The deficit remains latent until the mark converges, after which liquidation consumes insurance and records bad debt. The PoC also demonstrates a paired strategy that extracts profit while leaving protocol bad debt.

## — RECOMMENDATION

Consider to introduce a divergence-aware safety mode when the AMM price deviates materially from both the mark and beacon/index price. Before the position's executable-price equity becomes negative, permit bounded partial liquidation, auction, or backstop resolution using a manipulation-resistant eligibility price and protected execution. But do not let the live AMM price alone authorize liquidation.

Also Consider while such divergence persists to prevent risk-increasing opens, position increases, and margin withdrawals until the prices reconverge.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/main/test/guardianaudits/pocs/POCMarkAmmBasisGap.t.sol>

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

RESOLVED

# Deploy Script Still Uses Mock Contracts

CATEGORY Warning

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/68093f9c48dc05dc917ed94cc458f06893579a6f/script/Deploy.s.sol#L11-L15>

REVIEW Remediation Review 2

## DESCRIPTION

The deploy script still uses the mock contracts instead of the real implementations that are now implemented in the codebase.

## RECOMMENDATION

Update the deploy script.

## RESOLUTION

Status: Resolved.

MEDIUM **M-02**

RESOLVED

# Funding Cap Calc Uses Wrong Price

CATEGORY Math

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/68093f9c48dc05dc917ed94cc458f06893579a6f/src/modules/Funding.sol#L66>

REVIEW Remediation Review 2

## DESCRIPTION

As the `Funding` module documents correctly, funding should be capped as a percentage of the current `mark price`.

However, `funding()` calculates the cap using `spots.ammPrice`, which therefore can lead to wrong funding.

## RECOMMENDATION

Use the `mark price` instead.

## RESOLUTION

Status: Resolved.

## MEDIUM M-02.2

RESOLVED

# Out-of-domain beacon index lets liquidators drain shared collateral through mark-priced fees

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/PerpFactory.sol#L54-L57>

REVIEW Remediation Review 2

## DESCRIPTION

The factory validates only the bootstrap index against the fixed AMM price domain. Later accrual accepts any uint128 beacon index, so an otherwise valid update can make the mark unreachable by the AMM. A full liquidation values its fee at this mark but closes at the stale bounded AMM price; subtracting the oversized fee from closing equity books it as bad debt, after which the separate fee payout is only proportionally haircut by `socializeLoss` against `global totalMargin`.

Impact: A zero-capital permissionless liquidator can receive more than 90% of pooled USDC margin in one liquidation, leaving residual bad debt and impairing unrelated makers and takers. The passing PoC uses the same modules deployed by `Deploy.s.sol` and explicitly asserts the liquidator payout, `totalMargin` depletion, and remaining bad debt.

## RECOMMENDATION

Validate every live oracle index against the AMM-supported domain or disable forced actions when mark and executable price are incompatible. Also cap liquidation fees by realizable close economics and available position equity so a fee cannot itself manufacture system bad debt.

## RESOLUTION

Status: Resolved.

MEDIUM **M-03**

ACKNOWLEDGED

# Taker Limits Exclude Dynamic Swap Fees

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/modules/Fees.sol#L174-L190>

REVIEW Remediation Review 2

## DESCRIPTION

The newly added `Fees` module makes the LP fee an execution-time value derived from the pool's current active liquidity and the AMM spot-to-EMA deviation. `PerpAmmLogic.swap()` samples `startingLiquidity`, obtains the dynamic fee, and includes it in `sr.totalFeeAmt`.

However, `openTaker()` and `adjustTaker()` call `checkTakerAmountLimits(sr.delta, p.amt1Limit)` using only the AMM quote delta. After that check passes, `sr.totalFeeAmt` is deducted separately from the taker's margin. The caller's amount limit therefore does not bound the trade's total economic cost.

Between quoting/signing and execution, preceding liquidity changes or price/EMA updates can materially change the selected LP fee while the AMM quote remains within `amt1Limit`. The transaction can consequently succeed while charging more collateral and leaving the position with less margin and a smaller liquidation buffer than the taker authorized. This does not require public-mempool frontrunning; ordinary state changes before execution are sufficient.

## RECOMMENDATION

Add a caller-specified maximum total fee, maximum fee rate, or minimum post-fee margin to taker actions and validate it against `sr.totalFeeAmt` before applying the swap state. Alternatively, incorporate every swap fee into a signed total-cost limit.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

# Dormant markets force makers to reopen at stale AMM price and expose collateral to funding extraction

CATEGORY Warning  
LOCATION GLOBAL  
REVIEW Remediation Review 2

## — DESCRIPTION

V4 slot0 is written once at createPerp and cannot be reinitialized. After a zero-liquidity dormancy window with a large beacon move, accrual drives emaAmm toward the persistent stale slot0 while emaIndex converges to the live index. Production Pricing keeps the mark near stale AMM, and PricelImpact bounds swaps around emaAmm rather than the live index. Makers who bracket the observable stale tick are the only way to restore in-range liquidity; index-aligned ranges remain out of range and long attempts revert PricelImpactTooHigh. A permissionless searcher can back-run that reopen with a long at the stale execution price. The demonstrated realized extraction vector is protocol funding settlement on the persistent AMM/index spread—not immediate mark-to-index arbitrage, since execution and mark remain stale-anchored.

Impact: Dormant official markets can reopen in a broken state: index-following liquidity cannot serve longs at the live index, while stale-bracket liquidity enables permissionless cross-user USDC transfer via capped production funding. The PoC (production modules) shows >\$15 maker equity loss and >\$9 realized USDC withdrawal by a searcher over 30 days on a 0.4-perp long against \$200 maker margin, with the searcher retaining exposure. Exploitation requires a large publicly observable index/slot0 divergence, zero active liquidity, and an independent maker adding the uniquely functional stale-centered range.

## — RECOMMENDATION

Consider on the first liquidity add after a zero-liquidity epoch (or when  $|\text{amm-index}|$  exceeds a threshold), to require a re-anchoring: accrue and reset EMAs from the live beacon, clamp/refresh slot0 via a controlled path, or reject maker adds until AMM and index are within configured bounds. Or to document this behavior and perhaps recommend to redeploy the market in that case.

---

**M-04** Dormant markets force makers to reopen at stale AMM price and expose collateral to funding extraction **CONTINUED**

---

— **RESOLUTION**

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

## Backstops can hand over positions that remain liquidatable after the liquidation fee

CATEGORY Logical Error  
LOCATION —  
REVIEW Remediation Review 2

### — DESCRIPTION

backstopTaker() and backstopMaker() only require post-backstop equity to satisfy the liquidation margin ratio using full equity, while liquidateTaker() and liquidateMaker() decide liquidatability after subtracting the explicit liquidation fee. A minimum accepted backstop can therefore leave the transferred NFT immediately liquidatable in the next transaction.

### — RECOMMENDATION

After adding the backstop margin, require the position to pass the same fee-inclusive health calculation used to determine liquidation eligibility. This check should reserve the full prospective liquidation fee and any mandatory closing-cost buffer before transferring the NFT. Prefer a shared health helper for liquidation and backstop checks so an accepted backstop cannot be immediately liquidated under the same price snapshot.

### — RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

# Remote low-price maker ranges capture utilization rent with cheap capacity

CATEGORY Math

LOCATION <https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/libraries/PerpLogic.sol#L785-L817>

REVIEW Remediation Review 2

## — DESCRIPTION

When spot is above a maker range, `calcCapacityX64` credits the range's full potential base amount as short capacity regardless of its distance from spot. A near-MIN\_TICK quote-only range can therefore register base capacity comparable to a near-price maker while its mark-valued position is negligible and only minimum USDC margin is required. That remote capacity lowers the reference utilization rate yet receives the same capacity-proportional utilization growth even though it cannot be reached within ordinary price-impact bounds.

Impact: In the revised reference-Fees PoC, \$10 of margin and one atom of remote quote inventory register about 1.04 base units of short capacity versus the honest range's 1.01 units and \$100 margin. Over the modeled year, the remote maker withdraws about \$55.27 while the diluted honest maker receives about \$53.83. The honest-only \$1,702 comparison is theoretical because the \$100 taker would become liquidatable before a full year, so it should not be treated as collectible loss; nevertheless, the demonstrated rent capture is permissionless, capital-efficient, and scales with live OI. Remote capacity also suppresses the rate paid to legitimate near-price capacity without supplying executable liquidity in the allowed price band.

## — RECOMMENDATION

Weight capacity and utilization earnings by liquidity reachable within PriceImpact bounds or by distance from spot. Alternatively require position value or margin proportional to credited mark-price capacity so remote ranges cannot dominate base-capacity accounting at negligible collateral.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

# Margin operations lack bad-debt slippage protection

CATEGORY Validation  
LOCATION GLOBAL  
REVIEW Remediation Review 2

## — DESCRIPTION

Maker and taker opens, margin adjustments, backstops, withdrawals, and closes cannot bind execution to an acceptable `badDebt` value or recovery ratio. If market bad debt exists or changes between quoting and execution, deposits are credited at nominal value while improving existing claimants' recovery, whereas withdrawals and closes may return less USDC than expected through loss socialization.

In case of a market with a lot of bad debt a user could deposit funds into the system and by doing so lose a significant amount of them.

## — RECOMMENDATION

Consider to add slippage checks for this or to document this behavior.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

# Split maker liquidations repeatedly charge unchanged residual notional

CATEGORY Gaming

LOCATION <https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/libraries/PerpLiquidationLogic.sol#L62-L63>

REVIEW Remediation Review 2

## DESCRIPTION

The maker liquidation fee prorates the position's current value by liquidity removed, but that value includes residual directional notional which modifyLiq does not reduce. Splitting the same cumulative liquidity removal therefore includes the unchanged residual in the fee base repeatedly while still satisfying the strict health-improvement check.

Impact: A permissionless liquidator can split an otherwise equivalent partial liquidation sequence to extract more USDC from the distressed maker. The reference 1% fee PoC charges 235,371 atoms across splits versus 231,596 atoms in one removal; the excess scales with residual notional and can further erode collateral before the residual converts to a taker.

## RECOMMENDATION

Charge maker liquidation fees from value actually removed, or separately track residual exposure so unchanged notional is not re-prorated in every partial liquidation.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

# Self-matched trades can modestly bias maker funding receipts

CATEGORY Gaming  
LOCATION GLOBAL  
REVIEW Remediation Review 2

## — DESCRIPTION

A maker can use a sybil taker to trade against its own range, temporarily displacing the AMM and biasing the AMM-price EMA that determines cached funding. After unwinding the sybil leg, the decaying EMA premium can modestly increase funding paid by unrelated same-side takers and credited to the maker. This is largely an economic-design consequence of funding being based on the internal AMM premium and maker/taker accounts not being netted.

Impact: Under production modules, a one-day EMA, six-hour permissionless touches, and a 30-day comparison, the revised PoC increases victim funding cost by about 17.6 USDC on 50,000 USDC margin. The maker's incremental receipt is modest and the strategy requires substantial posted collateral plus a sybil round trip. This does not support a substantial collateral drain or High severity; it is an informational manipulation/design observation.

## — RECOMMENDATION

If this behavior is undesirable, derive funding from a less manipulable or externally anchored premium, further cap/smooth the AMM contribution, or consider exposure attribution/netting mechanisms. Monitor and encourage touch cadence so stale cached rates do not persist.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-10**

ACKNOWLEDGED

# Per-tick LP fee rounding slightly overcharges fragmented liquidity books

CATEGORY Validation  
LOCATION GLOBAL  
REVIEW Remediation Review 2

## — DESCRIPTION

LP fees are upward-rounded independently for every positive-volume price segment between initialized ticks. Fragmenting curve-equivalent liquidity into adjacent narrow ranges creates more fee calculations than one consolidated range, so an otherwise equivalent swap path can incur extra 6-decimal fee atoms.

Impact: The corrected PoC directly decodes SwapResult.lpFeeAmt and finds a 17-atom (\$0.000017) excess over a 0.4e6-perp open/close round trip. The effect is real but economically negligible, and fragmentation gas/collateral costs dominate it.

## — RECOMMENDATION

Accumulate quote volume before applying a single upward rounding where compatible with range attribution, carry/reconcile fractional fee dust across segments, or document this negligible fragmentation-dependent rounding behavior.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

# Beacon Updates Do Not Checkpoint Perp Markets

CATEGORY Compatibility  
LOCATION GLOBAL  
REVIEW Remediation Review 2

## — DESCRIPTION

Beacon indices update independently from the Perp markets that consume them. A Perp only refreshes its cached EMAs, funding rates, utilization rates, and lastMark when `touch()` or another accrual-dependent action is called. Consequently, multiple beacon observations can occur while the Perp remains untouched, causing intermediate oracle states to be omitted from its lazy accounting and leaving previously cached rates active over the idle interval.

## — RECOMMENDATION

Consider supporting optional best-effort hooks or an updater coordinator that calls `touch()` on registered dependent Perp markets after every beacon update, every configurable number of updates, or once a maximum elapsed-time threshold is reached.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-12**

ACKNOWLEDGED

# A Reverting Beacon Prevents Beacon Migration

CATEGORY DoS

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/68093f9c48dc05dc917ed94cc458f06893579a6f/src/Perp.sol#L273>

REVIEW Remediation Review 2

## DESCRIPTION

`Perp.setBeacon()` calls `PerpLogic accrue()` using the currently configured beacon before installing the replacement. If the old beacon's `index()` reverts or returns a value that cannot be converted to `uint128`, the accrual and migration both revert.

Because trading, adjustment, and liquidation paths also require accrual, a permanently broken beacon can freeze the entire affected market while simultaneously preventing governance from replacing it.

## RECOMMENDATION

Consider to add an emergency migration path that does not require a successful read from the old beacon.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-13**

ACKNOWLEDGED

# Blacklisted Users Are Not Liquidatable

|          |                                                                                                                                                                                                         |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | DoS                                                                                                                                                                                                     |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/main/src/Perp.sol">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/main/src/Perp.sol</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                    |

## — DESCRIPTION

The liquidation flows will transfer remaining USDC margin to the owner of the position.

If this owner is blacklisted by USDC the transfer will fail and therefore the whole liquidation transaction reverts.

This could lead to a position accruing more and more bad debt without anyone being able to liquidate it. A dangerous state for the market.

## — RECOMMENDATION

Consider to put the USDC transfer in a try catch block and in case of a revert, escrow the USDC margin so that the owner can claim it later.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-14**

RESOLVED

# Unnecessary Maker Position Close DoS

|          |                                                                                                                                                                                                                                                                                                                         |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | DoS                                                                                                                                                                                                                                                                                                                     |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/libraries/PerpLogic.sol#L209">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/libraries/PerpLogic.sol#L209</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                                                                                                                                    |

## DESCRIPTION

When a maker removes all liquidity, `adjustMaker()` first settles accrued maker funding/utilization fees directly against `pos.margin`:

```
pos.margin = (pos.margin.toInt256() + p.marginDelta + lpFees.toInt256() -  
timeFees.total()).toUInt256().toUInt128();
```

If accrued funding owed by the maker exceeds the position's settled margin, this conversion to `uint256` reverts unless the maker supplies additional margin in the same call. Later, in the full close/conversion branch, the protocol only nets `pos.margin + pos.delta.amount1()` before converting residual perp exposure into a taker position. Residual perp PnL is not usable to cover the funding debt during the maker close step.

As a result, a maker position can be blocked from closing when its cash margin is insufficient, even though the remaining perp exposure could potentially be converted to a taker position and closed immediately to settle the debt. The maker may be forced to acquire and deposit extra USDC only to unblock the close, paying external swap/bridge/transaction costs and taking on extra operational friction.

## RECOMMENDATION

Consider to allow full maker exits to account for the complete close-equity path instead of requiring nonnegative settled margin before conversion.

One approach is to support an atomic "remove maker liquidity and close residual taker exposure" flow, where funding debt, residual USD balance, residual perp close proceeds, and swap fees are netted before enforcing final nonnegative equity.

## RESOLUTION

Status: Resolved.

MEDIUM **M-15**

ACKNOWLEDGED

## Users can avoid socialized losses

CATEGORY Gaming  
LOCATION GLOBAL  
REVIEW Remediation Review 2

### — DESCRIPTION

Bad debt is created only when a full liquidation/close resolves with negative equity:

```
uint256 remainingEquity = accountBadDebt(s, equity);
```

If insurance is insufficient, the uncovered loss is added to `solvencyState.badDebt`.

Future margin withdrawals then call:

```
socializeLoss(s.solvencyState, marginDelta.abs())
```

and receive only:

```
withdrawal - socialized haircut
```

However, before the bad-debt transaction executes, `badDebt == 0`, so withdrawals are not haircut. A user who observes a pending liquidation expected to create bad debt can attempt to close/withdraw first and avoid the loss.

Example

1. Market has 100 USDC total margin and 0 badDebt.
2. Position A is liquidatable and will create 10 USDC bad debt after insurance is exhausted.
3. User B sees the pending liquidation and closes/withdraws 20 USDC first.
4. Since badDebt is still 0, User B receives the full 20 USDC.
5. Position A liquidation executes and records 10 USDC badDebt.
6. Remaining users' future withdrawals are haircut to repay the 10 USDC loss.

The loss is therefore not socialized across all users exposed at the time the bad debt was economically inevitable. It is socialized only across users who remain after the bad-debt transaction lands.

### — RECOMMENDATION

**M-15** Users can avoid socialized losses

CONTINUED

One potential mitigation would be a cooldown for payouts to make it more difficult to time this gaming vector.

**— RESOLUTION**

Status: Acknowledged.

MEDIUM **M-16**

ACKNOWLEDGED

# Taker liquidation eligibility ignores mandatory swap fees

CATEGORY Logical Error  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

`liquidateTaker()` determines eligibility using `isHealthy(equityBefore - liqFeeAmt, valBefore, pos.liqMarginRatio)` before executing the forced AMM close. The liquidation execution path then always charges LP/protocol/creator/insurance swap fees via `PerpAmmLogic.swap()` and debits `sr.totalFeeAmt` from the same position margin (`netMargin = settledMargin - sr.totalFeeAmt - liqFeeAmt`). As a result, a taker can be above the pre-check after only the liquidation fee but below maintenance after the mandatory close swap fees that liquidation itself will charge.

## RECOMMENDATION

Consider to include all mandatory closing costs in liquidation safety.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-17**

ACKNOWLEDGED

# Tightened PriceImpact bounds temporarily block incremental risk-reducing swaps

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/modules/PriceImpact.sol#L31-L33>

REVIEW Remediation Review 2

## DESCRIPTION

PriceImpact.setPriceImpactBound() immediately installs a smaller EMA-centered band without a transition rule for pools whose live spot is outside it. Because every swap must terminate inside the absolute band, small risk-reducing swaps that move spot toward the EMA but do not fully cross the gap revert.

Impact: After an honest shared-module bound reduction, granular closes and small liquidations can be unavailable until a sufficiently large corrective swap executes or permissionless accrual lets the EMA catch up. The PoC also confirms a full close and larger liquidations remain possible immediately, and small exits recover after two minutes in its setup, limiting this to a short-lived sizing and keeper inconvenience.

## RECOMMENDATION

When tightening the bound, allow swaps that monotonically move an out-of-band spot toward the permitted range, or use a transitional/grandfathered bound until spot or EMA convergence brings the pool inside the new range.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-18**

ACKNOWLEDGED

# Partial liquidations can sandwich pending maker backstops

CATEGORY Validation  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

backstopMaker commits only to a position ID, fixed margin deposit, and recipient. Since a backstoppable maker is also liquidatable, a searcher can partially liquidate it before a pending backstop, remove liquidity/capacity, and collect the liquidation fee while the unchanged backstop deposit still passes against execution-time state.

Impact: The controlled PoC shows the same 100 USDC backstop succeeds in both orderings, but the sandwich halves LP liquidity and capacity and reduces acquired equity by 0.597277 USDC, equal to the searcher's liquidation reward. This is bounded transaction-state slippage on an emergency recapitalization, not a solvency failure.

## — RECOMMENDATION

Allow backstoppers to bind execution to minimum remaining liquidity/capacity, minimum post-entry equity/value, or an expected position-state nonce/hash.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-19**

ACKNOWLEDGED

# Unsigned margin write blocks net-equity maker recapitalization

CATEGORY Logical Error  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

backstopMaker and adjustMaker cast the fee-settled maker margin to uint128 before their post-deposit health check includes unrealized PnL. If accrued funding or fees make settled margin negative while positive inventory PnL offsets much of the deficit, a deposit that would restore signed equity to the liquidation ratio reverts with SafeCastLib.Overflow unless it first repays the entire gross settled deficit.

Impact: Owners and backstoppers can be forced to deposit substantially more than the net-equity health requirement to remediate a distressed maker. The demonstrated state also blocks partial liquidation, so remediation may be delayed until someone supplies the gross amount or a full-removal path becomes available. The PoC does not show theft, permanent lock, or bad-debt socialization, so the impact is Low.

## — RECOMMENDATION

Evaluate post-deposit health using signed settled margin plus PnL before persistence. Preserve the unsigned margin invariant by explicitly representing the covered deficit—for example, with signed margin accounting or by folding it into the signed quote delta as the maker-to-taker conversion path does—rather than casting before the health gate.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-20**

ACKNOWLEDGED

# Opening collateral is omitted from same-transaction fee socialization

CATEGORY Math  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

openTaker executes the swap and socializes its protocol/creator fees against the existing  $\text{badDebt}/(\text{totalMargin}+\text{badDebt})$  ratio before transferMargin credits the opening collateral. Consequently, the fee haircut excludes fully backed collateral entering atomically with the fee-generating trade and is harsher than the post-opening solvency ratio.

Impact: During bad debt, this ordering reallocates slightly more protocol/creator revenue to debt repayment and slightly improves later margin payouts at fee recipients' expense. The revised PoC isolates roughly 0.034 USDC of excess diversion under its 500 USDC opening and high-fee configuration; it does not establish profitable insolvency laundering.

## — RECOMMENDATION

For openTaker, calculate fee socialization with a denominator that includes the incoming opening collateral, or restructure the atomic accounting so collateral is provisionally credited before swap-fee claims are socialized and rolled back on revert.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-21**

ACKNOWLEDGED

# Dust liquidity lets directional maker residuals bypass taker OI and utilization fees

CATEGORY Logical Error  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

Maker-to-taker conversion requires maker liquidity to reach exactly zero. After trading skews a maker's inventory, the maker can retain one liquidity unit while removing nearly the entire range, leaving material directional residual exposure that valPnl recognizes for health. The position nevertheless remains on the maker branch: its residual is excluded from open interest and settles through makerFeesAccrued rather than receiving taker utilization checkpoints and debits. The revised PoC uses the production Fees module and demonstrates this during concurrent nonzero short utilization: an explicit short taker accrues utilization and another maker earns it over the same interval, while the dust maker's parallel short residual accrues none.

Impact: A maker can defer conversion indefinitely and warehouse material residual exposure outside side-specific OI and taker utilization accounting. During active utilization this understates OI/rate-setting inputs and avoids future utilization compensation that would apply after a full-removal conversion, underpaying remaining capacity providers. The PoC establishes accounting and fee avoidance, but not a direct collateral drain, bad debt, or insurance loss; retaining the stricter maker margin tiers is also a collateral disadvantage.

## — RECOMMENDATION

Reclassify a maker when its uncovered directional residual materially exceeds the exposure supported by remaining liquidity, rather than only when liquidity is zero. Book that residual into OI and apply taker utilization checkpoints/debits, or reject adjustments that leave economically immaterial liquidity alongside a material residual.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-22**

ACKNOWLEDGED

# adjustTaker absolute health gate blocks incremental deleveraging

CATEGORY Validation  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

For a surviving taker with nonnegative marginDelta, adjustTaker requires absolute post-trade health at or above the stored liquidation ratio, even when the trade reduces exposure. Partial liquidation instead accepts positions whose health strictly improves. Thus an equity-positive but unhealthy holder cannot use repeated voluntary reductions that remain below the ratio; it must cross the floor in one adjustment, add collateral, or use liquidation where that path's stricter fee-adjusted checks permit it. Negative-equity voluntary full closes are intentionally excluded.

Impact: This creates Low-severity risk-management friction. A solvent unhealthy holder may be unable to reduce exposure gradually and may need collateral or a larger exact-fill swap. No direct fund loss or incremental bad debt is demonstrated, and self-liquidation can be used when its health-improvement and margin checks pass.

## — RECOMMENDATION

For exposure-reducing adjustments, allow the trade when post-trade health strictly improves, or apply the absolute liquidation-ratio floor only when exposure increases. Retain the NegativeEquity guard on voluntary full closes.

## — RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

# tokenURI returns metadata for nonexistent and burned position NFTs

|          |                                                                                                                                                                                                                                                                               |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Unexpected Behavior                                                                                                                                                                                                                                                           |
| LOCATION | <a href="https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/Perp.sol#L202-L204">https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/Perp.sol#L202-L204</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                                                                                          |

## DESCRIPTION

Perp.tokenURI ignores tokenId and returns the collection-level URI without checking that the position NFT currently exists. It therefore succeeds for never-minted identifiers and for positions whose NFTs were burned on closure, contrary to ERC-721 metadata's invalid-token behavior.

Impact: Wallets and indexers that use tokenURI success as an NFT-existence signal can display phantom or already-closed positions. The issue does not affect ownership checks or directly move funds.

## RECOMMENDATION

Check that tokenId has a nonzero owner and revert for never-minted or burned tokens before returning the collection URI.

## RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

## Six-decimal utilization flooring slightly underprices utilization fees

|          |                                                                                                                                                                                                                                                                                                             |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Rounding                                                                                                                                                                                                                                                                                                    |
| LOCATION | <a href="https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/libraries/PerpLogic.sol#L480-L483">https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/libraries/PerpLogic.sol#L480-L483</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                                                                                                                        |

### DESCRIPTION

refreshRatesAndEmas() floors OI/capacity to an E6-scaled utilization before the Fees module converts it to WAD and evaluates the curve. Consequently, every non-exact ratio is evaluated at the lower six-decimal utilization bucket, and positive utilization below one ppm is evaluated as zero.

Impact: Takers pay slightly less utilization income than a full-precision curve would charge, and makers receive correspondingly less. The updated live-market PoC shows a positive but sub-dollar difference over seven days. Under ordinary shipped parameters, the discarded tail is below one ppm of utilization and is economically negligible.

### RECOMMENDATION

Document E6 flooring as an intentional precision bound or pass a higher-precision utilization value to the fee calculation.

### RESOLUTION

Status: Resolved.

LOW **L-03**

ACKNOWLEDGED

## Swap-Fee Limit Excludes the Protocol Fee

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/68093f9c48dc05dc917ed94cc458f06893579a6f/src/modules/Fees.sol#L212>

REVIEW Remediation Review 2

### — DESCRIPTION

The `Fees` module ensures that creator, insurance, and maximum LP fees cannot exceed 100% of swap volume.

However, the protocol fee is configured independently through `ProtocolFeeManager` and is added during swap execution without being included in this validation.

Consequently, individually valid configurations can exceed 100% in aggregate.

The existing fuzz test has the same blind spot: despite asserting that the "total swap fee" does not exceed 100%, it only sums creator, insurance, and LP fees.

### — RECOMMENDATION

Consider to enforce the aggregate bound across all fee sources.

### — RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

# Unnormalized Composite Weights

|          |                                                                                                                                                                                                                                                                                                                                           |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Validation                                                                                                                                                                                                                                                                                                                                |
| LOCATION | <a href="https://github.com/GuardianOrg/beacons-team1-1779719143808/blob/01570f5ff0f33c086dea0b22d94d843d79275ad2/src/core/composite/composers/WeightedSum.sol#L20-L23">https://github.com/GuardianOrg/beacons-team1-1779719143808/blob/01570f5ff0f33c086dea0b22d94d843d79275ad2/src/core/composite/composers/WeightedSum.sol#L20-L23</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                                                                                                                                                      |

## DESCRIPTION

`WeightedSum` does not require its weights to sum to `WAD`. A configuration mistake can therefore scale the composite index incorrectly and affect downstream Perp pricing.

## RECOMMENDATION

Consider to validate that weights sum up to `WAD`, or clearly document that non-normalized weighted sums are intentionally supported.

## RESOLUTION

Status: Acknowledged.

LOW **L-06**

RESOLVED

# Timelock defaults to zero, allowing immediate admin changes

**CATEGORY** Configuration**LOCATION** <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/4bbe554fa5f5a1aed26190c9f1374f2f3b65655c/src/Perp.sol#L77-L78>**REVIEW** Remediation Review 2

## DESCRIPTION

`Perp.timelock` defaults to `0`. The owner can call `submit(data)` and then execute the timelocked function immediately because `executableAt[data] == block.timestamp`.

This affects risk-critical admin actions such as replacing pricing, funding, fees, margin ratio, price impact, and beacon modules.

## RECOMMENDATION

Set a non-zero initial timelock in the constructor or factory.

## RESOLUTION

Status: Resolved.

LOW **L-07**

ACKNOWLEDGED

## V4 rounding dust can block final maker exit without opposite capacity

CATEGORY DoS  
LOCATION —  
REVIEW Remediation Review 2

### — DESCRIPTION

Uniswap V4 amount0 deltas round up on liquidity add and down on removal, so an add/remove cycle can leave a 1-atom residual perp balance. On final maker removal, PerpLogic removes the maker capacity and then converts any nonzero residual amount0 into taker open interest; with no opposite capacity, updateOpenInterest reverts.

Impact: A maker's voluntary full exit, and final liquidation in the same residual path, can be temporarily blocked with margin/liquidity left in the NFT. The effect is dust-scale and permissionlessly remediable by adding small opposite-side capacity, so impact is liveness friction rather than material loss.

### — RECOMMENDATION

Treat de minimis rounding residuals as zero or absorb them into margin/accounting on final maker close, or defer/adjust capacity removal so dust conversion cannot fail solely because the removed maker was the last capacity.

### — RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

# Module Setters Can Install Unusable Addresses

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/b37469ec415c56beb75ab67497920fa3963008af/src/Perp.sol#L304-L317>

REVIEW Remediation Review 2

## DESCRIPTION

`setMarginRatiosModule()` and `setPriceImpactModule()` store the replacement module without validating that it is nonzero, contains code, or implements the expected interface. An accidental zero address, EOA, or incompatible contract can therefore pass the timelock and later cause dependent operations—including position openings, swaps, exits, and liquidations—to revert. This is an update-path variant of L-06, which covers arbitrary modules supplied during market creation.

## RECOMMENDATION

Reject zero addresses and addresses without deployed code. Additionally, validate compatibility by calling representative interface functions and checking returned values before storing the replacement module.

## RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

# Stale timelocked beacon migrations let third parties liquidate healthy positions

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/Perp.sol#L237-L244>

REVIEW Remediation Review 2

## DESCRIPTION

A submitted timelocked action has a minimum execution time but no deadline, and any caller may consume it once mature. Consequently, a `setBeacon` action remains permissionlessly executable indefinitely. Because `setBeacon` samples the candidate beacon's live index at execution while carrying forward the old index EMA, a migration submitted while two feeds are aligned can cause an abrupt mark change if a third party executes it after the candidate later diverges.

## RECOMMENDATION

Give queued actions a bounded execution window after maturity, after which a fresh owner submission is required. For beacon changes, additionally consider recording the candidate index at submission and rejecting execution after material deviation, or restrict execution to an authorized governance executor.

## RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

# Per-maker capacity remainders understate global capacity and inflate utilization fees

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

Global capacity sums each maker's floored whole-atom capacity while X64 remainders remain isolated per position. Splitting the same in-range V4 liquidity among many makers can therefore record less capacity than a consolidated position, and utilization rates are calculated from this understated denominator.

Impact: A maker can fragment liquidity to push existing OI into a higher utilization-fee tier and collect elevated payments despite unchanged aggregate V4 liquidity. The PoC shows capacity of 20 rather than 23 atoms and 869 rather than 577 USDC atoms charged over seven days. Since each position suppresses less than one capacity atom and requires margin and gas, practical losses are modest.

## — RECOMMENDATION

Aggregate fractional capacity before flooring at the global level, while preserving correct attribution and removal accounting, so equivalent V4 liquidity records equivalent capacity regardless of position fragmentation.

## — RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

# Swap fee ordering diverts protocol and creator fees into insurance

CATEGORY Rewards  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

`accrueSwapFees()` first socializes protocol and creator fees against existing `badDebt`, then applies the same swap's insurance fee through `creditInsuranceInternal()`. If the insurance component alone would have cleared the residual debt, the current ordering still haircuts protocol/creator fees first and then leaves the equivalent excess in `feeFund.insurance`.

Impact: During impaired periods, swap fee revenue can be systematically reallocated from protocol and market-creator claims into the insurance reserve. This does not demonstrate trader fund theft, so Low severity is appropriate, but it is a real fee-accounting priority inversion.

## — RECOMMENDATION

Apply the insurance fee to `badDebt` before socializing protocol/creator fee claims, or implement a single explicit swap-fee insolvency waterfall that preserves the intended priority.

## — RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

# High liquidity permits stationary zero-quote exact-input shorts

CATEGORY Validation  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

On a sufficiently low-priced market with high active liquidity, V4 can consume a nonzero exact-input currency0 amount while Q64.96 rounding leaves both sqrt price and currency1 output unchanged. Perp accepts the exact amount0 fill, burns matching ERC-6909 claims, and records matching short OI and position liability. A full close restores the claims; this is a stationary precision variant, not a monotonic inventory drain or accounting mismatch.

Impact: A trader can create atom-scale short exposure and consume corresponding short capacity without an observable AMM price or quote response. The trader receives no quote proceeds, retains the real signed liability, and a close reverses the claim change. No profit, third-party loss, undercollateralization, or practical claim exhaustion is demonstrated, so impact is limited to low-level precision behavior.

## — RECOMMENDATION

Reject nonzero exact-input swaps when both sqrt price and quote output remain unchanged, impose a minimum economically meaningful quote/output threshold, or document this sub-atom behavior.

## — RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

# Long swaps undercharge LP fees due to floor-vs-ceil replay rounding

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

For one-for-zero exact-output swaps, V4 rounds each segment's token1 input up, while PerpTickLogic reconstructs the same segment's quote volume with a floor-rounded liquidity amount. Consequently the LP-fee base can be lower than the quote actually debited by PoolManager, reducing both lpFeeAmt charged to the taker and fee growth credited to makers.

Impact: Long takers retain dust LP fees and makers receive correspondingly less. The passing PoC shows a 12-atom quote-volume mismatch and 3-atom LP-fee shortfall over 12 opens at an amplifying 50% LP fee. The discrepancy can accumulate but is economically negligible per trade, supporting Low severity.

## RECOMMENDATION

Match V4's direction-specific amount1 rounding when replaying each segment, or reconcile the replayed segment volumes to the authoritative executed quote delta while preserving range attribution.

## RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

## Forced partial maker liquidations discard LP and utilization fee remainders

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

### — DESCRIPTION

Maker LP and utilization earnings are floored when growth is multiplied by liquidity or capacity, after which liquidation advances each fee-growth checkpoint without carrying the discarded numerator remainder. A permissionless partial liquidation can therefore force settlement at an unfavorable fractional boundary. The corrected split-versus-deferred control holds liquidity and subsequent fee activity constant and proves that two checkpoint settlements pay one atom less than a single deferred settlement when the two discarded remainders together cross the divisor.

Impact: A liquidator can make a distressed maker forfeit sub-atom LP and utilization entitlements on each partial liquidation. Across settlement windows those discarded fractions can accumulate to whole 6-decimal accounting-token atoms; the PoC isolates a one-atom LP-fee difference. The loss is repeatable but economically negligible per operation and constrained by liquidation eligibility and gas, supporting Low severity.

### — RECOMMENDATION

Carry per-maker LP and utilization fee numerator remainders across checkpoint updates, including when liquidity or capacity changes. Continue advancing checkpoints during liquidation, but preserve the fractional remainder similarly to the existing capacity remainder accounting.

### — RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

# Chunked maker liquidations reduce fees through repeated flooring

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

Each partial maker liquidation computes the reward by flooring both the removed-liquidity share of current position value and the configured fee applied to that share. No fractional remainder is carried between calls, so splitting the same aggregate liquidity removal can produce a smaller total reward than one removal.

Impact: The PoC removes 1% of liquidity in 40 calls and pays 11,560 USDC atoms rather than the single-call 11,579 atoms, a 19-atom (\$0.000019) difference at roughly 16.5 million gas. This makes liquidation reward accounting partition-dependent, but the loss is dust-scale and a self-liquidating maker can already nominate itself as recipient.

## — RECOMMENDATION

Carry liquidation-fee numerator remainders across partial liquidations, or derive cumulative fees from an aggregate removed-liquidity basis so equivalent total removals charge the same amount.

## — RESOLUTION

Status: Acknowledged.

LOW **L-16**

ACKNOWLEDGED

# Partial liquidation can turn a pending full close into an opposite-side position

CATEGORY Unexpected Behavior  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

adjustTaker applies a relative perpDelta to execution-time exposure without an expected-position nonce, baseline, target exposure, or reduce-only constraint. A permissionless partial liquidation can reduce a distressed long before its pending fixed-size full close, causing that close to trade through zero and leave an unintended short.

Impact: The PoC demonstrates that a legitimate 1% partial liquidation followed by the stale close leaves a short equal to the liquidated amount, retains the victim's collateral in the NFT, and pays no close proceeds to the holder. The effect is bounded transaction-state slippage on an already-liquidatable position rather than insolvency or theft.

## — RECOMMENDATION

Allow taker adjustments to commit to expected current exposure or desired final exposure, add a position-state nonce/deadline, or provide reduce-only/close-all semantics that cannot invert the position after intervening liquidation.

## — RESOLUTION

Status: Acknowledged.

LOW **L-17**

ACKNOWLEDGED

## Utilization-curve updates require each market to refresh its cached rates

CATEGORY Rewards  
LOCATION —  
REVIEW Remediation Review 2

### — DESCRIPTION

setUtilFeeCurve updates the shared fee model, but each Perp continues applying its previously cached utilization rates to the whole uncheckpointed interval. The new curve takes effect for a market only after touch or another write path accrues the old interval and refreshes its rates. This is consistent with the protocol's documented lazy-accrual semantics rather than an authorization or accounting invariant violation.

Impact: Operators expecting a global curve reduction to propagate automatically may leave dormant markets charging the old cached rate. Extended inactivity can materially affect position health, although any user or keeper can permissionlessly call touch immediately after the public update and no adversary can block that mitigation.

### — RECOMMENDATION

Document that shared fee-model updates require an immediate touch on every affected market, and have governance automation or keepers checkpoint those markets when changing the curve.

### — RESOLUTION

Status: Acknowledged.

LOW **L-18**

ACKNOWLEDGED

# TWAP Accumulator Can Overflow

CATEGORY Math  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

TWAP observations store cumulative value in `uint216`, while beacon indices use `uint256`. A value and elapsed-time pair accepted by the beacon can exceed the cumulative storage domain:

```
cumulativeValue + currentValue * elapsedTime > type(uint216).max
```

The next observation write reverts, preventing an otherwise valid oracle update.

## RECOMMENDATION

Enforce a maximum index based on the maximum observation interval, use a wider cumulative type, or define overflow-safe modular cumulative arithmetic with formally bounded deltas.

## RESOLUTION

Status: Acknowledged.

LOW **L-19**

ACKNOWLEDGED

# CGBM historical variance permits abrupt moves on confidence-regime changes

CATEGORY Math  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

CGBM derives sigma from the historical squared-magnitude estimator but multiplies the resulting return by the current report magnitude without independently clipping the standardized innovation. After an authenticated, balanced sequence in a low-confidence magnitude regime, a legitimate maximum-confidence report can therefore be much larger than preceding updates. The updated PoC uses ECDSAVerifier and a single signed WAD report and demonstrates a greater-than-5% bounded-index move.

Impact: This is primarily model, configuration, and documentation risk rather than an attacker-controlled oracle manipulation. A normal signer confidence-regime transition can abruptly affect a dependent Perp's shipped mark calculation and materially reduce an exposed position's unrealized equity. The PoC does not demonstrate liquidation, realized loss, or fund theft, so Low severity is appropriate.

## — RECOMMENDATION

Consider clipping each report's standardized innovation or bounding  $m \cdot \sigma$  independently of the transform-wide  $z$  domain. Otherwise, qualify the second-moment calibration documentation for magnitude-regime changes and provide deployment/signing guidance on confidence granularity, cadence, and safe CGBM parameters.

## — RESOLUTION

Status: Acknowledged.

LOW **L-20**

ACKNOWLEDGED

# TernaryToBinary floor rounding directionally biases CGBM and worsens dust-sequence jumps

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

TernaryToBinary floor-divides  $\text{pos} / (\text{pos} + \text{neg})$ . Complementary near-balanced ternary reports can quantize to HALF\_WAD versus HALF\_WAD-1 before CGBM runs. CGBM treats HALF\_WAD as exact neutral and returns without mutating accumulators, while HALF\_WAD-1 enters the full negative decay/update path. Repeated mirror pairs therefore discard the slight-positive evidence while directionally aging and updating the estimator through the mirrored negative evidence. This differs from the prior variant's internal pHat complementarity deficit, which occurs only after a sample has already entered CGBM's stateful path.

Impact: Honest normalized classifier outputs near an integer tie can systematically discard weak positive evidence while retaining mirrored negative evidence. In the PoC, 60 mirror pairs followed by a maximum-positive report produce an approximately 22,026x Unbounded jump versus approximately 786x for a symmetric HALF\_WAD $\pm$ 1 control. Generic documented CGBM dust/MIN\_VARIANCE sensitivity is the dominant co-factor in both paths, but TernaryToBinary flooring creates the directional branch asymmetry and worsens the jump by roughly 28x. The exact long report sequence and absence of permissionless control limit severity to Low.

## RECOMMENDATION

Use symmetric nearest rounding, or explicitly derive complementary classifications, so label-swapped reports map to complementary CGBM predictions of equal magnitude. Also add a neutral tolerance/rejection band near HALF\_WAD in the preprocessor and/or CGBM; symmetric rounding alone does not prevent generic dust-sequence amplification.

## RESOLUTION

Status: Acknowledged.

LOW **L-21**

ACKNOWLEDGED

# CGBM over-corrects weak positive reports and lowers Unbounded prices

CATEGORY Math  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

CGBM computes a history-based second-order drift correction that does not scale with the current report magnitude. After an ordinary one-sided history, a weak positive report can have a directional return smaller than this approximate correction even though the exact same-magnitude two-branch log-MGF correction is smaller. StandaloneBeacon subtracts the oversized correction, causing the authenticated positive report to lower an Unbounded index.

Impact: The production-signed PoC shows both the beacon index and a dependent Perp's cached mark falling after a weak-positive report. This is a concrete oracle-model distortion, but no liquidation, realized loss, bad debt, or attacker profit is demonstrated; report content and cadence remain under the trusted signer assumption.

## — RECOMMENDATION

Use an exact, numerically stable correction under CGBM's stated conditional branch model, or otherwise scale the approximation with current magnitude. Document any remaining lack of same-sign monotonicity and add signed regression tests for weak reports after varied histories.

## — RESOLUTION

Status: Acknowledged.

LOW **L-22**

ACKNOWLEDGED

# CGBM stationary low-confidence regime decouples oracle moves from classifier confidence

CATEGORY Math  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

CGBM intentionally normalizes estimated conditional variance through  $\sigma = \text{SCALED\_SIGMA\_BASE} / \sqrt{\text{varianceFactor}}$ . In a stationary balanced regime of fixed low-confidence predictions, variance converges proportionally to the squared transformed magnitude, so inverse scaling largely cancels that magnitude and weak reports retain configured per-report volatility. With Unbounded, a long alternating signed sequence plus per-update drift correction can substantially lower the realized index path and distort a dependent Perp's mark.

Impact: An honestly reporting but persistently uncertain classifier, especially when sequential signed reports are batch-relayed, can produce oracle volatility disproportionate to apparent confidence. The PoC demonstrates a below-1%-of-initial Unbounded index and material unrealized Perp mark/equity distortion after 1,200 reports, but no liquidation, realized loss, fund transfer, bad debt, or attacker profit. The explicit per-report normalization and documented cadence sensitivity limit this to a Low-severity calibration risk.

## — RECOMMENDATION

Document that stationary confidence magnitude can be canceled by variance normalization and provide deployment/signing guidance for minimum confidence, report cadence, and batch limits. If undesirable, cap the standardized innovation or sigma, and prefer Bounded transforms for persistently near-neutral classifiers.

## — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-01

RESOLVED

# Backstop deposits above int128.max contradict the uint128 API

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/libraries/PerpLiquidationLogic.sol#L297>

REVIEW Remediation Review 2

## DESCRIPTION

backstopTaker() and backstopMaker() accept uint128 marginIn and apply it to position margin via marginIn.toInt256(), but the collateral pull narrows it with marginIn.toInt128(). Values above type(int128).max therefore revert despite fitting the public parameter and preceding position arithmetic.

Impact: This has no realistic production liveness impact: int128.max raw 6-decimal units is about 1.7e32 USDC, vastly beyond any plausible collateral supply or recapitalization need. It is an API/type-consistency defect only.

## RECOMMENDATION

Use consistent types end-to-end, or explicitly enforce and document marginIn <= type(int128).max at the external interface.

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-02

RESOLVED

# setSwapFees emits an unchanged LP-configuration event

|          |                                                                                                                                                                                                                                                                                               |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Events                                                                                                                                                                                                                                                                                        |
| LOCATION | <a href="https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/modules/Fees.sol#L131-L133">https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/modules/Fees.sol#L131-L133</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                                                                                                          |

## DESCRIPTION

setSwapFees routes through the combined \_setSwapFees helper, which reassigns the unchanged LP configuration and emits LpFeeConfigSet even though only creator and insurance fees changed.

Impact: Indexers or monitoring keyed to LpFeeConfigSet can misclassify a creator/insurance fee update as an LP-curve update. The issue has no direct on-chain value impact.

## RECOMMENDATION

Have setSwapFees update and emit only creator/insurance fee state, or emit LpFeeConfigSet only when the LP configuration actually changes.

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-03

RESOLVED

# Self-recipient backstops clear token-specific ERC-721 approvals

|          |                                                                                                                                                                                                                                                                               |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Informational                                                                                                                                                                                                                                                                 |
| LOCATION | <a href="https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/Perp.sol#L367-L371">https://github.com/GuardianAudits/perpcity-contracts-helix/blob/618f44a84e9a4ed2760739061cba5a5010fc6467/src/Perp.sol#L367-L371</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                                                                                          |

## DESCRIPTION

The permissionless backstop entry points unconditionally invoke ERC-721 `_transfer` after accepting the caller's margin deposit, even when `positionRecipient` is the position's current owner. Solady treats this as a self-transfer and clears the token-specific approval while leaving ownership unchanged. The PoC explicitly demonstrates this behavior on both maker and taker positions and confirms that the formerly approved keeper can no longer adjust the position.

Impact: A third party willing to fund a backstop can preserve the distressed owner's ownership yet revoke that owner's token-specific keeper approval, disrupting automated management until reapproval. The grief is costly because the caller must supply sufficient backstop margin, the position is simultaneously recapitalized, and approval-for-all operators are unaffected, so Low severity is appropriate.

## RECOMMENDATION

If `positionRecipient` equals the current owner, skip the ERC-721 transfer; alternatively reject self-recipient backstops.

## RESOLUTION

Status: Resolved.

INFORMATIONAL **I-04**

ACKNOWLEDGED

# Liquidation Fee Can Be Set to 100%

CATEGORY Trust Assumptions

LOCATION <https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/68093f9c48dc05dc917ed94cc458f06893579a6f/src/modules/Fees.sol#L235>

REVIEW Remediation Review 2

## DESCRIPTION

`Fees._setLiqFee()` permits `liqFee == 1e6` (100%).

## RECOMMENDATION

Consider enforcing a lower safe liquidation-fee cap.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-05

ACKNOWLEDGED

# Reorg can capture GroupManager component bindings through the shared factory CREATE nonce

|          |                                                                                                                                                                                                                                                                                                                           |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Informational                                                                                                                                                                                                                                                                                                             |
| LOCATION | <a href="https://github.com/GuardianAudits/perpcity-beacons-helix/blob/01570f5ff0f33c086dea0b22d94d843d79275ad2/src/core/group/GroupManagerFactory.sol#L18-L25">https://github.com/GuardianAudits/perpcity-beacons-helix/blob/01570f5ff0f33c086dea0b22d94d843d79275ad2/src/core/group/GroupManagerFactory.sol#L18-L25</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                                                                                                                                      |

## DESCRIPTION

GroupManagerFactory deploys permissionlessly with a shared CREATE nonce, while generic CallerBound components must be bound in later transactions. If the deployment and binding history is reorganized, an attacker can insert a manager creation at the address the honest binder already encoded in bind(), displacing the honest manager while receiving the verifier and group-function authorizations. The attacker-controlled manager can use a malicious transform and matching member count while retaining the expected manager/member addresses.

Impact: Authentic next-nonce reports are processed by an attacker-selected transform, allowing arbitrary family-index publication, oracle-family denial of service through TWAP-toxic bootstrap values, and consequent mispricing or liveness failures in dependent perp markets.

Reorgs on Arbitrum are unrealistic, therefore this is more of a best practice advice and a warning for other chains.

## RECOMMENDATION

Deploy the manager and bind all CallerBound components atomically, or use a caller/configuration-committed CREATE2 address and verify the deployed configuration before binding.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

# Aggregate capacity can wrap at uint128 boundary

|          |                                                                                                                                                                                                                                                                                                                         |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Warning                                                                                                                                                                                                                                                                                                                 |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/68093f9c48dc05dc917ed94cc458f06893579a6f/src/libraries/PerpLogic.sol#L963">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/68093f9c48dc05dc917ed94cc458f06893579a6f/src/libraries/PerpLogic.sol#L963</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                                                                                                                                    |

## DESCRIPTION

`updateCapacity` performs unchecked additions and subtractions.

Consequently, individually valid maker-capacity increments can exceed the aggregate uint128 limit and wrap to a smaller value, corrupting utilization, rate, and admission calculations in theory.

In practice this should not happen as long as perp city sticks to 6 decimal USDC precision.

## RECOMMENDATION

Be aware and consider to document this behavior.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

# Bundled group reports create live-spot moves absent from wall-clock TWAP

CATEGORY Informational  
LOCATION GLOBAL  
REVIEW Remediation Review 2

## — DESCRIPTION

Group functions evolve once per authenticated report, while MemberBeacon's wall-clock TWAP writes at most once per timestamp. A relayer can therefore submit a sequential signed backlog in one block, immediately moving the live index through several report-driven transitions; intermediate values have zero elapsed duration and correctly contribute no weight to twAvg. The PoC uses a properly scaled group, real sequential signatures, expanded cardinality, and shows that bundled and hourly-spaced reports reach similar final spots but produce different wall-clock TWAPs.

Impact: Integrators that incorrectly expect twAvg to preserve report-by-report excursions may miss same-block live-index movements. Perp intentionally consumes the live index and the PoC shows a mark move, but no liquidation, bad debt, attacker profit, or direct fund loss.

## — RECOMMENDATION

Document clearly that group recurrence is report-count based, relayers control publication timing, and twAvg is wall-clock rather than report-weighted. Consumers needing report-path history or freshness controls should monitor update events/live spot, enforce their own rate or freshness policy, or use a separate report-count metric.

## — RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

ACKNOWLEDGED

# TwAvg comment incorrectly describes timestamp deduplication as per-block

CATEGORY Informational  
LOCATION GLOBAL  
REVIEW Remediation Review 2

## — DESCRIPTION

TwAvg.write() is documented as writing at most once per block, but its equality guard deduplicates observations by block.timestamp. Multiple L2 blocks can share a timestamp, so the comment does not accurately describe implementation behavior. The implementation remains mathematically consistent with a seconds-based TWAP.

Impact: Integrators reading only the source comment may expect distinct same-timestamp blocks to create distinct history entries. There is no demonstrated fund loss or TWAP arithmetic error: an intermediate value installed and removed at the same timestamp has zero time weight, Perp does not consume twAvg(), and update events preserve the sequence.

## — RECOMMENDATION

Change the comment to state that observations are written at most once per timestamp/second, and document that zero-duration same-timestamp values receive no TWAP weight on chains where multiple blocks share a timestamp.

## — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-09

ACKNOWLEDGED

# Zero Price-Impact Bound Freezes Swaps

|          |                                                                                                                                                                                                                                                                                                                               |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Validation                                                                                                                                                                                                                                                                                                                    |
| LOCATION | <a href="https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/b37469ec415c56beb75ab67497920fa3963008af/src/modules/PriceImpact.sol#L47-L49">https://github.com/GuardianOrg/perpcity-contracts-team1-1779718822807/blob/b37469ec415c56beb75ab67497920fa3963008af/src/modules/PriceImpact.sol#L47-L49</a> |
| REVIEW   | Remediation Review 2                                                                                                                                                                                                                                                                                                          |

## DESCRIPTION

`PriceImpact` enforces a maximum bound but accepts zero. A zero bound requires every swap to finish exactly at the EMA price, effectively blocking normal trades, taker exits, and liquidations until the configuration is corrected.

## RECOMMENDATION

Consider enforcing a sensible nonzero minimum in `_setPriceImpactBound`, alongside the existing maximum.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

# Positive dust liquidity permits one-atom AMM spot displacement

|          |                      |
|----------|----------------------|
| CATEGORY | Informational        |
| LOCATION | —                    |
| REVIEW   | Remediation Review 2 |

## — DESCRIPTION

With only 128 active liquidity, V4 step rounding lets a one-atom exact-input short move the pool through most of the 10% PriceImpact band while returning zero quote. Perp accepts the exact token0 fill and resulting bounded spot even though active liquidity remains positive.

Impact: An attacker can persistently move the AMM by more than 6% and the reference mark by roughly half that amount at negligible accounting-token cost in an extremely thin market. The PoC proves only this bounded manipulation primitive, not realized third-party loss.

## — RECOMMENDATION

Require meaningful active liquidity/notional or nonzero quote for exposure-changing swaps, and reject price movement disproportionate to realized volume.

## — RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

ACKNOWLEDGED

# Unchecked uint80 accumulators can erase protocol and creator fee claims

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

FeeFund stores protocol and creator balances as uint80. `accrueSwapFees()` casts each new increment to uint80 but then adds it to the existing bucket inside an unchecked block, so the cumulative sum can wrap even though the increment itself fits. The latest swap still removes its live fee amount from `totalMargin`, while the wrapped accumulator no longer records the previously accrued claim.

Impact: Reaching the boundary would let one further accrual erase previously recorded protocol or creator fees and leave only the small wrapped remainder collectible, creating untracked USDC. The issue is theoretical at current scale because `uint80.max` is roughly  $1.2e18$  whole USDC, far beyond plausible token supply.

## — RECOMMENDATION

Perform the cumulative addition in checked wider arithmetic and cast the result, widen FeeFund fields, or otherwise reject accrual when a bucket would exceed `uint80.max`.

## — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-12

ACKNOWLEDGED

# Bitmap-word segmentation leaves irreducible exact-fill residual

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

Perp credits consolidated short capacity, while V4 independently floors exact-output amounts at every empty bitmap-word step. The passing PoC records 10 atoms of capacity, shows a full 10-atom close returning only 8 and reverting, then closes the executable 8 atoms and proves the remaining 2 atoms cannot be exact-filled. Unlike the separate per-maker X64 remainder issue, this overstates executable close capacity rather than understating global capacity.

Impact: A small rounding deficit can remain as irreducible micro-OI despite capacity accounting indicating backing. This is a residual position-liveness and accounting-consistency defect; the PoC does not demonstrate material bad debt or insurance depletion.

## RECOMMENDATION

Haircut capacity to conservatively deliverable exact-output across bitmap-word segments, or carry fractional swap-step remainders so aggregate output matches consolidated capacity.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-13

ACKNOWLEDGED

# Excessive permitted fees can block insolvent taker liquidations

CATEGORY Validation  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

The Fees module permits creator and insurance fees of up to 45% each, allowing non-LP fees to reach 90% of the closing volume.

During liquidation, these fees are accrued and removed from `totalMargin` before the closed position's bad debt is recognized. If the resulting fee claim exceeds the available `totalMargin`, the checked subtraction reverts. A deeply insolvent partial liquidation may similarly revert with `NegativeMargin`.

Consequently, an extreme fee configuration combined with severe market or collateral stress can prevent liquidation and delay bad-debt recognition. The position remains open, continues consuming open interest, and makers may withdraw before the loss is socialized.

This issue requires abnormally high fees and severe insolvency, so it is unlikely under ordinary fee configurations.

## RECOMMENDATION

Reduce the maximum permitted creator and insurance fee rates so their combined value cannot approach the full trade volume. Additionally, consider enforcing a conservative aggregate cap across all fees and ensuring liquidation fees cannot exceed the margin available to support them.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

ACKNOWLEDGED

# Zero backstop margin ratio permanently disables backstop rescue

CATEGORY Validation  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

MarginRatios accepts  $\text{backstop}=0$  under  $\text{init} > \text{liq} > \text{backstop}$ . Because `isHealthy` floors nonpositive health at zero, health is always at least a zero reference ratio. Positions snapshotting zero therefore never satisfy backstop eligibility, and both taker and maker rescue paths revert regardless of distress or top-up. This is distinct from prior execution-slippage and unsigned-margin-cast findings: its configuration-time zero-ratio root disables both sides in every state.

Impact: Markets configured with a zero backstop ratio silently disable the optional NFT-transfer rescue path while ordinary liquidation remains available. No permissionless loss is demonstrated, so this is a Low-severity trusted-configuration footgun.

## — RECOMMENDATION

Require backstop ratios to be positive if rescue must remain available. Otherwise explicitly document and surface zero as disabling backstopping.

## — RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-15**

ACKNOWLEDGED

# Unbounded symbol length enables metadata gas grieving

CATEGORY Gas Griefing  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

PerpFactory creation and Perp.setSymbol accept an unbounded ERC-721 symbol. A permissionless creator can therefore deploy factory-created markets with very large stored symbols, and the owner can later replace the symbol with similarly unbounded data.

Impact: Optional metadata consumers that batch-read factory markets may incur linear gas/return-data costs or need to skip the market. The revised PoC creates five distinct markets with 120KB symbols and measures more than 400K gas per read and 2M gas for the batch. The attacker bears the write cost, consumers can isolate or cap calls, and no trading, liquidation, solvency, or accounting path reads symbol, so this is informational hardening rather than protocol DoS.

## — RECOMMENDATION

Cap symbol length at creation and in setSymbol; consider analogous limits for name and tokenURI. Document that factory-created status does not imply safe metadata, and have integrations identify markets by address and beacon while isolating metadata responses.

## — RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-16**

ACKNOWLEDGED

# setSwapFees emits an unchanged LP-configuration event

|          |                      |
|----------|----------------------|
| CATEGORY | Events               |
| LOCATION | —                    |
| REVIEW   | Remediation Review 2 |

## — DESCRIPTION

setSwapFees routes through the combined \_setSwapFees helper, which reassigns the unchanged LP configuration and emits LpFeeConfigSet even though only creator and insurance fees changed.

Impact: Indexers or monitoring keyed to LpFeeConfigSet can misclassify a creator/insurance fee update as an LP-curve update. The issue has no direct on-chain value impact.

## — RECOMMENDATION

Have setSwapFees update and emit only creator/insurance fee state, or emit LpFeeConfigSet only when the LP configuration actually changes.

## — RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-17**

ACKNOWLEDGED

# The V4 donation callback is unreachable from production Perp flows

CATEGORY Superfluous Code  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

Perp.unlockCallback implements a DONATE action and PerpGuardHook enables beforeDonate, but every production PoolManager unlock initiated by Perp encodes only MODIFY\_LIQUIDITY or SWAP. Direct external donations are rejected because the hook authorizes only the registered Perp. Perp.donate is instead a USDC solvency contribution and does not invoke PoolManager.donate. Thus the V4 donation branch and hook permission are unused; LP fees are tracked by separate Perp-side fee-growth accounting.

Impact: There is no demonstrated fund loss or LP accounting failure. The deployment carries unused callback code and an unnecessary hook permission, while documentation describing V4 donation-based LP fee routing does not match the implemented call graph.

## — RECOMMENDATION

Remove the DONATE callback branch and beforeDonate permission if they are not planned, or connect the intended production fee flow to PoolManager.donate. Update documentation to describe the actual Perp-side LP fee-growth mechanism.

## — RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-18**

ACKNOWLEDGED

# Unchecked insurance credits can wrap the uint80 reserve balance

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

donate() and accrueSwapFees() add a safely cast uint80 increment to the existing uint80 insurance balance inside unchecked blocks. The cast bounds only the increment, so a sum above uint80.max wraps modulo  $2^{80}$ . This is not duplicated by the supplied prior findings: none concern FeeFund insurance accumulation, unchecked addition, or the donation/swap-fee credit paths.

Impact: At the type boundary, recorded insurance can collapse while the corresponding USDC remains in the contract, causing later losses to bypass part of the reserve and enter bad-debt socialization. This is theoretical: uint80.max is about  $1.2e18$  whole USDC, far beyond plausible supply or isolated-market inflows, and the PoC must manufacture the boundary state with vm.store.

## RECOMMENDATION

Perform the sum in uint256 and cast the result to uint80, or otherwise explicitly reject a sum above uint80.max, in both credit paths.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-19**

ACKNOWLEDGED

# Unchecked insurance credits can wrap the uint80 reserve balance

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

donate() and accrueSwapFees() add a safely cast uint80 increment to the existing uint80 insurance balance inside unchecked blocks. The cast bounds only the increment, so a sum above uint80.max wraps modulo  $2^{80}$ . This is not duplicated by the supplied prior findings: none concern FeeFund insurance accumulation, unchecked addition, or the donation/swap-fee credit paths.

Impact: At the type boundary, recorded insurance can collapse while the corresponding USDC remains in the contract, causing later losses to bypass part of the reserve and enter bad-debt socialization. This is theoretical: uint80.max is about  $1.2e18$  whole USDC, far beyond plausible supply or isolated-market inflows, and the PoC must manufacture the boundary state with vm.store.

## RECOMMENDATION

Perform the sum in uint256 and cast the result to uint80, or otherwise explicitly reject a sum above uint80.max, in both credit paths.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-20

ACKNOWLEDGED

# sFullMulDiv reverts on the representable int256 minimum

CATEGORY Math  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

sFullMulDiv casts the unsigned magnitude to int256 before restoring the sign. For  $\text{type}(\text{int256}).\text{min} * 1 / 1$ , the correct signed result is representable, but its intermediate magnitude  $2^{255}$  is not a positive int256 and the cast reverts.

Impact: No reachable production path was identified: current call sites use bounded position, price, and shipped-funding operands. This is an informational boundary-case correctness defect in the shared helper rather than a demonstrated protocol denial of service.

## — RECOMMENDATION

When a negative result has magnitude  $2^{255}$ , return  $\text{type}(\text{int256}).\text{min}$  directly; otherwise retain the existing checked cast and sign restoration. Add a unit test for this boundary.

## — RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-21**

ACKNOWLEDGED

# Utilization proration discards sub-atom maker earnings dust

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

Each lazy-accrual checkpoint adds the full utilization charge to the taker payment index but floors  $\text{feeAccrued} * \text{OI} / \text{capacity}$  before adding to the maker earnings index. The division remainder is not carried, so repeated checkpoints accumulate discarded fractional X96 maker-credit dust.

Impact: The discrepancy is economically negligible under realistic conditions. A twin-market PoC isolates the checkpoint effect and shows that 64 hourly checkpoints still discard less than one 6-decimal USDC atom; realizing even one atom at the tested capacity would require an astronomical number of transactions. This does not support the originally claimed Medium severity.

## RECOMMENDATION

Carry the utilization-proration remainder between checkpoints if exact conservation is desired, or document the bounded X96 rounding dust.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-22

ACKNOWLEDGED

## E6 health flooring rejects atom-scale maker liquidations within capacity slack

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

### — DESCRIPTION

Partial maker liquidation requires a strict increase in a health ratio floored to E6. When long OI is one atom below capacity, utilization permits only atom-scale liquidity removal. A positively collateralized, liquidatable maker's exact health can improve across that tiny removal while both E6 ratios remain equal, causing HealthNotImproved; any larger removal fails  $OI \leq \text{capacity}$ . The revised PoC isolates positive equity and finds a maximum permitted removal of about 384 out of 335,000,000 liquidity.

Impact: E6 quantization extends the known exact-utilization liquidation deferral into a one-atom slack window. In the PoC the entire additionally blocked window is only about 1.15 ppm of maker liquidity, roughly \$0.00018 of a \$158 position, so economically meaningful deleveraging is already blocked by the prior utilization issue rather than this precision loss.

### — RECOMMENDATION

Compare pre/post health at higher precision or by a safe cross-multiplied exact-ratio comparison. Preserve the  $OI \leq \text{capacity}$  check because removing maker liquidity does not reduce taker OI and relaxing it could break exact-fill unwind capacity.

### — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-23

ACKNOWLEDGED

# Post-socialization fee split assigns rounding residuals to creator

|          |                      |
|----------|----------------------|
| CATEGORY | Rounding             |
| LOCATION | —                    |
| REVIEW   | Remediation Review 2 |

## — DESCRIPTION

After protocol and creator fees are jointly socialized, the surviving amount is split by flooring the protocol's proportional share and assigning the entire remainder to creator fees. This deterministically favors the creator whenever the proportional protocol share is fractional. It is independent of the prior insurance-ordering issue: that finding changes the insolvency waterfall between insurance and non-insurance fees, whereas this bias occurs later when dividing the already-surviving non-insurance amount between protocol and creator.

Impact: The protocol can lose at most one raw 6-decimal USDC atom per fee split to the creator. Repeated fragmentation accumulates the bias, but gas and swap fees overwhelmingly exceed the dust, so there is no economically meaningful extraction demonstrated.

## — RECOMMENDATION

Use an explicit neutral remainder policy, assign the residual to protocol, or track fractional/remainder amounts for later reconciliation.

## — RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-24**

ACKNOWLEDGED

# Oversized maker margin bricks converted taker adjustments

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

adjustMaker can store a uint128 margin above `type(int128).max`. If the maker is fully removed with residual perp exposure, the converted taker inherits that margin, but adjustTaker narrows it through `toInt128` and reverts on every adjustment.

Impact: The affected NFT cannot be voluntarily adjusted or closed while its margin exceeds the signed ceiling. Reaching the condition requires more than roughly 1.7e32 USDC and the demonstrated deposit harms only its owner, so this is an impractical API/type-consistency defect rather than a realistic denial of service.

## RECOMMENDATION

Use int256 arithmetic for stored margin reads and payout handling, or consistently cap every margin write at `type(int128).max`.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-25

ACKNOWLEDGED

# Malformed UTF-8 market names can disrupt strict metadata consumers

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

Permissionless market creation and the owner-only metadata setter accept arbitrary byte sequences as Solidity strings. A market can therefore persist and emit a name that is not well-formed UTF-8. ABI decoding remains functional, but optional presentation or indexing layers that apply strict UTF-8 validation may reject that market's metadata.

Impact: Strict metadata consumers may omit the hostile market or fail a poorly isolated metadata batch. Core factory registration, trading, accounting, and other on-chain market operations remain unaffected, so this is integration hardening rather than a protocol denial of service.

## RECOMMENDATION

Validate UTF-8 in metadata inputs if well-formed text is intended, or explicitly document metadata as untrusted bytes and require integrations to decode defensively and isolate failures per market.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-26

ACKNOWLEDGED

# Clipped Bounded updates amplify CGBM reversal sensitivity

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

StandaloneBeacon invokes the stateful CGBM before clamping the resulting  $z$  value to Bounded's transform domain. Authenticated same-direction reports submitted while  $z$  is already at the ceiling therefore leave the published index effectively pinned but continue aging and updating CGBM's directional and variance estimators. The next authenticated opposite report is priced from the more one-sided estimator and can move farther than it would without the clipped reports. This is consistent with CGBM continuing to learn from signed observations, but is a noteworthy saturation behavior for integrations.

Impact: The production-verifier PoC shows 40 clipped majority reports amplify the next reversal relative to an unprimed control. The absolute demonstrated move is only about 0.0193% of the 0-100 bounded range (approximately 99.99999979 to 99.98070339), and no liquidation or realized loss is shown, so this is a Low-severity oracle calibration/integration caveat rather than a practical fund-loss exploit.

## RECOMMENDATION

Document that Bounded saturation freezes published  $z$  movement but does not freeze CGBM learning, and provide deployment guidance for boundary behavior. If this is not desired, redesign the base-function interface or update flow so accumulator commits follow an explicit clamp-aware saturation policy.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-27

ACKNOWLEDGED

# Reorg can substitute hostile DiscreteAllocation at a factory child address

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

DiscreteAllocationFactory uses nonce-based CREATE, so the resulting address commits only to the factory and its nonce, not the caller or constructor configuration. If an operator broadcasts dependent deployment transactions against an unfinalized child receipt and the chain reorganizes, an attacker can place a differently configured, constructor-valid DiscreteAllocation at the previously observed address before the honest factory call is replayed. GroupManager stores that address without attesting its immutable economics. The corrected PoC uses compatible Softmax seeds, real ECDSAVerifiers and binding, and identical signed reports on hostile and honest managers; the hostile zero-magnitude configuration consumes all reports while its member index stays frozen, whereas the honest index advances. A downstream Perp can consequently be deployed against the frozen member beacon.

Impact: During the narrow reorg window, an unsafe dependent-deployment workflow can silently bind a manager and downstream market to substituted allocation parameters. Signed reports can continue succeeding while the member oracle remains stale or follows different allocation economics. Exploitation requires pre-finality dependent transactions and failure to re-verify configuration, so this is a Low-severity deployment-hardening issue rather than an exploit against finalized markets.

## — RECOMMENDATION

Use CREATE2 with a salt committing to the caller and constructor arguments, and have GroupManager verify an expected configuration hash. Alternatively deploy and register the configuration atomically. Deployment operators should wait for finality and re-read child configuration before binding verifiers or launching dependent markets.

## — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-28

ACKNOWLEDGED

# Dust pseudocount can round RelativeDominance ratio to zero and revert updates

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

RelativeDominance first rounds  $\text{normFast} / \text{normSlow}$  with  $\text{divWad}$ , then multiplies the rounded result by  $\text{slowSum}$  before applying  $\text{lnWad}$ . With the constructor-accepted but economically extreme  $\text{ALPHA}=1$ , a positive observation followed by zero can round the first ratio to zero, causing  $\text{lnWad}(0)$  to revert the atomic GroupManager update and roll back its verifier nonce.

Impact: A group deployed with a dust pseudocount can reject otherwise valid sparse reports and temporarily delay its nonce sequence until the trusted signer supplies a non-triggering replacement report at the blocked nonce. The PoC does not establish persistent staleness or Perp impact, and realistic tested pseudocounts do not trigger the edge case.

## RECOMMENDATION

Compute  $\text{normFast} * \text{slowSum} / \text{normSlow}$  in one full-precision operation, floor the logarithm input to a positive minimum, or reject pseudocount configurations too small for the supported score range.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-29

ACKNOWLEDGED

# z-clamping breaks exact martingale claim at finite Unbounded bounds

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

DGBM computes a two-point log-MGF correction assuming both counterfactual returns are fully applied. StandaloneBeacon subsequently clamps the corrected  $z$  value to Unbounded's finite bounds. At the upper bound, positive corrected increments are clipped while negative increments remain reachable; the lower bound has the mirrored behavior. DGBM state still advances on clipped observations. Consequently the documented unconditional equality  $E[\exp(dz-c)|F]=1$  does not hold at saturation, although clamping is an intentional tradeoff to avoid hidden latent movement and update freezes.

Impact: A saturated beacon has directional bias and can move discontinuously on reversal relative to the advertised martingale behavior. Practical exploitation is limited: the upper bound is roughly 485 million times the transform reference, the PoC seeds that extreme state rather than reaching it through realistic signed updates, relayers cannot choose future reports, and no Perp loss or liquidation is demonstrated. This is chiefly a specification and integration caveat for extreme boundary states.

## RECOMMENDATION

Qualify the martingale guarantee for finite  $z$  bounds, or derive correction from the clamp-aware reachable branch returns. Document the intentional tradeoff between exact boundary martingality and immediate reversal responsiveness.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-30

ACKNOWLEDGED

# CGBM zero-history fallback zeroes drift correction on nonzero branches

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

When aged mTotal is zero, CGBM substitutes synthetic 50/50 branch probabilities while historical squared-magnitude state remains zero. rawVarianceFactor and driftCorrection therefore become exactly zero although MIN\_VARIANCE still produces amplified nonzero branches. The PoC reaches this boundary with constructor-valid decay=1 wei and shows a greater-than-10x authenticated jump; a 0.99 WAD control retains positive history and correction. This materially extends the prior ordinary nonzero-variance approximation issue with a distinct zero-history fallback trigger.

Impact: A near-memoryless, misconfigured CGBM+Unbounded beacon can publish extreme uncorrected exponential jumps on authenticated reports. This is a trusted BeaconDeployer configuration footgun, not a permissionless exploit under production-style decay.

## — RECOMMENDATION

At mTotal==0, derive correction from the synthetic equiprobable branch distribution, such as  $\ln(\cosh(dz))$ , or reject a nonzero update when historical variance is zero.

## — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-31

ACKNOWLEDGED

# DGBM+Bounded can bias indices under asymmetric high-volatility configurations

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

DGBM makes its two binary z-space branches first-moment neutral, while Bounded deliberately applies no drift correction before mapping z through a nonlinear sigmoid. With a low estimated positive rate and large configured sigma, the rare positive branch saturates more heavily than frequent small negative branches, so the expected published index can move below the midpoint despite a zero expected z increment. The effect is parameter-dependent and is negligible under the symmetric, low-sigma shipped example configuration.

Impact: A trusted deployer choosing an asymmetric, high-volatility DGBM+Bounded configuration can create directional oracle bias that downstream integrations may not anticipate. The PoC demonstrates the mathematical calibration issue but no permissionless fund extraction, and shipped example parameters avoid the material regime.

## — RECOMMENDATION

Document and enforce safe DGBM+Bounded parameter ranges, cap per-update z magnitude to keep the sigmoid locally linear, or introduce a transform-aware correction if index-level neutrality is required.

## — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-32

ACKNOWLEDGED

# CGBM+Unbounded martingale claim is not satisfied by the second-order correction

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

Unbounded and StandaloneBeacon describe correctsDrift as subtracting  $c_t = \ln(E[\exp(dz)|F])$  so the published exponential index is a martingale. CGBM instead exposes the explicitly documented second-order approximation  $0.5 \cdot \sigma^2 \cdot \text{rawVarianceFactor}$ . At a symmetric initial state with production-style parameters, that value differs from the same-magnitude two-branch log-MGF and leaves  $E[\exp(dz - c_t)]$  unequal to one. Thus the generic exact-martingale documentation does not hold for the permitted CGBM+Unbounded pairing.

Impact: Deployers may rely on a consumer-facing martingale guarantee that this pairing does not exactly satisfy. The residual bias can propagate into a Perp's beacon-dependent mark, funding, and health calculations. This is a specification and integration caveat rather than a demonstrated permissionless theft vector; its magnitude and direction depend on configuration and report distribution.

## — RECOMMENDATION

Either supply an exact correction under CGBM's stated stochastic model, restrict exact-martingale claims/pairings to base functions such as DGBM that provide one, or qualify Unbounded and StandaloneBeacon documentation to state that CGBM provides only a second-order approximation and recommend suitably small moves or Bounded.

## — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-33

ACKNOWLEDGED

# BinaryDGBM sample scripts miswire parameters and revert on nonce zero

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

Both BinaryDGBM simulation scripts pass `DECAY` as `scalingFactor` and `SCALING_FACTOR` as `decay`, producing `DECAY=WAD` and a 0.99-scaled sigma instead of the named configuration. They also sign their first report with nonce zero although `ECDSAVerifier` requires the first nonce to be one. This is wholly unrelated to the prior dust-range gas finding: it affects off-chain sample-script configuration in the beacon package, not maker ranges, Perp swaps, or gas consumption.

Impact: The scripts revert on their first synthetic update. If the nonce error alone is fixed, their logged index behavior still models materially different DGBM dynamics than their constants indicate. Because these scripts do not broadcast or deploy a persistent oracle, no live Perp fund impact is demonstrated.

## RECOMMENDATION

Pass `(SIGMA_BASE, SCALING_FACTOR, DECAY, INITIAL_POSITIVE_RATE, binder)` and sign reports with nonce `i + 1`. Add tests asserting the resulting DGBM immutables and successful script completion.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-34**

ACKNOWLEDGED

# Nested WeightedSum compositions are non-associative due to per-node flooring

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

Each WeightedSum floors its result before a parent CompositeBeacon consumes it, so a nested integer composition need not equal a one-pass basket using representable, flattened WAD weights. The corrected PoC demonstrates the arithmetic difference. At realistic Q96 price scales, however, the relative difference is negligible; part of the measured nested-versus-flat difference also comes from flooring the derived effective flat weights themselves.

Impact: No material Perp pricing or fund-loss impact is demonstrated. This is an operator/deployment precision consideration, principally relevant to very low-magnitude integer beacons or integrations that incorrectly assume exact associativity.

## RECOMMENDATION

Document that nested WeightedSum trees are not exactly associative in integer arithmetic. Flatten price-critical baskets at deployment when a single, well-defined rounding point is desired.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-35

ACKNOWLEDGED

# GMNormalize compression factor theoretically truncates to zero at unreachable dispersion

CATEGORY —

LOCATION —

REVIEW Remediation Review 2

## DESCRIPTION

GMNormalize calculates a WAD compression scale as  $\text{EXP\_OVERFLOW.sDivWad}(\text{maxAbs})$ . If centered dispersion exceeds  $\text{EXP\_OVERFLOW} \times \text{WAD}$ , the quotient floors to zero and all exponent inputs become zero, yielding  $\text{INDEX\_SCALE}$  for every member. The isolated arithmetic behavior is real, but DiscreteAllocation's capped increments require roughly  $1e21$  or more genuine one-sided reports to reach the threshold from mean-centered seeds, making it infeasible in production.

Impact: No practical repricing or fund-loss path is reachable through the supported accumulation process. Only an abstract execution lasting an infeasible number of authenticated reports would collapse a dispersed group to uniform outputs.

## RECOMMENDATION

Prevent a zero compression scale by enforcing a positive minimum, reverting above the representable threshold, or bounding/recentering accumulated latent dispersion.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-36

ACKNOWLEDGED

# DGBM quantizes sub-WAD branch probability to zero

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

DGBM derives the positive branch probability as `aUp.divWad(aUp + aDown)`. After a sufficiently long one-sided run, `aUp` can remain nonzero while this WAD probability floors to zero, so the drift-correction MGF treats the branch as impossible. This is a fixed-point modeling limit rather than a material martingale failure: under plausible BinaryDGBM parameters, the omitted contribution and resulting index difference are dust. Separately, `_driftCorrection` eagerly exponentiates both branch returns; an extremely large, incompatible sigma can exceed `expWad`'s domain even when the branch's quantized weight is zero.

Impact: No material oracle or Perp pricing deviation is demonstrated under plausible configuration. The revised PoC shows the rational and quantized corrections and reversal indices agree within dust. The eager-exponentiation behavior is only a trusted-deployer configuration footgun at extreme sigma, not a permissionless High-impact feed attack.

## RECOMMENDATION

For exactness, retain rational branch weights through MGF accumulation or use a numerically stable log-sum-exp calculation. Short-circuit truly zero-weight branches and document/enforce a safe `SCALED_SIGMA_BASE` bound.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-37

ACKNOWLEDGED

# Softmax allocation scaling incurs avoidable double rounding

CATEGORY Rounding  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

Softmax first floors each exponential ratio to WAD precision with  $\text{divWad}(\text{expSum})$ , then scales that rounded ratio to `INDEX_SCALE`. A single full-precision  $\text{mulDiv}$  by `INDEX_SCALE` would retain more precision. For three equal members with Q96 as `INDEX_SCALE`, the current outputs sum 79,228,162,516 Q96 units below the scale.

Impact: Published member indices contain an avoidable rounding bias of roughly  $1e-18$  relative to Q96 in the demonstrated equal-member case. This does not compound across updates, and no economically observable collateral, pricing, or funding impact was demonstrated; the documented simplex sum is only approximate.

## RECOMMENDATION

Compute each output in one full-precision step, such as  $\text{indices}[i].\text{mulDiv}(\text{INDEX\_SCALE}, \text{expSum})$ . If exact sum conservation is desired, explicitly assign the residual rounding remainder according to a documented rule.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-38

ACKNOWLEDGED

# Shipped Argmax preprocessor has no compatible multiclass base function

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

Argmax emits `largestIndex * WAD` and rejects class-zero winners. The shipped CGBM accepts only values up to WAD, while DGBM accepts only zero or WAD. Consequently class 2 or higher is incompatible with both base functions, and Argmax cannot supply DGBM's negative branch. This is a shipped-component interoperability defect; trusted beacon deployers remain responsible for compatible pipeline construction.

Impact: Argmax has no useful multiclass pairing among the currently shipped base functions. An incompatibly assembled beacon rejects normal class-2-or-higher reports, but no permissionless exploit or impact to a correctly configured beacon is demonstrated.

## RECOMMENDATION

Remove or clearly document Argmax until a compatible multiclass base function exists, add a multiclass-to-supported-domain mapping, or validate component compatibility during deployment.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-39

ACKNOWLEDGED

# Softmax member TWAPs can sum above INDEX\_SCALE at one timestamp

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## DESCRIPTION

Softmax conserves INDEX\_SCALE for each instantaneous GroupManager output, but every MemberBeacon has an independently expandable TwAvg ring. timeWeightedAvg() caps an overlong request to that member's oldest retained observation without reporting the effective window. After allocation rotates between members, same-block twAvg() calls using the same nominal lookback can therefore average different actual intervals and sum above INDEX\_SCALE, even though every live vector remained normalized.

Impact: An external integrator that sums or collateralizes multiple member TWAPs without ensuring identical effective histories can overstate aggregate group mass. No in-repo Perp path consumes twAvg(), and Softmax does not promise conservation across different effective averaging intervals, so this is an integration hazard rather than demonstrated protocol loss.

## RECOMMENDATION

Document that independently retained member TWAPs must not be treated as a synchronized group vector. Consider exposing a group-level synchronized TWAP, sharing cardinality/history policy across members, or returning the effective lookback timestamp so consumers can verify aligned windows.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-40**

ACKNOWLEDGED

# GMNormalize compression creates capped equal-unit basket convexity

CATEGORY Informational  
LOCATION —  
REVIEW Remediation Review 2

## — DESCRIPTION

For a two-member GMNormalize group, proportional compression fixes the centered exponent pair near the reciprocal safe-range boundary once dispersion exceeds EXP\_OVERFLOW. A supported no-decay DiscreteAllocation sequence can reach this boundary: further wins by the leading member then barely change the equal-unit arithmetic basket, while enough wins by the lagging member reconverge the pair and lower that basket. This is intentional geometric-mean-preserving transform behavior, not a Perp fund-extraction exploit.

Impact: Integrators may incorrectly infer that equal-unit arithmetic-basket convexity can be replicated with isolated member Perps. The revised PoC demonstrates the reachable oracle behavior but no realized USDC gain, maker loss, liquidation, insurance consumption, or bad debt; separate member shorts settle against their own AMMs and do not reproduce a native basket short.

## — RECOMMENDATION

Document the compression boundary and distinguish the group's geometric-mean invariant from equal-unit arithmetic-basket behavior. If native basket exposure is intended, provide an explicit composite instrument or weighting guidance rather than implying it is replicable through isolated member markets.

## — RESOLUTION

Status: Acknowledged.

## § 05

# Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

# About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

**\$45B+**DIGITAL ASSETS UNDER  
REVIEW**200+**

ENGAGEMENTS COMPLETED

**500+**CRITICAL & HIGH FINDINGS  
REPORTED

Reach us at [audits@guardianaudits.com](mailto:audits@guardianaudits.com).